

COPENHAGEN BUSINESS SCHOOL



BUSINESS ADMINISTRATION AND MATHEMATICAL BUSINESS
ECONOMICS

**Pricing of CDOs: A Comparative Study of the Gaussian
and Double t Copula Models**

MASTER'S THESIS

Emil Rosendal Madsen

S144844

Supervisor:

Jacob Lundbeck Serup

74 pages and 81424

characters including spaces

May 15, 2026

Resumé

I denne opgave undersøger vi, hvordan CDO trancher prissættes med den Gaussiske copula model og double t copula modellen.

Vi sammenligner korrelationssmilet og fallitfordelingerne i de to modeller. Vi undersøger, hvordan fair tranche spreads og upfronts varierer, når modelparametrene ændres, og vi diskuterer kompleksiteten, der tilføjes i double t copula modellen og dens implikationer på implementeringen af modellen i industrien.

Vi bygger den Gaussiske copula model og double t copula modellen under homogenitetsantagelsen ved at integrere de betingede fallitsandsynligheder over den systematiske faktor. For double t copula modellen anvender vi karakteristiske funktioner og adaptiv Gauss-Hermite kvadratur til at finde fordelingen af fallitvariablen — en detaljeret implementering, som, så vidt vi ved, ikke tidligere er dokumenteret i litteraturen.

Vi finder, at double t copula modellen har et fladere korrelationssmil med en spændvidde på 0.48 sammenlignet med 0.76 for den Gaussiske copula model, og at dens fallitfordeling er skubbet ud mod halerne. Vi finder, at de fair tranche spreads og upfronts er følsomme over for værdierne af modelparametrene i begge modeller, og at den ekstra kompleksitet i double t copula modellen kan stå i vejen for implementering i industrien.

Vi konkluderer, at double t copula modellen har et stærkere teoretisk fundament end den Gaussiske copula model, men at implementering i industrien kan være upraktisk grundet den øgede beregningstid og de modelspecifikke base korrelationer.

Abstract

In this thesis, we analyze how CDO tranches are priced using the Gaussian and double t copula models.

We compare the correlation smiles and default distributions produced by the two models.

We examine how fair tranche spreads and upfronts respond to changes in the model parameters, and we discuss the additional complexity introduced by the double t copula model and its implications for industry adoption.

We build the Gaussian and double t copula models under the homogeneity assumption by integrating the conditional default probabilities over the systematic factor. For the double t copula model, we recover the distribution of the default variable through characteristic functions and adaptive Gauss-Hermite quadrature — a detailed implementation that, to our knowledge, has not previously been documented in the literature.

We find that the double t copula model produces a flatter correlation smile, with a range of 0.48 compared to 0.76 for the Gaussian copula model, and that its default distribution is shifted towards the tails. We find that the fair tranche spreads and upfronts are sensitive to the values of the model parameters for both models, and that the added complexity of the double t copula model could hinder an industry-wide adoption.

We find that the double t copula model has a stronger theoretical foundation than the Gaussian copula model, but that industry implementation can be impractical because of increased computation time and model-specific base correlations.

Table of Contents

1	Introduction	3
2	Financial Instruments and Their Valuation	4
2.1	Zero-Coupon Bonds	4
2.2	Credit Default Swaps	5
2.3	Valuation of Credit Default Swaps	6
2.3.1	Discretized Hazard Curve Estimation Algorithm	9
2.4	Collateralized Debt Obligations	10
2.5	Valuation of Synthetic CDOs	11
2.6	Fair Spreads and Upfronts	12
3	Copula Models	13
3.1	Copula Functions	13
3.1.1	Gaussian Copula	14
3.1.2	t Copula	15
3.1.3	Copula Tail Dependence	15
3.2	Gaussian Copula Model	17
3.2.1	Gauss-Hermite Quadrature	21
3.2.2	Critique of the Gaussian Copula Model	23
3.3	Double t Copula Model	25
3.3.1	Adaptive Gauss-Hermite Quadrature	30
3.4	Implied Correlation	33
3.4.1	Compound Correlation	33
3.4.2	Base Correlation	34
4	Data for the Experiment	37
5	Numerical Results	43
5.1	Tail Dependence in the Models	44

5.2	Default Distributions of the Models	49
5.3	Compound Correlation	53
5.4	Base Correlation	56
5.5	Fair Spreads and Upfronts	59
5.5.1	Sensitivity of Spreads and Upfronts to Changes in ρ	61
6	Discussion	63
6.1	Homogeneous Versus Heterogeneous Portfolio	64
6.2	Sensitivity Analysis of Recovery Rate	65
6.3	Sensitivity Analysis of Degrees of Freedom	69
6.4	Increased Complexity Introduced by the Double t Copula Model	71
7	Conclusion	73
A	Appendix	77
A.1	Brent's Method	77
A.2	Inverse Fast Fourier Transform of Characteristic Functions	77
A.3	Convergence Check of n_{herm} , L , and N	79
A.4	Markit iTraxx Europe Index Bloomberg Description	83
A.5	Gaussian Copula Model Python Code	83
A.6	Double t Copula Model Python Code	88

1 Introduction

Modelling CDO tranche prices is challenging. In order to do so accurately, one needs to estimate the joint default probability of the entire underlying portfolio. This requires either extensive default data, which unfortunately is scarce, or an assumption on the dependence structure of the underlying portfolio. Li (2000) introduced a simple assumption on the dependence structure using a Gaussian copula. Combined with the one-factor model introduced by Vasicek (1987), the Gaussian copula model was quickly implemented as the standard model in the financial industry for pricing CDOs, though its limitations became apparent during the 2008 financial crisis. Hull and White (2004) introduced a framework for building the Gaussian copula model without using Monte Carlo techniques and also introduced the double t copula model.

The double t copula model captures tail risk more accurately than the Gaussian copula model and Hull and White (2004) illustrated that the double t copula with $\nu = 4$ fits market prices relatively well. However, the double t copula is harder to implement because a sum of t distributed variables is no longer t distributed. Hull and White (2004) solved this by using probability bucketing, which builds up the loss distribution discretely. In this paper, we implement the double t copula model using an alternative method suggested by Brigo, Pallavicini, and Torresetti (2010): we recover the CDF of a sum of t distributed variables through characteristic functions, then mirror the construction of the Gaussian copula model using adaptive Gauss-Hermite quadrature in place of standard Gauss-Hermite quadrature. Our principal methodological contribution is a detailed implementation of this approach. We believe this detailed implementation has not previously been documented in the literature.

The thesis addresses four questions. First, how does replacing the Gaussian distribution with a t distribution in the one-factor copula framework affect the correlation smile observed when pricing iTraxx Europe S42 CDO tranches? Second, how does tail dependence in the double t copula redistribute probability mass in the default distribution, and how does this affect the pricing of equity, mezzanine, and senior tranches differently? Third, how do the fair spreads and upfronts of each tranche respond to changes in the correlation ρ , recovery rate R , and (for the double t

copula model) the degrees of freedom ν ? Fourth, what additional complexity does the double t copula model introduce relative to the Gaussian copula model, and does this complexity constitute a practical barrier to adoption?

In the thesis writing process, artificial intelligence (AI) has been used for proof-reading, writing code, discussing ideas, and searching the literature.

The thesis is structured as follows. Section 2 presents the financial instruments involved in the modelling and their valuation. Section 3 introduces the Gaussian and double t copula models and the building blocks for constructing them. In Section 4, we present the market data we use to generate numerical results for the models in Section 5. Lastly, in Section 6, we discuss the assumptions made along the way and the additional complexity of the double t copula model before presenting our conclusions in Section 7.

2 Financial Instruments and Their Valuation

In this section, we define the financial instruments credit default swaps and collateralized debt obligations and their valuations as well as zero-coupon bonds that are necessary to value them.

2.1 Zero-Coupon Bonds

Zero-coupon bonds (ZCBs) are bonds that do not pay coupons during their lifetime. With a notional of 1, this means they only pay back 1 at maturity. If we assume a ZCB to be risk-free, it means that the price of a ZCB tells us how much the market is willing to pay now to receive a value of 1 at some point in the future. We assume ZCBs to be risk-free throughout the paper. This property makes them perfectly suited for discounting future cash flows to the present. For a deterministic continuous risk-free rate function $r(s)$, we define the time- t price of a ZCB with maturity T as

$$Z(t, T) = e^{-\int_t^T r(s) ds}.$$

Later, we will need to discretize the risk-free rate function, and we will therefore assume the function to be constant in certain intervals. Formally, for rates $r(T_1), r(T_2), \dots, r(T_m)$ and maturities $T_1, T_2, \dots, T_m$, we will assume that over the interval $t \in (T_{k-1}, T_k]$ the risk-free rate is $r(t) = r_k$ for $k = 1, 2, \dots, m$. The time-0 ZCB price with maturity T_1 is given by $Z(0, T_1) = e^{-r_1 T_1}$ and the time-0 ZCB price with maturity T_2 is given by $Z(0, T_2) = e^{-r_1 T_1 - r_2 (T_2 - T_1)}$. We generalize the time-0 ZCB price with maturity T_m to

$$Z(0, T_m) = \exp \left(- \sum_{k=1}^m r_k (T_k - T_{k-1}) \right)$$

with $T_0 = 0$. These ZCB prices are essential for the valuation of the cash flows in credit default swap and collateralized debt obligation contracts.

2.2 Credit Default Swaps

This section is largely based on Hull (2022).

Credit risk arises when borrowers or counterparties in a trade can default. If a counterparty defaults, the lender will often suffer a loss as a result of not receiving future payments. Credit risk is therefore something financial institutions must manage throughout their operations.

If a borrower defaults, the debt holders take control of the assets of the borrower and sell them off to cover their losses resulting from the default. In most cases, this means that the debt holders will suffer a loss in case of default of a borrower. For a loan with notional value 1, the loss following a default is $1 - R$, where R is the recovery rate. To mitigate some of the credit risk associated with a loan, a lender can enter into a credit default swap (CDS) agreement, which is effectively an insurance contract on the loan, though, unlike traditional insurance, the protection buyer is not required to hold the underlying debt. Upon occurrence of a credit event, the protection seller compensates the protection buyer for the loss of $1 - R$. Credit events include default, failure to pay, and restructuring (International Swaps and Derivatives Association 2014). To simplify, we will assume that default is the only credit event that can occur. The named entity whose default triggers the contract is called the reference entity of the CDS. In exchange for being protected against a potential loss, the protection buyer pays a quarterly premium until either the maturity

of the contract is reached or a default of the reference entity occurs. The yearly premium is equal to a fraction s , called a CDS spread, times the total notional value of the loan. The type of settlement where the protection seller pays the protection buyer $1 - R$ in case of default is called cash settlement. In practice, the value of R is determined by a post-default auction. Another settlement scheme is called physical settlement. For a physical settlement, the protection seller reimburses the protection buyer the face value of the bond in exchange for the defaulted bond. In this paper, we will assume that CDS contracts are settled with cash settlements with the market convention using a fixed value of $R = 0.4$. Typically, if a default occurs between quarterly payments, the protection buyer has to pay an accrual payment representing the payment accrued between payment dates. The most common maturity is 5 years, but CDSs with maturities 1, 2, 3, 4, 7, and 10 years also exist, with maturity on a set of standard roll dates: March 20, June 20, September 20, and December 20.

We next describe how CDSs are valued.

2.3 Valuation of Credit Default Swaps

This section is based on Castellacci (2012).

Valuing a CDS comes down to estimating survival probabilities of the reference entity or equivalently default probabilities. Based on the observed CDS spread, one can infer the implied risk neutral survival probabilities and by extension the implied risk neutral default probabilities.

We will now look at how one can compute implied risk neutral survival probabilities of reference entities given the spread curve of their CDS and a term structure. In the field of survival analysis, the main quantity of interest for an entity is the probability that the entity has survived until time t , given that the entity is still alive at time 0. If we define a default time τ , this is equivalent to

$$S(t) = P(\tau > t), \quad t \geq 0.$$

We then call $S(t)$ the *survival probability*. Conversely, we define the default probability as

$$1 - S(t) = P(\tau \leq t), \quad t \geq 0,$$

that is, the probability that default has occurred before time t . For a deterministic and continuous function $h(t)$, we can write the survival probability as

$$S(t) = e^{-\int_0^t h(u)du},$$

where h is the hazard rate. We think of $h(t)$ as the instantaneous rate of default at time t , given that default has not yet occurred at time t .

As implied previously, a CDS has two cash flow legs. The buyer of the protection pays a quarterly spread payment until either default or maturity, whichever comes first, and an accrual payment in case of default in order to transfer credit risk to the protection seller. The protection seller pays the buyer a payment of $1 - R$ at the time of default. If no default has occurred during the life of the contract, the protection seller pays 0 to the protection buyer. To make the contract fair at inception, the expected discounted value of the two cash flow legs must be exactly equal, i.e., $\pi_{PL} = \pi_{SP} + \pi_{AP}$. Here π_{PL} is the expected present value of the protection leg, i.e., what the protection seller pays, π_{SP} is the expected present value of the spread payments, and π_{AP} is the expected present value of the accrual payment. The paradigm we will use to find the risk neutral default probabilities is the so-called JPMorgan Model, a common market convention described in Castellacci (2012). The JPMorgan Model assumes that the risk-free interest rate process is independent of the default process and that the protection seller's payment in the event of default is made at the end of the period in which the default occurs. It is important to note that the assumption forcing the interest rate process and the default process to be independent is a necessary simplifying assumption. In the real world, these processes will often be correlated. For example, during crises with high default rates, interest rates are often lowered. These assumptions allow us to write the expected present value of the protection leg at inception as

$$\begin{aligned}\pi_{PL} &= (1 - R) \sum_{i=1}^n Z(0, T_i) P(T_{i-1} < \tau \leq T_i) \\ &= (1 - R) \sum_{i=1}^n Z(0, T_i) (S(T_{i-1}) - S(T_i)),\end{aligned}$$

with $S(T_0) = S(0) = 1$. The protection buyer's cash flow leg consists of regular spread payments of s paid at the end of each quarter until maturity or default, whichever comes first, and in case

a default occurs before maturity, the protection buyer is obligated to pay an accrual payment representing the spread payment accumulated since the last spread payment. To make the accrual payment tractable, we assume that defaults can only occur at the halfway point between payments and that accrual payments are made at the end of a quarter. This means that the accrual payment is $\frac{s}{2} \cdot \delta_i$. We discount the accrual payment using the ZCB at the next payment time T_i .

We write the expected present value of the spread payments with $S(0) = 1$ as

$$\pi_{SP} = s \sum_{i=1}^n \delta_i Z(0, T_i) S(T_i).$$

We write the expected present value of the accrual payments as

$$\pi_{AP} = \frac{s}{2} \sum_{i=1}^n \delta_i Z(0, T_i) (S(T_{i-1}) - S(T_i)).$$

We combine the two to write the expected present value of the payment leg:

$$\begin{aligned} \pi_{SP} + \pi_{AP} &= s \sum_{i=1}^n \delta_i Z(0, T_i) S(T_i) + \frac{s}{2} \sum_{i=1}^n \delta_i Z(0, T_i) (S(T_{i-1}) - S(T_i)) \\ &= s \sum_{i=1}^n \delta_i Z(0, T_i) \frac{S(T_{i-1}) + S(T_i)}{2}, \end{aligned}$$

where δ_i represents how many years are between each regular spread payment. In practice, the year-fraction δ_i depends on the day-count convention of the contract. Since we have already established that payments are made quarterly, we approximate the year-fraction by $\delta_i = \delta = 1/4$, and we get

$$\pi_{SP} + \pi_{AP} = \frac{s}{4} \sum_{i=1}^n Z(0, T_i) \frac{S(T_{i-1}) + S(T_i)}{2}.$$

It should be noted that the year-fraction $\delta = 1/4$ is an assumption that implies that all quarters are of equal length. This is of course not true, and to achieve better accuracy a standard market day-count convention such as Act/360 where δ_i is set to the actual number of days in the period divided by 360 should be used. In order to find the hazard function $h(t)$, we need to find the values $h(T_1), \dots, h(T_n)$ that solve

$$\frac{s}{4} \sum_{i=1}^n Z(0, T_i) \frac{S(T_{i-1}) + S(T_i)}{2} = (1 - R) \sum_{i=1}^n Z(0, T_i) (S(T_{i-1}) - S(T_i)),$$

where $h(t)$ enters through the survival probabilities. In the next section, we describe the bootstrapping procedure to estimate the discretized hazard curve.

2.3.1 Discretized Hazard Curve Estimation Algorithm

Since spread curves of CDSs are discrete and not continuous, we will have to assume that $h(t)$ is piecewise constant. Formally, just like for the risk-free rate function, we will assume that we can observe spreads $s_1, s_2, \dots, s_m$ for maturities $T_{n_1}, T_{n_2}, \dots, T_{n_m}$, where n_k is the index of the payment date coinciding with the k -th maturity. Let $T_1, T_2, \dots, T_N$ denote the quarterly payment dates, and assume we have a risk-free term structure such that $Z(0, T_i)$ is defined for all T_i . Lastly, we assume that hazard rates are piecewise constant between each of the maturities. That is, over the interval $t \in (T_{n_{k-1}}, T_{n_k}]$, $h(t) = h_k$, for $k = 1, 2, \dots, m$, with $T_{n_0} = 0$. Using s_1 alone, we then find the value of h_1

$$\begin{aligned} 0 &= (1 - R) \sum_{i=1}^{n_1} Z(0, T_i) (S(T_{i-1}) - S(T_i)) - \frac{s_1}{4} \sum_{i=1}^{n_1} Z(0, T_i) \frac{S(T_{i-1}) + S(T_i)}{2} \\ &= (1 - R) \sum_{i=1}^{n_1} Z(0, T_i) e^{-h_1 T_{i-1}} (1 - e^{-h_1 (T_i - T_{i-1})}) - \frac{s_1}{4} \sum_{i=1}^{n_1} Z(0, T_i) e^{-h_1 T_{i-1}} \frac{1 + e^{-h_1 (T_i - T_{i-1})}}{2} \\ &= \sum_{i=1}^{n_1} Z(0, T_i) e^{-h_1 T_{i-1}} \left(1 - R - \frac{s_1}{8} - e^{-h_1 (T_i - T_{i-1})} \left(1 - R + \frac{s_1}{8} \right) \right). \end{aligned}$$

Since h_1 is the only unknown in the equation, the value of h_1 can be found by a root-finding algorithm such as Brent's Method described in Appendix A.1.

We then want to find all the subsequent constant values of h . We can find h_k assuming we have found $h_1, h_2, \dots, h_{k-1}$ by solving the equation for h_k using Brent's Method:

$$0 = (1 - R) \sum_{i=1}^{n_k} Z(0, T_i) (S(T_{i-1}) - S(T_i)) - \frac{s_k}{4} \sum_{i=1}^{n_k} Z(0, T_i) \frac{S(T_{i-1}) + S(T_i)}{2}$$

where

$$S(T_i) = \exp \left(- \sum_{j=1}^{k-1} h_j (T_{n_j} - T_{n_{j-1}}) - h_k (T_i - T_{n_{k-1}}) \right),$$

with h piecewise constant taking values $h_1, \dots, h_{k-1}$ on the determined intervals and h_k on the final interval being solved for.

Now that we have a method to estimate the implied risk neutral survival probabilities of reference entities using their observed spreads, we are ready to describe collateralized debt obligations (CDOs). In the next section, we will describe the structure of CDOs.

2.4 Collateralized Debt Obligations

This section is based on Hull (2022).

CDOs are a type of securitized asset, and there exist two types of CDOs. The cash CDO is made up of a portfolio of bonds, while the synthetic CDO is made up of a portfolio of CDSs. The focus of this paper is on the synthetic CDO, and will simply be referred to as a CDO. In the market, it is generally recognized that holding a corporate bond issued by a reference entity has a similar risk to selling a CDS contract on the same reference entity. Therefore, similar risks can be obtained by entering into either of these two trades. This also means that the risk of holding a portfolio of corporate bonds is similar to that of a portfolio of CDSs on the same reference entities.

A synthetic CDO is structured around a reference portfolio of bonds (or reference entities) and a chosen maturity. A special-purpose vehicle (SPV) sells credit protection on each reference entity via CDSs with the same maturity as the CDO, generating spread income and incurring losses when reference entities default. Investors purchase tranches of the CDO, each defined by an attachment point A and a detachment point B . The tranche investors absorb the portfolio losses according to a waterfall: losses are first allocated to the most junior tranche (the equity tranche), and only once that tranche is wiped out do losses begin to affect more senior tranches. In return, each tranche receives a spread reflecting its loss exposure — high for the equity tranche, progressively lower for more senior tranches. To give a simple illustrative example of how cash flows are allocated to the tranches, imagine that we have a simple CDO with three tranches: equity, mezzanine, and senior.

The equity tranche is the most junior tranche and is responsible for the payouts on the CDSs until they reach 5% of the principal of the CDO. To compensate for this, it earns a spread of 500 basis points (bps), or 5%, per year on the outstanding tranche principal. In this case, the equity tranche has an attachment point of 0 and a detachment point of 5%.

The mezzanine tranche is more senior than the equity tranche and is responsible for the payouts on the CDSs after they surpass 5% and until they reach 15% of the CDO principal. Since the risk is significantly lower compared to the equity tranche, the mezzanine tranche earns a spread of 100 bps per year on the outstanding tranche principal.

Lastly, the senior tranche is responsible for the payouts on the CDSs when the payouts exceed 15% until the entire principal of the CDO is paid out. It earns a significantly lower spread at 20 bps per year on the outstanding tranche principal.

For a synthetic CDO with principal 10,000,000, the equity tranche would have principal 500,000, the mezzanine tranche would have principal 1,000,000, and the senior tranche would have principal 8,500,000. Until a default occurs, all tranches earn the spread on the full tranche principal. But if defaults occur leading to decreases in the CDO principal of 200,000, the equity tranche principal decreases to 300,000, while the other tranche principals are unchanged. As a result of the decrease in the equity tranche principal, the equity tranche spread is now earned on a lower basis and the annual spread payments go from $500000 \cdot 0.05 = 25000$ to $300000 \cdot 0.05 = 15000$. If total losses exceed the equity tranche principal, the mezzanine tranche principal will start to decrease, thus decreasing the spread payments for the mezzanine tranche. By dividing the CDO into tranches, we are able to package the portfolio into different securities with different risk profiles.

In the next section, we present the discretized framework for valuing tranches of a CDO.

2.5 Valuation of Synthetic CDOs

We assume that spread payments on a synthetic CDO tranche are made at times $\tau_1, \tau_2, \dots, \tau_m$ with $\tau_0 = 0$. We again assume spread payments to be quarterly. As with CDSs, we also define two cash flow legs for CDOs. For a given tranche (A, B) with attachment point A and detachment point B , we define the protection leg as the expected value of the payouts on the tranche. The payment leg we again split in the accrual payments and the spread payments. We assume that losses to a tranche occur in the middle of the interval between two spread payment dates. We denote the expected tranche principal for tranche (A, B) at time τ_j by $E_j^{A,B}$, and we define the

expected present value of the protection leg for tranche (A, B) as

$$\pi_{PL}^{A,B} = \sum_{j=1}^m \left(E_{j-1}^{A,B} - E_j^{A,B} \right) Z \left(0, \frac{\tau_{j-1} + \tau_j}{2} \right). \quad (1)$$

The expected present value of the expected regular spread payments for tranche (A, B) is

$$\pi_{SP}^{A,B} = \frac{s^{A,B}}{4} \sum_{j=1}^m E_j^{A,B} Z(0, \tau_j), \quad (2)$$

where $s^{A,B}$ is the spread earned on tranche (A, B) . Lastly, the expected present value of the accrual payments for tranche (A, B) is

$$\pi_{AP}^{A,B} = \frac{s^{A,B}}{8} \sum_{j=1}^m \left(E_{j-1}^{A,B} - E_j^{A,B} \right) Z \left(0, \frac{\tau_{j-1} + \tau_j}{2} \right). \quad (3)$$

In contrast to the CDS valuation, where the protection leg and accrual payments were discounted at the next payment date, here we discount both at the midpoint of the period — the assumed time of default — giving a more accurate present value. The fair spread on the tranche is then found by solving $\pi_{PL}^{A,B} = \pi_{SP}^{A,B} + \pi_{AP}^{A,B}$ for $s^{A,B}$. From equations 1, 2, and 3 we notice that we only need to know the payment times, expected tranche principals, and zero-coupon bond prices at the payment times to find the fair spread. Whereas zero-coupon bond prices are easy to find, finding the expected tranche principal relies on finding a distribution of defaults on the reference entities. We define solutions to this problem in Section 3, but first we define how fair spreads are quoted using the framework of the expected present values of the cash flow legs.

2.6 Fair Spreads and Upfronts

Now that we can compute the expected present values of the cash flow legs, we are able to find the fair spread and upfront of a tranche. As noted previously, we find the fair spread of tranche (A, B) , $s^{A,B}$, by solving $\pi_{PL}^{A,B} = \pi_{SP}^{A,B} + \pi_{AP}^{A,B}$ for $s^{A,B}$. If we define $\pi_{SP}^{A,B} = s^{A,B} \cdot A_{SP}^{A,B}$ and $\pi_{AP}^{A,B} = s^{A,B} \cdot A_{AP}^{A,B}$, we find $s^{A,B}$ as

$$s^{A,B} = \frac{\pi_{PL}^{A,B}}{A_{SP}^{A,B} + A_{AP}^{A,B}}.$$

It is a market convention to fix the spread of the equity tranche and sometimes also the most junior mezzanine tranche to 100 bps (i.e., $s_*^{A,B}$) because of their higher risk. In this case, in order for the contract to still be fair at inception, an upfront payment U is then given by

$$U = \pi_{PL}^{A,B} - s_*^{A,B} \left(A_{SP}^{A,B} + A_{AP}^{A,B} \right),$$

where $s_*^{A,B} = 0.01$.

In the next section, we define the Gaussian and double t copula models that allow us to estimate $E_j^{A,B}$, which is the missing input in the valuation of the tranches.

3 Copula Models

In this section, we define the Gaussian and double t copula models, as well as the quadrature procedures needed to build them. We also provide a critique of the Gaussian copula model that also serves as a motivation for the double t copula model.

However, we first need to define copula functions that provide the backbone of the two copula models.

3.1 Copula Functions

To define a copula function, we first need to note the following property of continuous random variables. For a continuous random variable Y with cumulative distribution function F_Y , the transformation $F_Y(Y)$ gives a random uniformly distributed variable on the interval $[0, 1]$. This transformation allows for the isolation of the dependency structure from the choice of marginal distributions.

A copula function is a function that links univariate marginal distributions to their joint multivariate distribution (Li 2000). Suppose that we have m uniform univariate random variables, $Y_1, Y_2, \dots, Y_m$. Then we can define the joint distribution with the use of a copula, C :

$$C(y_1, y_2, \dots, y_m, \rho) = P(Y_1 \leq y_1, Y_2 \leq y_2, \dots, Y_m \leq y_m),$$

where ρ represents the correlation structure between the variables. For marginal distribution functions $F_1, F_2, \dots, F_m$, copula functions can be used to establish a link to a joint distribution:

$$C(F_1(y_1), F_2(y_2), \dots, F_m(y_m)) = F(y_1, y_2, \dots, y_m).$$

This property was proven by Sklar (1959) and is part of Sklar's Theorem. Formally, we state Sklar's Theorem following Li (2000).

Sklar's Theorem

Let F be an m -dimensional distribution function with marginals $F_1, \dots, F_m$. Then there exists an m -copula C such that for all $\mathbf{x} \in \mathbb{R}^m$:

$$F(x_1, \dots, x_m) = C(F_1(x_1), \dots, F_m(x_m)).$$

If $F_1, \dots, F_m$ are continuous, then C is unique.

Sklar's Theorem establishes that any multivariate distribution function can be written in the form of a copula function.

There exist many different types of copulas, and we will now define two of the copulas we will apply in this paper. First, we define the Gaussian copula, and then we define the t copula. We also define the copula property of tail dependence.

3.1.1 Gaussian Copula

Suppose again that we have m univariate marginal distributions. For uniform $[0, 1]$ random variables $Y_1, Y_2, \dots, Y_m$, the Gaussian copula defines a joint distribution function

$$C_{Gauss}(y_1, y_2, \dots, y_m) = \Phi_m(\Phi^{-1}(y_1), \Phi^{-1}(y_2), \dots, \Phi^{-1}(y_m); \rho).$$

Here Φ_m denotes the m -dimensional cumulative normal distribution function and ρ denotes the correlation structure of Φ_m . Φ^{-1} is the univariate quantile function of the standard normal distribution.

Given univariate uniform marginals, this allows us to find their joint probability distribution.

3.1.2 t Copula

Let $F_m(\cdots; \nu, \rho)$ denote the m -dimensional cumulative t distribution function with ν degrees of freedom where ρ denotes the correlation structure of F_m . Then let $F^{-1}(\cdots; \nu)$ be the univariate quantile function of the t distribution with ν degrees of freedom. We can then construct a t copula for uniform $[0, 1]$ variables

$$C_t(y_1, y_2, \cdots, y_m) = F_m \left(F^{-1}(y_1; \nu), F^{-1}(y_2; \nu), \cdots, F^{-1}(y_m; \nu); \rho \right).$$

In the next section, we analyze the property of copulas called *tail dependence*.

3.1.3 Copula Tail Dependence

This section is based on Ruppert and Matteson (2016).

For a continuous bivariate random vector $Y = (Y_1, Y_2)$ with copula C_Y , the coefficient of lower tail dependence is denoted by λ_ℓ . For a quantile q , we define λ_ℓ as

$$\begin{aligned} \lambda_\ell &= \lim_{q \downarrow 0} P(Y_2 \leq F_{Y_2}^{-1}(q) | Y_1 \leq F_{Y_1}^{-1}(q)) \\ &= \lim_{q \downarrow 0} \frac{P(Y_1 \leq F_{Y_1}^{-1}(q), Y_2 \leq F_{Y_2}^{-1}(q))}{P(Y_1 \leq F_{Y_1}^{-1}(q))} \\ &= \lim_{q \downarrow 0} \frac{P(Y_1 \leq F_{Y_1}^{-1}(q), Y_2 \leq F_{Y_2}^{-1}(q))}{P(F_{Y_1}(Y_1) \leq q)} \\ &= \lim_{q \downarrow 0} \frac{C_Y(q, q)}{q}. \end{aligned}$$

λ_ℓ measures the probability of Y_2 being in the extreme lower tail given that Y_1 is in the extreme lower tail. In other words, it gives us a way to quantify the clustering in the extreme lower tail.

Similarly, we define the coefficient of upper tail dependence λ_u as

$$\begin{aligned}
\lambda_u &= \lim_{q \uparrow 1} P(Y_2 \geq F_{Y_2}^{-1}(q) | Y_1 \geq F_{Y_1}^{-1}(q)) \\
&= \lim_{q \uparrow 1} \frac{P(Y_2 \geq F_{Y_2}^{-1}(q), Y_1 \geq F_{Y_1}^{-1}(q))}{P(Y_1 \geq F_{Y_1}^{-1}(q))} \\
&= \lim_{q \uparrow 1} \frac{1 - P(Y_2 < F_{Y_2}^{-1}(q)) - P(Y_1 < F_{Y_1}^{-1}(q)) + P(Y_2 \leq F_{Y_2}^{-1}(q), Y_1 \leq F_{Y_1}^{-1}(q))}{1 - P(Y_1 \leq F_{Y_1}^{-1}(q))} \\
&= \lim_{q \uparrow 1} \frac{1 - P(F_{Y_2}(Y_2) < q) - P(F_{Y_1}(Y_1) < q) + P(Y_2 \leq F_{Y_2}^{-1}(q), Y_1 \leq F_{Y_1}^{-1}(q))}{1 - P(F_{Y_1}(Y_1) \leq q)} \\
&= \lim_{q \uparrow 1} \frac{1 - q - q + C_Y(q, q)}{1 - q} \\
&= \lim_{q \uparrow 1} \frac{1 - 2q + C_Y(q, q)}{1 - q} \\
&= \lim_{q \uparrow 1} \frac{2 - 2q - 1 + C_Y(q, q)}{1 - q} \\
&= \lim_{q \uparrow 1} \frac{2 - 2q}{1 - q} - \frac{1 - C_Y(q, q)}{1 - q} \\
&= 2 - \lim_{q \uparrow 1} \frac{1 - C_Y(q, q)}{1 - q}.
\end{aligned}$$

Similarly to the lower dependence coefficient, the upper dependence coefficient quantifies the clustering in the extreme upper tail. Closed-form results exist for the lower and upper dependence coefficients for Gaussian and t copulas. For any bivariate Gaussian copula, with $\rho \in (-1, 1)$, the lower and upper dependence coefficients are $\lambda_\ell = \lambda_u = 0$ (Ruppert and Matteson 2016). This means that the Gaussian copula has no clustering in the extreme tails. The bivariate t copula also has symmetry in its lower and upper dependence coefficients. For a bivariate t copula with ν degrees of freedom and correlation $\rho \in (-1, 1)$, the lower and upper dependence coefficients are

$$\lambda_\ell = \lambda_u = 2F_t \left(-\sqrt{\frac{(\nu+1)(1-\rho)}{1+\rho}}; \nu+1 \right),$$

where $F_t(\cdot; \nu+1)$ is the CDF of a t distribution with $\nu+1$ degrees of freedom as stated by Ruppert and Matteson (2016).

Now that we have defined copula functions and their tail dependence, we turn to the Gaussian copula model, which uses a Gaussian copula to model the time- τ_j expected tranche principal $E_j^{A,B}$

of a CDO tranche (A, B) . The absence of tail dependence in the Gaussian copula, noted above, will later motivate the introduction of the double t copula model as an alternative.

3.2 Gaussian Copula Model

This section is based on Hull (2022).

The economic intuition for using a copula model to model defaults in a credit portfolio is as follows. Suppose that we have M loans to M counterparties all with principal 1. We might then be able to estimate a marginal cumulative default probability curve for all M loans, but if we want to estimate an expected portfolio loss over time, we need to know how defaults cluster together. In other words, we need to find the joint default distribution of the M loans. This is what a copula allows us to find given a set of univariate uniform random variables and a correlation structure assumption.

As mentioned previously in Section 2.5, in order to estimate expected tranche principals, we need to know how the principal of the CDO behaves. This means finding a distribution of defaults while also taking into account the sizes of losses sustained on the default of each reference entity, i.e., what is the recovery rate R_i of entity i .

First, we will introduce the one-factor Gaussian copula model under the assumption that the portfolio is heterogeneous. This means that all reference entities act differently. Recovery rates, pairwise correlations, and default probabilities are varying across all reference entities, although no restriction makes sure that, for example, default probabilities of two reference entities cannot be equal.

We denote the default variable that determines the default time of the i 'th reference entity as X_i . We will let the default variable X_i be defined by two independent standard normal latent factors: S and Z_i . We call S the systematic factor, since it affects all M default variables, and we call Z_i the idiosyncratic factor since it only affects the i 'th reference entity. All Z_i are independent. For

a default correlation parameter ρ_i , we define the i 'th default variable as

$$X_i = \sqrt{\rho_i}S + \sqrt{1 - \rho_i}Z_i$$

where $0 \leq \rho_i < 1$. We invoke the assumption $\rho_i \geq 0$ to avoid complex numbers, and we require $\rho_i < 1$ to avoid dividing by zero later on. Non-negative correlation between defaults and the systematic factor also implies more defaults in times of stress and fewer defaults in times of stability, which is what we would expect in the real world. This assumption holds throughout the paper. Since S and Z_i are both standard normal, it is easy to verify that X_i is standard normal as well. The covariance between X_i and X_k is then

$$\begin{aligned} \text{Cov}(X_i, X_k) &= \text{E}[X_i X_k] - \text{E}[X_i]\text{E}[X_k] \\ &= \text{E} \left[\sqrt{\rho_i \rho_k} S^2 + \sqrt{\rho_i} \sqrt{1 - \rho_k} S Z_k + \sqrt{\rho_k} \sqrt{1 - \rho_i} S Z_i + \sqrt{1 - \rho_i} \sqrt{1 - \rho_k} Z_i Z_k \right] \\ &= \sqrt{\rho_i \rho_k} \text{E}[S^2] + \sqrt{\rho_i} \sqrt{1 - \rho_k} \text{E}[S Z_k] \\ &\quad + \sqrt{\rho_k} \sqrt{1 - \rho_i} \text{E}[S Z_i] + \sqrt{1 - \rho_i} \sqrt{1 - \rho_k} \text{E}[Z_i Z_k] \\ &= \sqrt{\rho_i \rho_k} (\text{Var}[S] + \text{E}[S]^2) + \sqrt{\rho_i} \sqrt{1 - \rho_k} \text{E}[S] \text{E}[Z_k] \\ &\quad + \sqrt{\rho_k} \sqrt{1 - \rho_i} \text{E}[S] \text{E}[Z_i] + \sqrt{1 - \rho_i} \sqrt{1 - \rho_k} \text{E}[Z_i] \text{E}[Z_k] \\ &= \sqrt{\rho_i \rho_k} (1 + 0) + 0 + 0 + 0 \\ &= \sqrt{\rho_i \rho_k}. \end{aligned}$$

This means that the correlation between X_i and X_k is

$$\begin{aligned} \text{Corr}(X_i, X_k) &= \frac{\text{Cov}(X_i, X_k)}{\sqrt{\text{Var}(X_i) \text{Var}(X_k)}} \\ &= \frac{\sqrt{\rho_i \rho_k}}{\sqrt{1 \cdot 1}} = \sqrt{\rho_i \rho_k}. \end{aligned}$$

We denote the time- τ_j marginal cumulative default probability of reference entity i by $Q_i(\tau_j)$. These are found by the method laid out in Section 2.3.1 using the CDS spreads. Let τ_i denote the default time of reference entity i , with cumulative distribution function $Q_i(\cdot)$. To link the latent default variable X_i to the default time τ_i , we use the following transform. Since X_i is standard

normal, $\Phi(X_i)$ is uniformly distributed on $[0, 1]$, and therefore

$$\tau_i = Q_i^{-1}(\Phi(X_i))$$

has cumulative distribution function Q_i by construction. This mapping preserves the marginal cumulative default distribution implied by the CDS spread of reference entity i , while the dependence structure across entities is governed by the Gaussian copula induced by the systematic factor S . The event of default before time τ_j can then be expressed as

$$\{\tau_i < \tau_j\} = \{Q_i^{-1}(\Phi(X_i)) < \tau_j\} = \{\Phi(X_i) < Q_i(\tau_j)\},$$

where the last equality assumes monotonicity of Q_i . Equivalently, the default condition can be written as

$$\Phi\left(\sqrt{\rho_i}S + \sqrt{1 - \rho_i}Z_i\right) < Q_i(\tau_j).$$

Because the standard normal cumulative distribution function is invertible, we can write

$$\sqrt{\rho_i}S + \sqrt{1 - \rho_i}Z_i < \Phi^{-1}(Q_i(\tau_j))$$

where Φ^{-1} naturally is the standard normal quantile function. Solving for Z_i , we get

$$Z_i < \frac{\Phi^{-1}(Q_i(\tau_j)) - \sqrt{\rho_i}S}{\sqrt{1 - \rho_i}}.$$

Since Z_i is standard normal, we can find the time- τ_j conditional cumulative default probability of reference entity i as

$$Q_i(\tau_j|S) = \Phi\left(\frac{\Phi^{-1}(Q_i(\tau_j)) - \sqrt{\rho_i}S}{\sqrt{1 - \rho_i}}\right).$$

A popular simplifying assumption is that the portfolio is homogeneous, which means that all recovery rates, default correlations, and default probabilities are equal across all the reference entities in the portfolio. This also implies bootstrapping the hazard curve using the corresponding index spread of the CDO instead of the CDS spreads of reference entities (Hull 2022). In the homogeneous case, the conditional default probability becomes

$$Q(\tau_j|S) = \Phi\left(\frac{\Phi^{-1}(Q(\tau_j)) - \sqrt{\rho}S}{\sqrt{1 - \rho}}\right).$$

Under this assumption, the correlation between X_i and X_k becomes

$$\text{Corr}(X_i, X_k) = \sqrt{\rho_i \rho_k} = \sqrt{\rho \rho} = \rho.$$

This implies that the default correlation between a pair of reference entities (X_i, X_k) is the same for all i and $k, i \neq k$. In comparison, for a portfolio of M reference entities, the correlation matrix is $M \times M$ with $M(M-1)/2$ off-diagonal elements. This means that for a portfolio of 125 reference entities, there are $125 \cdot (125 - 1)/2 = 7750$ off-diagonal elements. The model then assumes that all reference entities affect each other uniformly, meaning that all 7750 correlations are equal (Brigo, Pallavicini, and Torresetti 2010). This simplification is, of course, distorting the reality, especially in portfolios with more volatility. Perhaps the assumption is not too limiting when modeling high credit quality CDOs.

Conditional on the systematic factor and assuming homogeneity, the amount of defaults in the portfolio at time τ_j is then binomial distributed, which means that the conditional probability of observing m defaults in a portfolio of size M at time τ_j is

$$P(m, \tau_j | S) = \frac{M!}{m!(M-m)!} Q(\tau_j | S)^m (1 - Q(\tau_j | S))^{M-m}.$$

Under homogeneity, each default reduces the portfolio principal by $(1 - R)/M$. For a tranche (A, B) and a recovery rate R , we define quantities

$$M_A = \frac{A \cdot M}{1 - R}$$

$$M_B = \frac{B \cdot M}{1 - R}$$

as the lower and upper limits of when losses affect tranche (A, B) . If we assume that the principal of tranche (A, B) is 1 it means that when total losses on the CDO principal are less than M_A , the principal of tranche (A, B) is 1, and when losses exceed M_B , the principal of the tranche is 0. When the number of defaults m results in the principal lost on the tranche being between M_A and M_B , the tranche principal is

$$\frac{B - \frac{m(1-R)}{M}}{B - A}.$$

We define the S -conditional expected tranche principal of tranche (A, B) at time τ_j as

$$E_j^{A,B}(S) = \sum_{m=0}^{\lceil M_A \rceil - 1} P(m, \tau_j | S) + \sum_{m=\lceil M_A \rceil}^{\lceil M_B \rceil - 1} P(m, \tau_j | S) \frac{B - \frac{m(1-R)}{M}}{B - A}.$$

$\lceil \cdot \rceil$ handles non-integer limits. Conditioning the expected present values of the cash flow legs (equations 1, 2, and 3) on S gives:

$$\begin{aligned} \pi_{PL}^{A,B}(S) &= \sum_{j=1}^m \left(E_{j-1}^{A,B}(S) - E_j^{A,B}(S) \right) Z \left(0, \frac{\tau_{j-1} + \tau_j}{2} \right), \\ \pi_{SP}^{A,B}(S) &= \frac{s^{A,B}}{4} \sum_{j=1}^m E_j^{A,B}(S) Z(0, \tau_j), \\ \pi_{AP}^{A,B}(S) &= \frac{s^{A,B}}{8} \sum_{j=1}^m \left(E_{j-1}^{A,B}(S) - E_j^{A,B}(S) \right) Z \left(0, \frac{\tau_{j-1} + \tau_j}{2} \right). \end{aligned}$$

Since S is standard normal, we can find the unconditional values of equations 1, 2, and 3 by integrating them over a standard normal distribution. This is done using Gauss-Hermite quadrature, which is defined in the next section.

3.2.1 Gauss-Hermite Quadrature

We want to calculate the integral

$$\mathbb{E}[g(S)] = \int_{-\infty}^{\infty} g(S) \phi(S) dS,$$

where $g(S)$ is conditional on S . Hull (2022) describes a method for approximating this integral by using Gauss-Hermite quadrature.

Gauss-Hermite quadrature is an approximation of the integral as

$$\int_{-\infty}^{\infty} h(x) \phi(x) dx \approx \sum_{i=1}^n \frac{1}{\sqrt{\pi}} w_i h(x_i).$$

Finding the appropriate values of w_i and x_i involves Hermite polynomials. Hermite polynomials are given by the formula

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} e^{-x^2}.$$

For instance, the first Hermite polynomial reduces to $H_1(x) = 2x$.

We define $x_i (1 \leq i \leq n)$ as the n roots of $H_n(x)$ and define the weights w_i as

$$w_i = \frac{2^{n-1} n! \sqrt{\pi}}{n^2 [H_{n-1}(x_i)]^2}.$$

The key result is that using Gauss-Hermite quadrature, we can approximate the integral

$$\int_{-\infty}^{\infty} f(x) dx \approx \sum_{i=1}^n w_i e^{x_i^2} f(x_i).$$

Letting $f(x) = \frac{1}{\sqrt{\pi}} e^{-x^2} g(\sqrt{2}x)$ we get

$$\int_{-\infty}^{\infty} \frac{1}{\sqrt{\pi}} e^{-x^2} g(\sqrt{2}x) dx \approx \sum_{i=1}^n w_i e^{x_i^2} \frac{1}{\sqrt{\pi}} e^{-x_i^2} g(\sqrt{2}x_i),$$

$$\frac{1}{\sqrt{\pi}} \int_{-\infty}^{\infty} e^{-x^2} g(\sqrt{2}x) dx \approx \frac{1}{\sqrt{\pi}} \sum_{i=1}^n w_i g(\sqrt{2}x_i).$$

Letting $x = S/\sqrt{2}$ and hence $dx = 1/\sqrt{2}dS$ we get

$$\frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} e^{-S^2/2} g(S) dS \approx \frac{1}{\sqrt{\pi}} \sum_{i=1}^n w_i g(S_i).$$

We have now proven that we can approximate the integral with

$$\int_{-\infty}^{\infty} g(S) \phi(S) dS \approx \frac{1}{\sqrt{\pi}} \sum_{i=1}^n w_i g(S_i).$$

If we let $g(S) = \pi_{SP}^{A,B}(S)$, we can approximate the unconditional expected value of the spread payments as

$$\int_{-\infty}^{\infty} \pi_{SP}^{A,B}(S) \phi(S) dS \approx \frac{1}{\sqrt{\pi}} \sum_{i=1}^n w_i \pi_{SP}^{A,B}(S_i)$$

and likewise for the other cash flow legs. The value of n , i.e., the number of integration points, determines the accuracy of the approximation. A higher number of integration points results in a finer grid, similarly to standard discretized Riemann integration. However, a higher number of

integration points also requires more calculations, and thus one has to make a compromise between accuracy and use of computing power.

Having defined Gauss-Hermite quadrature, which allows us to build the Gaussian copula model, we are now able to dive into the pricing of tranches using the model. However, in the next section, we present a critique of the Gaussian copula model that also motivates the usage of the double t copula model.

3.2.2 Critique of the Gaussian Copula Model

This section is based on an article published in Wired in 2009 called "The Formula That Killed Wall Street" (Salmon 2009).

The Gaussian copula model was introduced by Li (2000) and was soon adopted by the financial sector. Determining how different events co-associate has always been a challenge in risk management. In large portfolios, one may be able to determine the default probability of single components independently, but determining how the portfolio behaves in extreme cases will always be dependent on the joint default probability of components. In practice, to determine default probabilities of companies, you may look at historical default data to get an idea of how often firms in, for example, a given industry or with a given credit rating default. This data is sparse, but does exist. When you have to determine the joint probability of two or more defaults in the same time period, it becomes virtually impossible to estimate. According to Salmon (2009), this is one of the problems solved by the Gaussian copula model. Instead of estimating default probabilities of single components through historical data, one can estimate implied default probabilities of firms using the traded spread on CDSs issued on them. Then, with a standard normal assumption on the systematic and idiosyncratic factors driving the default, one can tie together the implied marginal probabilities to the implied joint probabilities. Lastly, with a homogeneity assumption on the constituents of the portfolio, one gets a Gaussian copula model with a single correlation parameter. As explained previously, the homogeneity assumption is, of course, a very limiting assumption. Academics such as Paul Wilmott and Nassim Taleb have also warned against the usage of correlations between securities because of their instability. Paul Wilmott warned "the

correlations between financial quantities are notoriously unstable” as cited in Salmon (2009). Nassim Taleb is also cited for being against the notion of using correlations to quantify risk in Salmon (2009), stating: ”Co-association between securities is not measurable using correlation”.

The usage of the Gaussian copula model is a classic example of something working until it does not. The model was used to quantify the correlations that were then used to determine the risk of portfolios, and the model was especially used to price CDOs and their tranches. According to Salmon (2009), the Gaussian copula model ”was used to price hundreds of billions of dollars worth of CDOs filled with mortgages”. A mortgage, of course, being a loan typically made to a household to finance the purchase of real estate. As households default on their loans, the most junior tranche of the CDO gets wiped out starting with the equity tranche. Under normal conditions, mortgage defaults are rare and are mostly uncorrelated. For example, a mortgage default may be caused by a single individual losing their job. This means that if one bases the risk of CDOs on data from normal conditions, one would judge the risk of the CDO tranches to be very low. If house prices are also increasing, and by extension increasing the equity values of the homes those mortgages were financing, the illusion of low risk becomes even more prominent. However, if house prices are falling, then home equity values are falling too, and this increases the rates of mortgage defaults in the economy. This is exactly what started happening in the USA in 2006. The housing market started to fall, leading to more mortgage defaults, which in turn wiped out some of the principal in the CDOs that investors had not foreseen.

Using the normal distribution has typically been the default approach to model financial data because of its simplicity, but has proven insufficient in many cases because of the distribution’s exponential tails. This means that extreme events such as a nationwide decrease in real estate prices that leads to increased rates of mortgage default were assigned probabilities of virtually 0, thus causing the events to not be priced in. It is this inability to capture extreme marginal and joint observations of the Gaussian copula model that necessitates the usage of a heavy-tailed distribution like the t distribution and a copula model with tail dependence like the t copula.

In the next section, we will introduce the *double t copula model*. As we saw in Section 3.1.3, using a t distribution based copula model introduces tail dependence.

3.3 Double t Copula Model

The one-factor double t copula model is an alternative version of the one-factor Gaussian copula model that we have just examined. In the Gaussian version, the default variable is given by $X_i = \sqrt{\rho}S + \sqrt{1 - \rho}Z_i$, where both S and Z_i are standard normal. This results in the default variable also being standard normal.

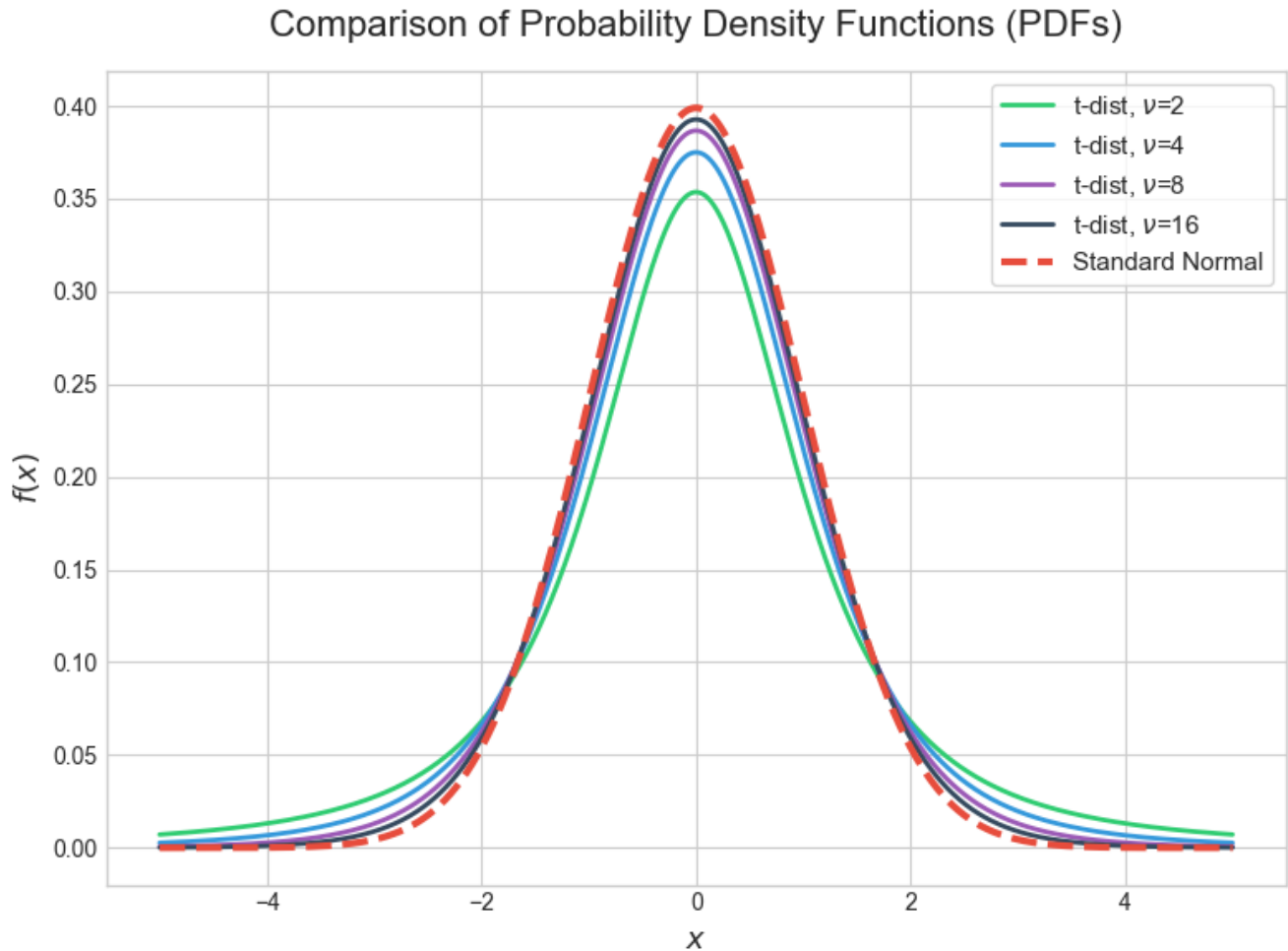


Figure 1: This graph shows the probability density functions of the standard normal distribution as well as t distribution with 4 different choices of ν .

One motivation for using the t distribution instead of the normal distribution is that whereas the normal distribution has exponential tails that quickly decay to zero, the t distribution is

considered a heavy-tailed distribution. This is illustrated in Figure 1, where we see that the tails of the standard normal distribution go to zero very quickly. The t distribution on the other hand has tails that decrease slower, especially for low values of ν . The other motivation is that the t copula has positive tail dependence coefficients $\lambda_\ell = \lambda_u > 0$ for $\rho \in (-1, 1)$ in contrast to the Gaussian copula's tail dependence coefficients of 0.

The double t copula model was introduced by Hull and White (2004) and works similarly to the Gaussian copula model, we just change the distributions of the latent systematic factor and idiosyncratic factors. As the name implies, we change the distribution to a t distribution. We continue to assume homogeneity and, therefore, the default variable is still defined by

$$X_i = \sqrt{\rho}S + \sqrt{1 - \rho}Z_i,$$

we have just changed the distribution of S and Z_i . S and all Z_i are still independent. Hull and White (2004) suggest scaling S and Z_i to give the factors unit variance. The variance of a t distributed random variable with $\nu > 2$ degrees of freedom is $\frac{\nu}{\nu-2}$. For $1 < \nu \leq 2$ the variance is ∞ , and for $\nu \leq 1$ it is undefined. Therefore, we will require $\nu > 2$. The mean of a t distributed random variable with $\nu > 1$ is 0, and otherwise undefined. After scaling the factors, the default variable is then defined by

$$X_i = \sqrt{\rho}\sqrt{\frac{\nu_S - 2}{\nu_S}}S + \sqrt{1 - \rho}\sqrt{\frac{\nu_{Z_i} - 2}{\nu_{Z_i}}}Z_i.$$

We point out that a scaled t distribution is not a t distribution, but this does not affect the model. To simplify, we will assume S and all Z_i to have the same degrees of freedom, ν , therefore, the default variable is

$$X_i = \sqrt{\rho}\sqrt{\frac{\nu - 2}{\nu}}S + \sqrt{1 - \rho}\sqrt{\frac{\nu - 2}{\nu}}Z_i.$$

Assuming the same degrees of freedom for S and Z_i improves the tractability of the model and also makes the model more parsimonious. Having all Z_i being i.i.d. also ensures that the homogeneity

assumption is not violated. The mean of X_i is clearly 0, and the variance is given by

$$\begin{aligned}
 \text{Var}(X_i) &= \rho \frac{\nu-2}{\nu} \text{Var}(S) + (1-\rho) \frac{\nu-2}{\nu} \text{Var}(Z_i) \\
 &= \rho \frac{\nu-2}{\nu} \frac{\nu}{\nu-2} + (1-\rho) \frac{\nu-2}{\nu} \frac{\nu}{\nu-2} \\
 &= \rho + 1 - \rho \\
 &= 1.
 \end{aligned}$$

The covariance between X_i and X_k is

$$\begin{aligned}
 \text{Cov}(X_i, X_k) &= \text{E}[X_i X_k] - \text{E}[X_i] \text{E}[X_k] \\
 &= \text{E} \left[\sqrt{\rho} \sqrt{\frac{\nu-2}{\nu}} S^2 + \sqrt{\rho} \sqrt{1-\rho} \sqrt{\frac{\nu-2}{\nu}} S Z_k \right. \\
 &\quad \left. + \sqrt{\rho} \sqrt{1-\rho} \sqrt{\frac{\nu-2}{\nu}} S Z_i + \sqrt{1-\rho} \sqrt{1-\rho} \sqrt{\frac{\nu-2}{\nu}} \sqrt{\frac{\nu-2}{\nu}} Z_i Z_k \right] - 0 \cdot 0 \\
 &= \rho \frac{\nu-2}{\nu} \text{E}[S^2] + \sqrt{\rho} \sqrt{1-\rho} \frac{\nu-2}{\nu} \text{E}[S Z_k] \\
 &\quad + \sqrt{\rho} \sqrt{1-\rho} \frac{\nu-2}{\nu} \text{E}[S Z_i] + (1-\rho) \frac{\nu-2}{\nu} \text{E}[Z_i Z_k] \\
 &= \rho \frac{\nu-2}{\nu} (\text{Var}[S] + \text{E}[S]^2) + \sqrt{\rho} \sqrt{1-\rho} \frac{\nu-2}{\nu} \text{E}[S] \text{E}[Z_k] \\
 &\quad + \sqrt{\rho} \sqrt{1-\rho} \frac{\nu-2}{\nu} \text{E}[S] \text{E}[Z_i] + (1-\rho) \frac{\nu-2}{\nu} \text{E}[Z_i] \text{E}[Z_k] \\
 &= \rho \frac{\nu-2}{\nu} \left(\frac{\nu}{\nu-2} + 0 \right) + 0 + 0 + 0 \\
 &= \rho.
 \end{aligned}$$

With correlation

$$\begin{aligned}
 \text{Corr}(X_i, X_k) &= \frac{\text{Cov}(X_i, X_k)}{\sqrt{\text{Var}(X_i) \text{Var}(X_k)}} \\
 &= \frac{\rho}{\sqrt{1 \cdot 1}} \\
 &= \rho.
 \end{aligned}$$

By scaling the factors, we get the same mean, variance, and correlation of the default variable that we did in the Gaussian model. With the moments established, we turn to finding the distribution of X_i , which is needed to evaluate the default condition.

In the Gaussian model, we could easily find the distribution of the default variable because a linear combination of normal distributed variables is also normal distributed. For the t distribution, however, that does not hold. In the original paper, Hull and White (2004) suggest the procedure of "probability bucketing" to estimate the time- τ_j expected tranche principal of the double t copula model. However, Brigo, Pallavicini, and Torresetti (2010) suggest that we similarly to the Gaussian copula model integrate the conditional default distribution over the systematic factor to find the expected tranche principal. This is the approach we will take. Since we do not know the distribution of the default variable, we first need to find it. To do this, we make use of a handy property of characteristic functions.

We remind ourselves that the characteristic function of a random variable Y , φ_Y , is the Fourier transform of the probability density function of Y , assuming that Y has a density.

For independent random variables $Y_1, Y_2, \dots, Y_n$, we can find the distribution of the sum

$$S_n = \sum_{i=1}^n a_i Y_i,$$

where a_i are constants, by first computing the product of characteristic functions

$$\varphi_{S_n}(x) = \prod_{i=1}^n \varphi_i(a_i x),$$

where φ_i is the characteristic function of Y_i and then using an inverse Fourier transform to find the density of S_n .

The t distribution with ν degrees of freedom has density

$$f_{t_\nu}(x) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\pi\nu}\Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}, \quad \nu > 0,$$

where Γ denotes the gamma function. The characteristic function of the t distribution is

$$\varphi_{t_\nu}(x) = \frac{(\sqrt{\nu}|x|)^{\nu/2} \cdot K_{\nu/2}(\sqrt{\nu}|x|)}{\Gamma(\nu/2) \cdot 2^{\nu/2-1}}, \quad \nu > 0,$$

where K_ν is the modified Bessel function of the second kind.

Referring back to the default variable $X_i = \sqrt{\rho}\sqrt{\frac{\nu-2}{\nu}}S + \sqrt{1-\rho}\sqrt{\frac{\nu-2}{\nu}}Z_i$, the characteristic function of X_i is then given by:

$$\varphi_{X_i}(t) = \varphi_{t_\nu} \left(\sqrt{\rho}\sqrt{\frac{\nu-2}{\nu}}t \right) \varphi_{t_\nu} \left(\sqrt{1-\rho}\sqrt{\frac{\nu-2}{\nu}}t \right).$$

The t distribution is symmetric around 0, and therefore the sum of t distributions is also symmetric around 0. Symmetric distributions have the convenient feature that their characteristic functions are real-valued (Feller 1971).

The density of X_i can then be recovered by Fourier inversion:

$$f_{X_i}(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} e^{-i\xi x} \varphi_{X_i}(\xi) d\xi.$$

The Fourier inversion is carried out numerically as inverse fast Fourier transforms (IFFT), which are described in the Appendix A.2.

The cumulative distribution function of X_i can then be found

$$\begin{aligned} F_{X_i}(t) &= \int_{-\infty}^t f_{X_i}(s) ds \\ &= \int_{-\infty}^t \frac{1}{2\pi} \int_{-\infty}^{\infty} e^{-i\xi s} \varphi_{X_i}(\xi) d\xi ds \\ &= \frac{1}{2\pi} \int_{-\infty}^t \int_{-\infty}^{\infty} e^{-i\xi s} \varphi_{X_i}(\xi) d\xi ds. \end{aligned}$$

The CDF is recovered by numerically integrating the density recovered using IFFT.

Since the densities of scaled t distributions are positive, the density of X_i is also positive, which means that the CDF of X_i is strictly increasing and, therefore, the quantile function of X_i , $F_{X_i}^{-1}$, exists.

By the same probability transform argument used in Section 3.2, defining $\tau_i = Q_i^{-1}(F_{X_i}(X_i))$ gives a default time with marginal cumulative distribution Q_i . The event of default before time τ_j is therefore equivalent to $F_{X_i}(X_i) < Q_i(\tau_j)$. Under homogeneity, $Q_i(\tau_j) = Q(\tau_j)$ and $\rho_i = \rho$, so the

default condition becomes:

$$\begin{aligned}
 F_{X_i} \left(\sqrt{\rho} \sqrt{\frac{\nu-2}{\nu}} S + \sqrt{1-\rho} \sqrt{\frac{\nu-2}{\nu}} Z_i \right) &< Q(\tau_j), \\
 \sqrt{\rho} \sqrt{\frac{\nu-2}{\nu}} S + \sqrt{1-\rho} \sqrt{\frac{\nu-2}{\nu}} Z_i &< F_{X_i}^{-1}(Q(\tau_j)), \\
 Z_i &< \frac{F_{X_i}^{-1}(Q(\tau_j)) - \sqrt{\rho} \sqrt{\frac{\nu-2}{\nu}} S}{\sqrt{1-\rho} \sqrt{\frac{\nu-2}{\nu}}}.
 \end{aligned}$$

The last inequality holds because $\sqrt{1-\rho} \sqrt{\frac{\nu-2}{\nu}}$ is strictly positive. Finally, we apply the cumulative distribution function of Z_i, F_{Z_i} to get the conditional cumulative default probabilities

$$Q(\tau_j|S) = F_{Z_i} \left(\frac{F_{X_i}^{-1}(Q(\tau_j)) - \sqrt{\rho} \sqrt{\frac{\nu-2}{\nu}} S}{\sqrt{1-\rho} \sqrt{\frac{\nu-2}{\nu}}} \right).$$

We define $E_j^{A,B}(S)$ as in Section 3.2 only changing the distribution of S and the conditional default probabilities $Q(\tau_j|S)$.

To calculate the unconditional values of $\pi_{PL}^{A,B}(S)$, $\pi_{SP}^{A,B}(S)$ and $\pi_{AP}^{A,B}(S)$, we can no longer use Gauss-Hermite quadrature because S is t distributed, and we instead have to resort to the procedure of adaptive Gauss-Hermite quadrature, which is described in the next section.

3.3.1 Adaptive Gauss-Hermite Quadrature

This section is based on Liu and Pierce (1994).

We unfortunately are not able to simply plug in the t distribution cumulative distribution function (CDF) in place of the Gaussian CDF. We want to evaluate the integral

$$\int_{-\infty}^{\infty} g(t) dt$$

for $g(t) > 0$ and where the ratio between $g(t)$ and some Gaussian probability density function (PDF) $\phi(t; \mu, \sigma)$ is a smooth function, $g(t)$ is unimodal and twice differentiable at the mode. For a function $f(t) = \frac{g(t)}{\phi(t; \mu, \sigma)}$, where $\phi(t; \mu, \sigma)$ is an arbitrary Gaussian density, we can reexpress the integral as

$$\int_{-\infty}^{\infty} f(t) \phi(t; \mu, \sigma) dt. \quad (4)$$

By change of variables, the sampling nodes for a Gaussian density $\phi(t; \mu, \sigma)$ become $t_i = \mu + \sqrt{2\sigma}x_i$, and we scale the weights used in Gauss-Hermite quadrature by $w_i/\sqrt{\pi}$. We use a Laplace approximation, choosing $\hat{\mu}$ as the mode of $g(t)$ and $\hat{\sigma} = 1/\sqrt{j}$, where

$$j = -\frac{\partial^2}{\partial t^2} \log g(t) \Big|_{t=\hat{\mu}}$$

is the observed information of $g(t)$. This gives a Gaussian PDF $\phi(t; \hat{\mu}, \hat{\sigma})$ with the same first and second derivatives at the mode as the integrand $g(t)$. This is to ensure that we are sampling the function in the region where the probability mass is greatest. We define

$$h(t) = \frac{g(t)}{\phi(t; \hat{\mu}, \hat{\sigma})},$$

so that we can write

$$\int_{-\infty}^{\infty} g(t) dt = \int_{-\infty}^{\infty} h(t) \phi(t; \hat{\mu}, \hat{\sigma}) dt.$$

If we then apply Gauss-Hermite quadrature in the form of (4) using $\phi(t; \hat{\mu}, \hat{\sigma})$ and $h(t)$, $g(t)$ will

be sampled in the relevant range, giving

$$\begin{aligned}
\int_{-\infty}^{\infty} g(t) dt &\approx \sum_{i=1}^n \frac{w_i}{\sqrt{\pi}} h(\hat{\mu} + \sqrt{2}\hat{\sigma}x_i) \\
&= \sum_{i=1}^n \frac{w_i}{\sqrt{\pi}} g(\hat{\mu} + \sqrt{2}\hat{\sigma}x_i) \frac{1}{\phi(t; \hat{\mu}, \hat{\sigma})} \\
&= \sum_{i=1}^n \frac{w_i}{\sqrt{\pi}} g(\hat{\mu} + \sqrt{2}\hat{\sigma}x_i) \frac{1}{\frac{1}{\sqrt{2\pi}\hat{\sigma}} e^{-\frac{(\hat{\mu} + \sqrt{2}\hat{\sigma}x_i - \hat{\mu})^2}{2\hat{\sigma}^2}}} \\
&= \sum_{i=1}^n \frac{w_i}{\sqrt{\pi}} g(\hat{\mu} + \sqrt{2}\hat{\sigma}x_i) \sqrt{2\pi}\hat{\sigma} e^{\frac{2\hat{\sigma}^2 x_i^2}{2\hat{\sigma}^2}} \\
&= \sum_{i=1}^n \frac{w_i}{\sqrt{\pi}} g(\hat{\mu} + \sqrt{2}\hat{\sigma}x_i) \sqrt{2\pi}\hat{\sigma} e^{x_i^2} \\
&= \sqrt{2}\hat{\sigma} \sum_{i=1}^n w_i^* g(\hat{\mu} + \sqrt{2}\hat{\sigma}x_i),
\end{aligned}$$

with $w_i^* = w_i e^{x_i^2}$. The value of n determines the accuracy of the approximation as we discussed for standard Gauss-Hermite quadrature.

If we let $g(S) = \pi_{SP}^{A,B}(S)f_S(S)$, that is, the conditional expected present value of the spread payments for a tranche (A, B) times the density function of S , we get the unconditional value

$$\pi_{SP}^{A,B} = \int_{-\infty}^{\infty} \pi_{SP}^{A,B}(S)f_S(S) dS = \int_{-\infty}^{\infty} \frac{s^{A,B}}{4} \sum_{j=1}^m E_j^{A,B}(S)Z(0, \tau_j)f_S(S) dS,$$

where f_S is the PDF of the systematic factor. To avoid confusion, we point out that $s^{A,B}$ is the yearly spread payment for tranche (A, B) and S is the systematic factor. If we transform $S_i = \hat{\mu} + \sqrt{2}\hat{\sigma}x_i$, the approximation is then

$$\begin{aligned}
&\int_{-\infty}^{\infty} \frac{s^{A,B}}{4} \sum_{j=1}^m E_j^{A,B}(S)Z(0, \tau_j)f_S(S) dS \approx \\
&\sqrt{2}\hat{\sigma} \sum_{i=1}^n w_i^* \left(\frac{s^{A,B}}{4} \sum_{j=1}^m E_j^{A,B}(\hat{\mu} + \sqrt{2}\hat{\sigma}x_i)Z(0, \tau_j)f_S(\hat{\mu} + \sqrt{2}\hat{\sigma}x_i) \right).
\end{aligned}$$

We then need to find the mode and observed information of $g(S)$. Since this is a relatively complex function, we will refrain from trying to find the mode and observed information analytically.

Instead, we find the mode and observed information numerically for each tranche (A, B) . By evaluating $g(S)$ over a large enough range around the mode of the t distribution, we can find the maximum value of $g(S)$ and the value S that supplies it. We choose to evaluate $g(S)$ over the interval $S \in [-10, 10]$. If we assume that $g(S)$ is unimodal, this value of S is the mode of $g(S)$. When we want to find the observed information, we first need to find the value of j . We therefore need to find the second derivative of $\log g(S)$ evaluated at the mode.

We do this simply by discretizing and approximating the second-derivative

$$(\log g)''(S) \approx \frac{\log g(S+h) + \log g(S-h) - 2\log g(S)}{h^2}$$

for some small h (Press et al. 2007). We choose $h = 10^{-4}$ as suggested by Press et al. (2007).

Having defined both copula models along with how fair spreads and upfronts are computed, we turn next to the paradigms of implied correlation.

3.4 Implied Correlation

Just as the Black-Scholes model can be inverted to recover an implied volatility from an observed option price, the Gaussian and double t copula models can be inverted to recover an implied correlation from an observed tranche spread or upfront. Under the homogeneity assumption and with the recovery rate fixed at the market convention of $R = 0.4$, the only free parameter remaining in the Gaussian copula model is the correlation ρ . If ν is also fixed, the only free parameter in the double t copula model likewise is ρ . Implied correlation is therefore the value of ρ that makes the model price match the market-observed tranche spread and upfront.

There are two distinct paradigms for implied correlation in the CDO literature: compound correlation and base correlation. Each has different properties and practical uses, and understanding why both exist requires first understanding the failure of the simpler one.

3.4.1 Compound Correlation

This section is based on Hull (2022).

Compound correlation is the correlation paradigm that is the most analogous to implied volatility. For a given tranche (A, B) with an observed spread and no upfront, compound correlation is the value of ρ that solves

$$\pi_{PL}^{A,B} = s^{A,B} \left(A_{SP}^{A,B} + A_{AP}^{A,B} \right).$$

If an upfront is used, compound correlation is the solution of

$$\pi_{PL}^{A,B} = U + s_*^{A,B} \left(A_{SP}^{A,B} + A_{AP}^{A,B} \right), \quad (5)$$

where $s_*^{A,B} = 0.01$. Compound correlation is simply the single correlation parameter that makes the model fair given the market quotes. Compound correlation unfortunately suffers from inconsistencies. For certain tranche quotes, the compound correlation does not exist in the interval $[0, 1)$. For example, Torresetti, Brigo, and Pallavicini (2006) show that for iTraxx S5, 10Y maturity the $(0.06, 0.09)$ tranche on 3 August, 2005 had no compound correlation under the Gaussian copula model. They also claim that for some tranche quotes there can even exist two values of compound correlation. The uncertain existence and uniqueness of compound correlations make the measure impractical. It is also common to observe a compound correlation skew, sometimes called the correlation smile. This means that the inverted correlation is different across tranches even though the model assumption is that there should only exist one value across all tranches. The compound correlation smile, in particular, is evidence that the Gaussian copula model is misspecified, and motivates the alternative double t copula model defined in the previous section.

3.4.2 Base Correlation

Base correlation was introduced to address the inconsistencies of compound correlation. The base correlation is based on the following decomposition of the time- t tranche loss on tranche (A, B) , $L_t(A, B)$ (Torresetti, Brigo, and Pallavicini 2006)

$$L_t(A, B) = \frac{\min[B - A, \max[L_t - A, 0]]}{B - A},$$

where L_t is the total portfolio loss. This can be rewritten to

$$L_t(A, B) = \frac{\max[L_t - A, 0] - \max[L_t - B, 0]}{B - A}.$$

We then introduce the time- t loss of an equity tranche $L_t(0, A)$.

$$\begin{aligned} L_t(0, A) &= \frac{\max[L_t - 0, 0] - \max[L_t - A, 0]}{A} \\ &= \frac{L_t - \max[L_t - A, 0]}{A}. \end{aligned}$$

This holds because portfolio losses are non-negative, $L_t \geq 0$. Rearranging, we get

$$L_t(0, A) \cdot A = L_t - \max[L_t - A, 0]. \quad (6)$$

We then add and subtract L_t in the numerator of $L_t(A, B)$

$$L_t(A, B) = \frac{\max[L_t - A, 0] - \max[L_t - B, 0] + L_t - L_t}{B - A},$$

and rearrange terms

$$L_t(A, B) = \frac{L_t - \max[L_t - B, 0] - (L_t - \max[L_t - A, 0])}{B - A}.$$

We recognize the numerator as the difference between the losses of two equity tranches with detachment points B and A . We insert equation 6

$$L_t(A, B) = \frac{L_t(0, B) \cdot B - L_t(0, A) \cdot A}{B - A}. \quad (7)$$

We have now shown that the time- t loss on a tranche (A, B) can be expressed with the losses on equity tranches with detachment points A and B , respectively.

The decomposition in equation 7 motivates assigning a separate correlation parameter to each equity tranche $(0, B)$ instead of assigning it to the tranche (A, B) . This correlation is called the base correlation, and it is defined as the value of ρ that prices the equity tranche $(0, B)$ consistently with the market observed price.

However, the approach carries a conceptual inconsistency. Because the two equity tranches $(0, A)$ and $(0, B)$ are each priced with different correlations and therefore with different default distributions, the resulting valuation of (A, B) is internally inconsistent. In particular, it cannot guarantee that model-implied expected tranche losses are monotonically increasing in the detachment point, and in some cases model-implied expected tranche losses can even turn negative (Torresetti, Brigo,

and Pallavicini 2006). This violates the standard no-arbitrage requirement imposed on financial markets. Despite this limitation, base correlation remains widely used in practice due to its tractability and the smoothness of the resulting correlation surface.

The algorithm of finding base correlations is the following.

Let the CDO consist of n tranches with detachment points $0 = B_0 < B_1 < \dots < B_n$, so that tranche i spans (B_{i-1}, B_i) . The base correlation is calculated iteratively for each $k = 1, \dots, n$ as follows (Hull 2022).

1. **Compound correlations.** For each tranche (B_{i-1}, B_i) , $i = 1, \dots, k$, solve for the compound correlation ρ_i by matching the model spread to the observed market spread.
2. **Tranche expected losses.** Using the compound correlation ρ_i , compute the present value of the expected loss on tranche (B_{i-1}, B_i) , $i = 1, \dots, k$ as a fraction of the total tranche notional. Denote this quantity π_{PL}^i .
3. **Cumulative equity tranche expected losses.** Accumulate the tranche expected losses to obtain the present value of the expected loss on the equity tranche $(0, B_k)$ as a fraction of the total portfolio notional:

$$L^{0, B_k} = \sum_{i=1}^k \pi_{PL}^i \cdot (B_i - B_{i-1})$$

4. **Equity tranche loss rate.** Normalize by the detachment point to obtain the expected loss rate of the equity tranche:

$$\pi_{PL}^{0, B_k} = \frac{L^{0, B_k}}{B_k}.$$

5. **Base correlation.** The base correlation ρ^{0, B_k} is the unique value of the correlation parameter that reproduces π_{PL}^{0, B_k} within the model. This root-finding step is performed iteratively for $k = 1, \dots, n$.

To solve for the compound and base correlation of tranches, we will use Brent's Method as described in the Appendix A.1.

Having defined all the building blocks of the Gaussian and double t copula models, we are ready to apply these models to data.

4 Data for the Experiment

The data we will be using to examine the models consist of a risk-free rate curve, index spreads, and tranche spreads and upfronts on the Markit iTraxx Europe Index S42.

The following information is from a Bloomberg Terminal. For a reference on this, we refer to Appendix A.4. The Markit iTraxx Europe Index consists of 125 equally weighted credit default swaps on investment grade European corporations. The index is divided into 4 sub-groups: Senior debt in Financials, Subordinated debt in Financials, Non-Financials, and High Volatility. The index components are chosen based on dealers' rankings of most liquid names, and a new series is rolled out every six months on March 20 and September 20. When a new series is rolled out, it has a maturity of 5.25 years, and thus has a maturity date of either June 20 or December 20.

For a risk-free rate curve, we will use the Euro Short-Term Rate (EUROSTR). A linear interpolation scheme has been used to get yields for maturities not observed. The risk-free rate curve is displayed below in Figure 2.

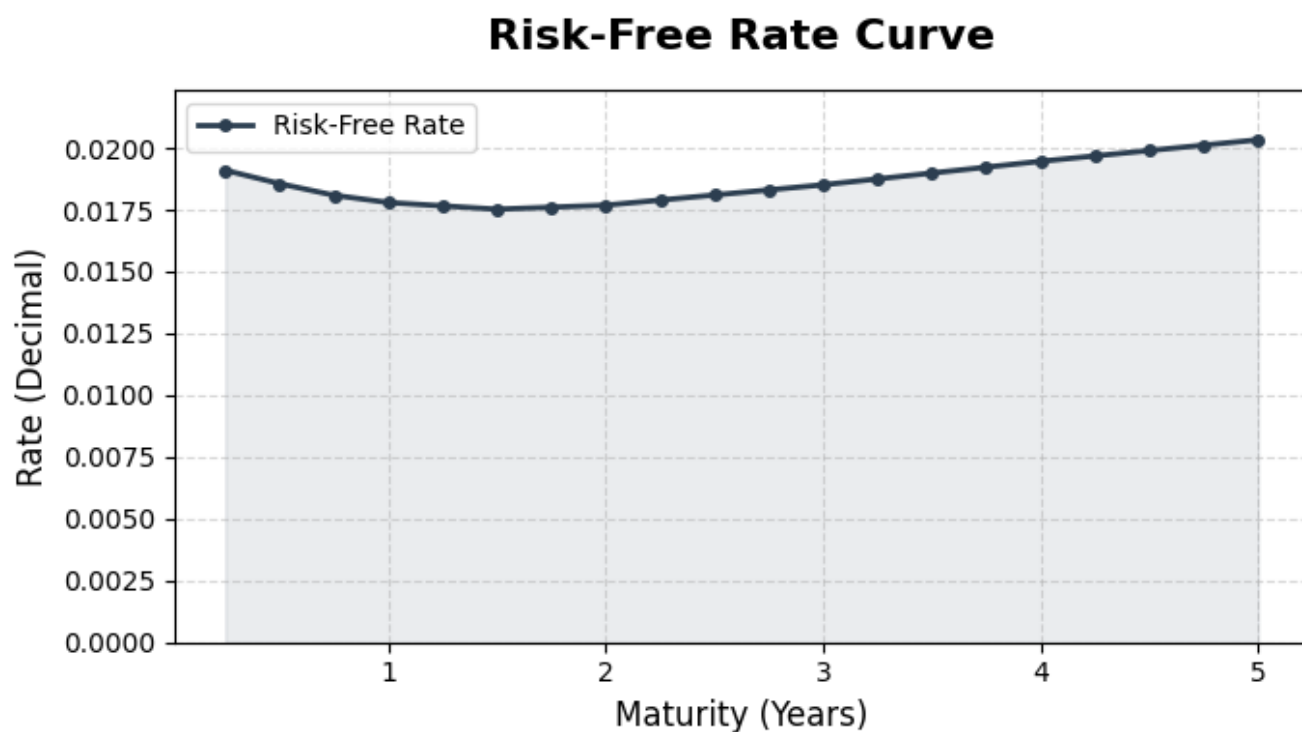


Figure 2: This graph shows the interest rate term structure that we will use for the experiment.

We will use the following tranches, spreads and upfronts of iTraxx S42 that were observed in the market on June 20, 2025 and sourced from a Bloomberg Terminal. Bloomberg also supplies their calculated base correlations for the tranches.

Tranche	Spread (bps)	Upfront	BBG Base Correlation
(0, 0.03)	100	0.26346	0.5874
(0.03, 0.06)	100	0.03927	0.6934
(0.06, 0.12)	98.06	0	0.8055
(0.12, 1)	26.78	0	N/A

Table 1: This table shows the tranche spreads and upfronts for iTraxx Europe Index S42 on June 20, 2025 as well as their base correlations calculated by Bloomberg.

This date was chosen because it is exactly 5 years before maturity. This avoids the need of handling

partial-period accrual at inception.

The Bloomberg base correlation for the senior tranche $(0.12, 1)$ is reported as not available. Since base correlations in practice tend to increase across detachment points, and the $(0.06, 0.12)$ base correlation is already 0.8055, the senior tranche base correlation would need to be higher still — and presumably exceeds the $[0, 1)$ range that the model admits, making it unattainable under the Gaussian copula model.

For index spreads, unfortunately we cannot observe the index spreads of S42 with many different maturities. We can only observe the index spreads for maturities 3Y and 5Y. Ideally, we would like to know multiple index spreads from now until the maturity of S42 5Y with maturities from 0Y to 5Y. However, the other maturities of S42 are rarely traded. Instead, the 5Y maturity index spreads of previous series are used. For example, to get the 4Y index spread for S42, we can use the 5Y index spread for S40 as a proxy, since it was rolled out exactly 1 year before S42. By using the previous series as proxies, we can thus get the full index spread curve. There is a slight caveat to this approach. If the portfolio has changed a lot over time, and with it the general credit quality, the index spreads would reflect this change. Since we are dealing with a high credit quality portfolio, we judge this possible change to be minor. However, it should be noted that this could introduce bias.

We use the following index spreads for different maturities to find the hazard rates shown in Table 2.

Maturity	Spread (bps)	Hazard Rate
0.5Y	3.789	0.000632
1Y	5.157	0.001090
1.5Y	11.080	0.003849
2Y	17.262	0.006034
2.5Y	23.929	0.008564
3Y	31.280	0.011579
3.5Y	38.651	0.014186
4Y	45.469	0.016035
4.5Y	48.514	0.012452
5Y	55.711	0.021041

Table 2: Implied hazard rates for a list of index spreads and maturities.

Using the bootstrapping method from Section 2.3.1, we estimate the piecewise constant hazard curve as well as the cumulative default probabilities over time. Figure 3 shows the hazard rates and cumulative default probabilities estimated from the CDS index spreads.

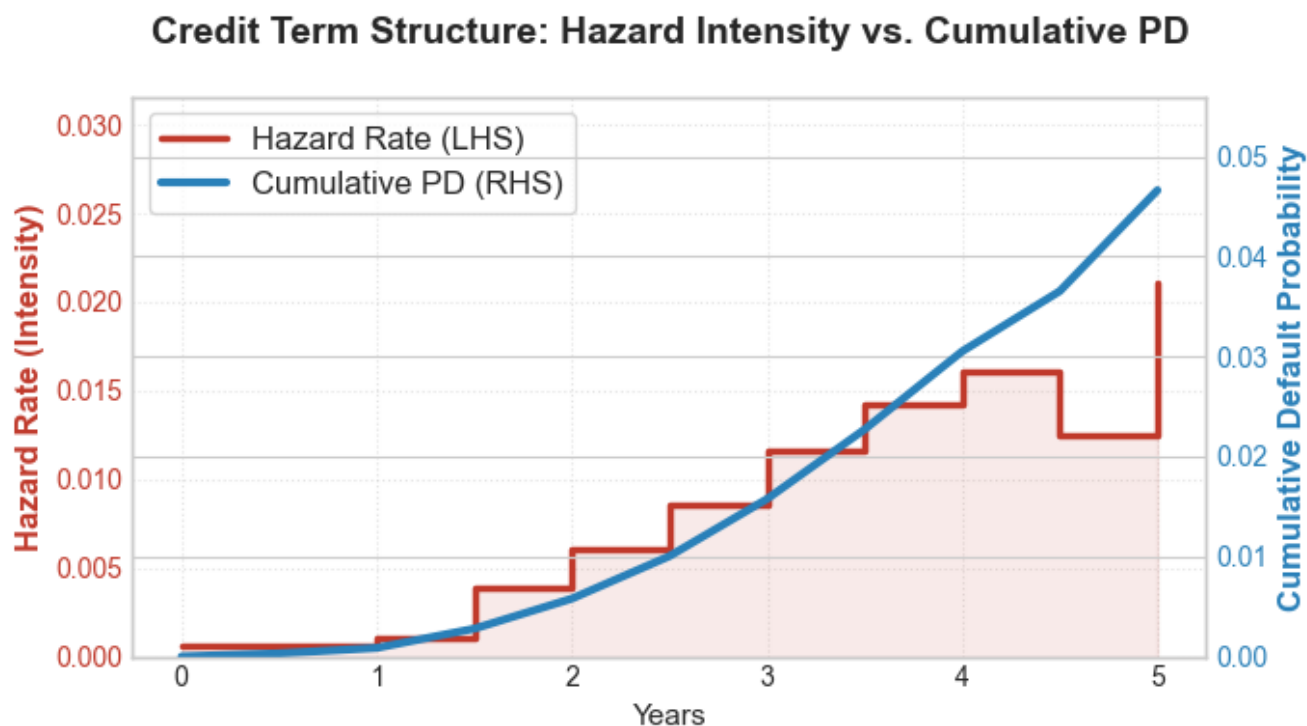


Figure 3: This graph shows the hazard rates and cumulative default probabilities estimated from the CDS index spread.

We note in Figure 3 that the hazard rate is not monotonically increasing over time. This implies that the probability of default for each year is in fact not increasing over time, and it seems that the probability of default in the interval $[4Y, 4.5Y)$ is lower than the probability of default in the preceding interval $[3.5Y, 4Y)$. This is because the 4.5Y spread of 48.514 bps is only a marginal increase over the 4Y spread of 45.469 bps.

In Figure 4, we plot the relationship between the cumulative default probabilities bootstrapped using the method in Section 2.3.1 and the CDS spread.

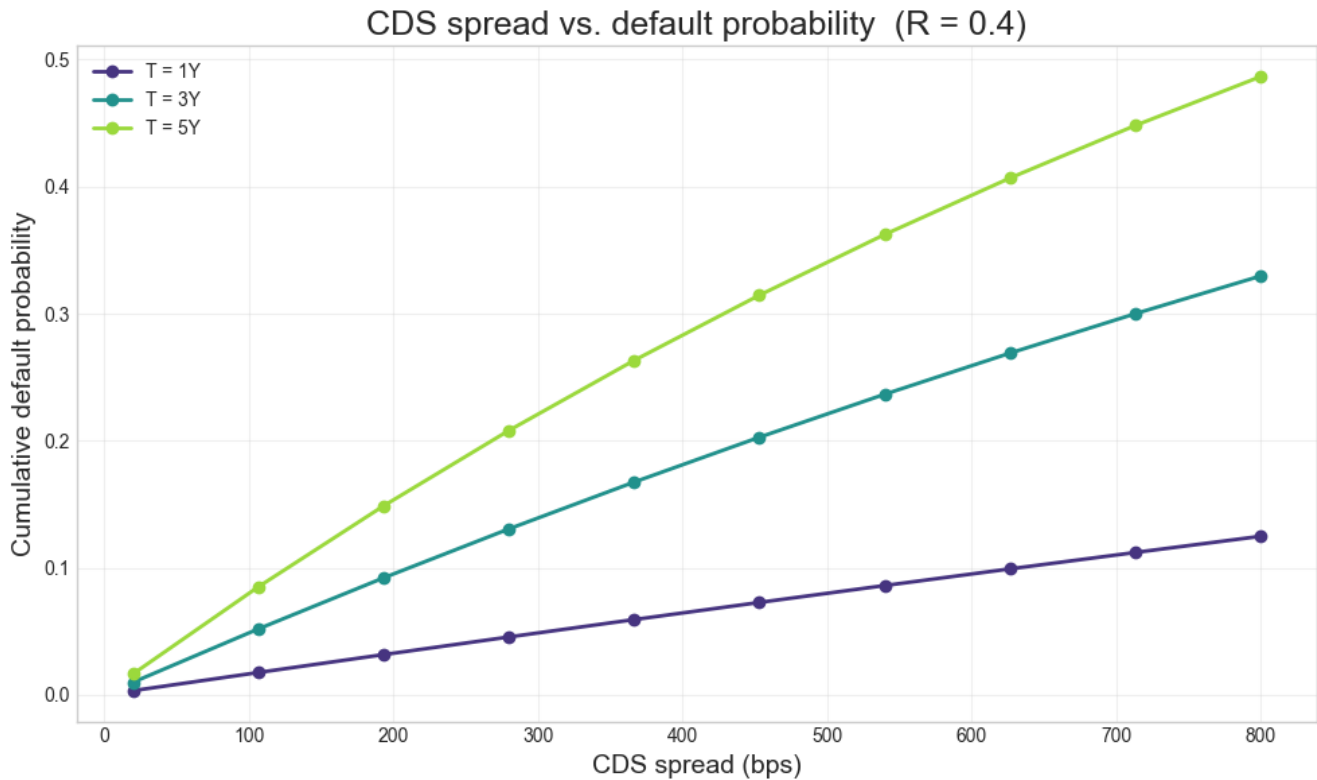


Figure 4: This plot shows the cumulative default probabilities implied by the bootstrap of Section 2.3.1 for ten flat CDS spread curves between 20 bps and 800 bps, quoted at every half-year maturity from 0.5Y to 5Y. Probabilities are shown at maturities 1Y, 3Y, and 5Y, with the recovery rate fixed at $R = 0.4$.

Using flat CDS spread curves (constant across maturities) and $R = 0.4$, we see that the relationship between the CDS spread and the cumulative default probability is concave at each maturity. The 1Y curve is nearly linear over the plotted range, while the 3Y and 5Y curves exhibit progressively stronger concavity. This is because the cumulative default probability is given by $1 - \exp\left(-\int_0^T h(s) ds\right)$ which saturates at 1, so the marginal effect of a higher spread diminishes as the cumulative hazard grows.

In the next section, we apply the models to produce compound correlations and base correlations based on the market data. We also examine the tail dependence of both models as well as look at the fair spreads and upfronts generated by the models for different values of ρ .

5 Numerical Results

Both the Gaussian copula model and the double t copula model have been coded in Python and the code is in the Appendices A.5 and A.6, respectively.

In this section, we will assume that the portfolio is homogeneous and that the recovery rate is $R = 0.4$. For the double t copula model, we will assume that the systematic and idiosyncratic factors follow a t distribution with $\nu = 4$. The choice of degrees of freedom was found by Hull and White (2004) to fit market prices reasonably well. We have performed convergence checks of the number of integration points in Gauss-Hermite n_{herm} , and the grid size N and grid range $(-L, L)$ in the IFFT algorithm. The convergence checks were performed on the fair tranche spreads and upfronts using $\rho = 0.5$ and $\nu = 4$ (for the double t copula model) and are listed in the Appendix A.3. We find that for the Gaussian copula model, the fair tranche spreads and upfronts converge around $n_{herm} = 140$. For the double t copula model, the convergence happened around $n_{herm} = 200$ and we used $N = 2^{16}$ and $L = 30$. To ensure comparability, we choose to use 200 integration points for the Gauss-Hermite and adaptive Gauss-Hermite quadratures in the models. The choice of the number of integration points is a trade-off between accuracy and computation time. The grid size convergence in the IFFT algorithm was found using $n_{herm} = 200$ and $L = 30$, and the convergence occurred around $N = 2^{16} = 65536$. The convergence of the range used for IFFT was found using $n_{herm} = 200$ and $N = 2^{16}$. The convergence of fair tranche spreads and upfronts occurred around $L = 15$ across tranches. Therefore, we choose the parameter values $n_{herm} = 200$, $N = 2^{16}$ and $L = 15$ for building the models.

In the following sections, we first examine the tail dependence of the two copulas, since this property drives most of the pricing differences between the models. We then show how tail dependence reshapes the default distributions of the underlying portfolio. Building on these results, we examine the correlations implied by the models — first compound correlation, then base correlation — using the observed tranche spreads and upfronts. Lastly, we examine the fair spreads and upfronts produced by the models and their sensitivity to ρ .

5.1 Tail Dependence in the Models

We define two standard normal random variables $X_1 = \sqrt{\rho}S_{Gauss} + \sqrt{1-\rho}Z_{Gauss,1}$ and $X_2 = \sqrt{\rho}S_{Gauss} + \sqrt{1-\rho}Z_{Gauss,2}$, where $S_{Gauss}, Z_{Gauss,1}, Z_{Gauss,2}$ are independent standard normal random variables. From these we can construct bivariate copulas and examine their tail behaviour. Similarly, we define two random variables constructed from independent t distributed random variables with ν degrees of freedom, $Y_1 = \sqrt{\rho}\sqrt{\frac{\nu-2}{\nu}}S_t + \sqrt{1-\rho}\sqrt{\frac{\nu-2}{\nu}}Z_{t,1}$ and $Y_2 = \sqrt{\rho}\sqrt{\frac{\nu-2}{\nu}}S_t + \sqrt{1-\rho}\sqrt{\frac{\nu-2}{\nu}}Z_{t,2}$, where $S_t, Z_{t,1}$, and $Z_{t,2}$ are independent t distributed random variables with ν degrees of freedom. We convert X_i and Y_i to uniform variables by applying their CDF. For the Gaussian model, we apply the standard normal CDF, and for the double t copula, we apply the resulting CDF outlined in 3.3. We define the resulting uniform variables by

$$\begin{aligned} U_{Gauss,1} &= \Phi(X_1) \quad \text{and} \quad U_{Gauss,2} = \Phi(X_2), \\ U_{Doublet,1} &= F_{Y_1}(Y_1) \quad \text{and} \quad U_{Doublet,2} = F_{Y_2}(Y_2) \end{aligned}$$

In the below plot, we examine the extreme upper and lower tails of the resulting bivariate copulas $C_{Gauss}(U_{Gauss,1}, U_{Gauss,2}, \rho = 0.5)$ and $C_{Doublet}(U_{Doublet,1}, U_{Doublet,2}, \rho = 0.5)$.

Tail Dependence Comparison ($\rho=0.5$)

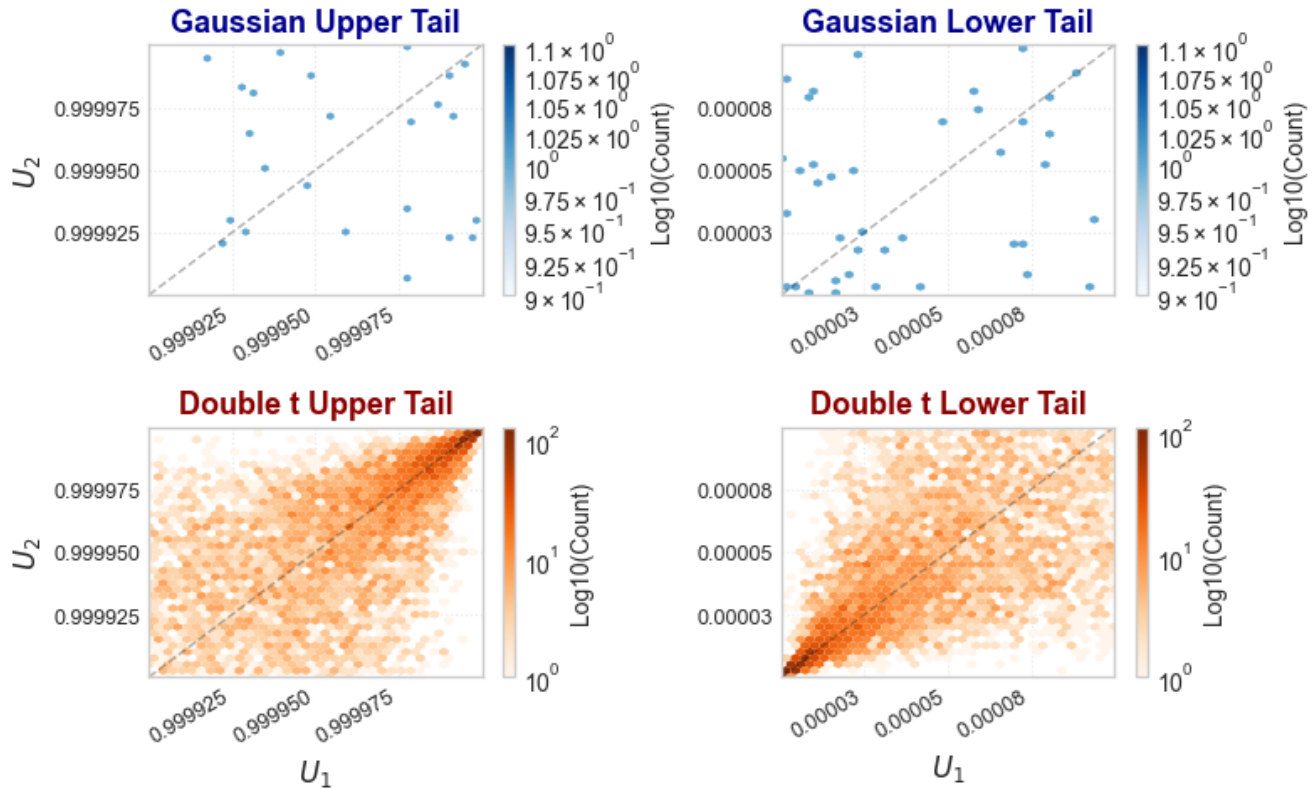


Figure 5: This plot shows the upper and lower tail dependence of the Gaussian copula and the double t copula for $\rho = 0.5$. The axes show the extreme quantiles of the $[0, 1]$ variables. For each copula, 10,000,000 samples were generated.

In Figure 5, we see the upper and lower tails of the Gaussian and double t copulas for $\rho = 0.5$. It is evident that the tail behaviour of the two copulas is different. The Gaussian copula shows very limited tail clustering whereas the double t copula clearly shows clustering in the tails. This matches the theoretical prediction. We know from Section 3.1.3 that the Gaussian copula has lower and upper tail dependence coefficients of 0. The t copula on the other hand has lower and

upper tail dependence coefficients of

$$\lambda_\ell = \lambda_u = 2F_{t,\nu+1} \left(-\sqrt{\frac{(\nu+1)(1-\rho)}{1+\rho}} \right), \quad (8)$$

which for $\nu = 4$ and $\rho = 0.5$ evaluates to 0.25317.

We would like to also estimate the tail dependence coefficients of the double t copula. In the plot below, we have made an effort to estimate the tail dependence of the Gaussian, t , and double t copulas numerically.

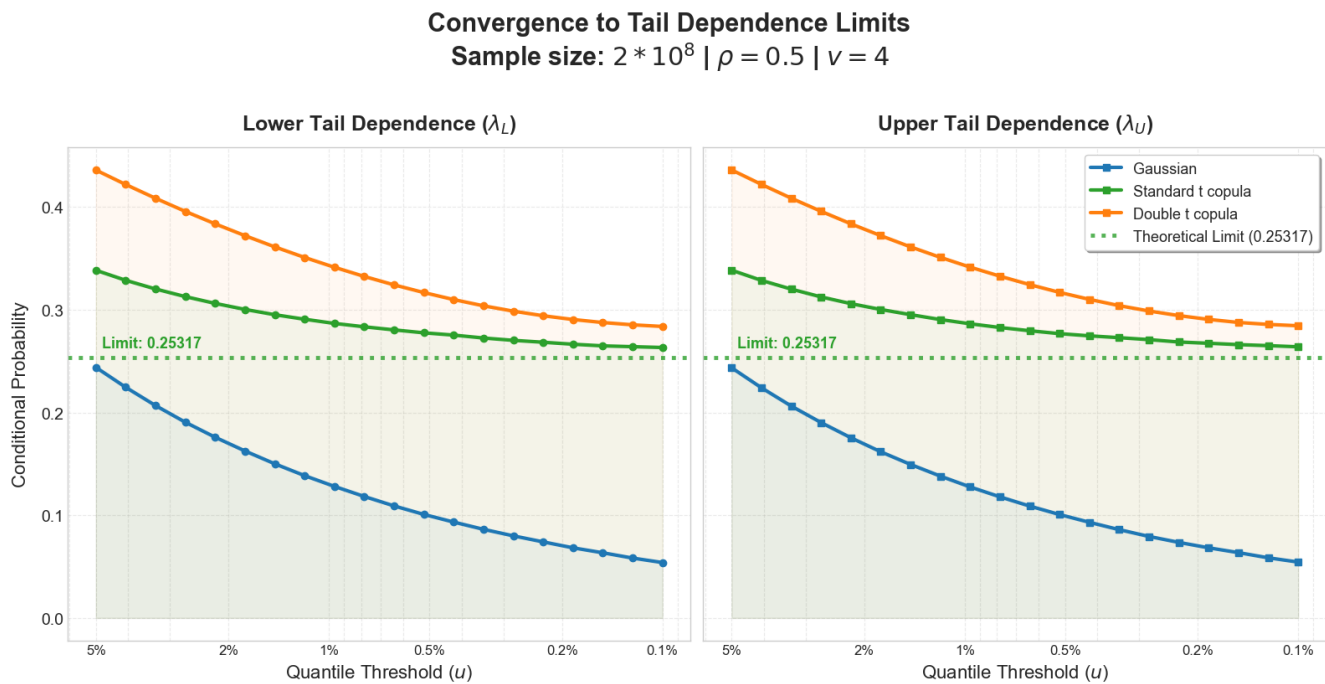


Figure 6: Convergence of lower and upper tail dependence coefficients for the Gaussian, t , and double t copulas for $\rho = 0.5$ and $\nu = 4$.

The convergence experiment shown in Figure 6 confirms the tail dependence in the double t copula. It even seems to be higher than that of the t copula for $\rho = 0.5$ and $\nu = 4$. However, the experiment does not reveal the behaviour in the extreme tails. This is because the sample size of the experiment limits how far out in the tail we can go while still getting stable results. Generating more samples was limited by the RAM capacity of the author's computer.

For the t copula, the theoretical tail dependence coefficient at $\nu = 4$, $\rho = 0.5$ is 0.25317; our simulated estimate at the 0.1% quantile reaches 0.26333, close to the theoretical value. The double t copula simulation gives 0.28381 at the same quantile — slightly higher, suggesting the double t copula has marginally stronger tail dependence than the t copula for these parameter values. We also observe that the double t copula seems to have symmetric tail dependence. However, it is important to clarify that the near-convergence values of the double t copula tail dependence coefficients are not the true values, and therefore should not be treated as such.

The existence of tail dependence for the double t copula model displays that not only are extreme marginal observations more likely compared to the Gaussian copula model, but extreme joint observations are also more likely. Since this tail clustering is caused by S , this means that when S attains an extreme value, both X_1 and X_2 are more likely to also attain extreme values in the double t copula model, whereas this is not the case in the Gaussian copula model. The tail dependence therefore should lead to increased tail risks, causing the fair spreads and upfronts of the most junior and most senior tranches to increase.

Next, we increase the tail dependence of the models using $\rho = 0.9$.

Tail Dependence Comparison ($\rho=0.9$)

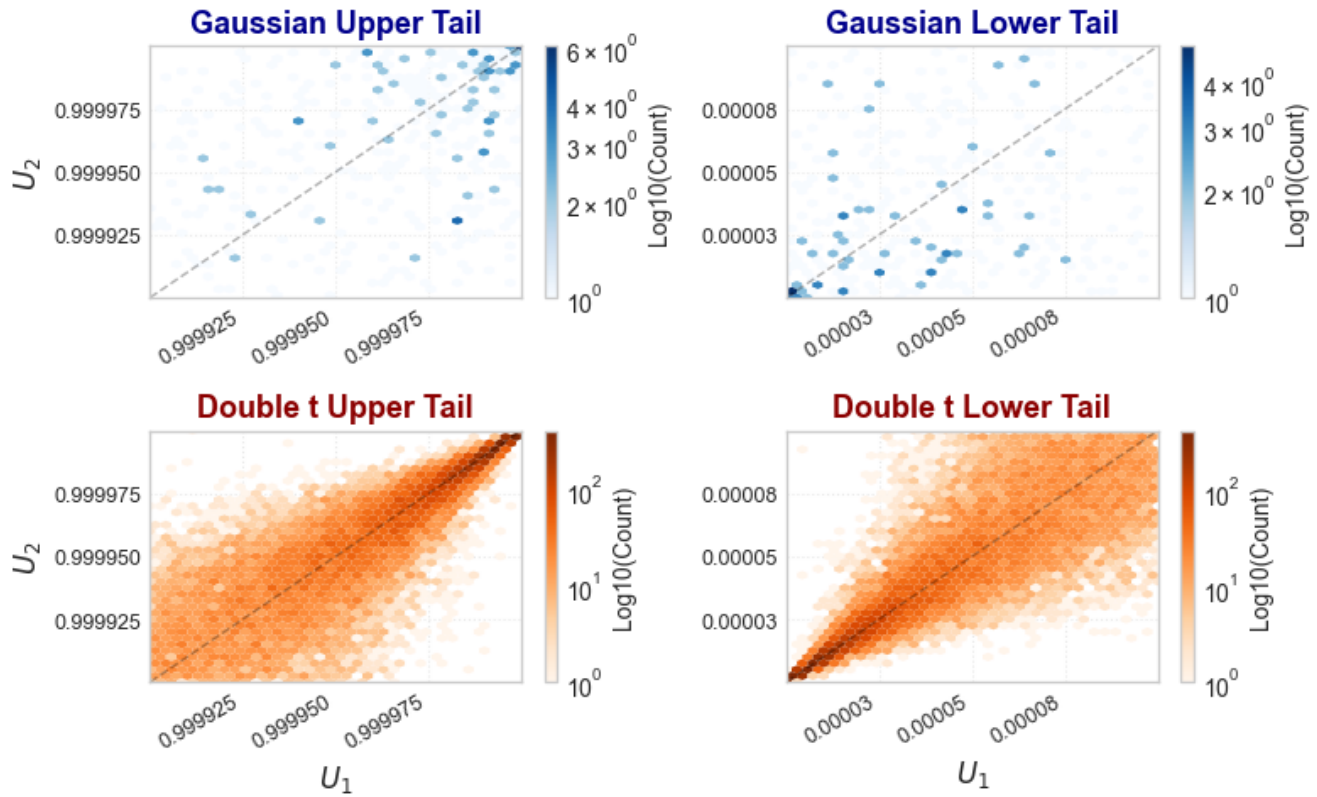


Figure 7: This plot shows the upper and lower tail dependence of the Gaussian copula and the double t copula for $\rho = 0.9$. The axes show the quantiles of the $[0, 1]$ variables. For each copula, 10,000,000 samples were generated.

In Figure 7 where $\rho = 0.9$, the tail dependence of the double t copula is even more pronounced, meaning that extreme joint events are even more likely for high values of ρ . This is because the systematic factor S that causes the clustering is weighted higher for higher values of ρ .

In the next section, we examine how the tail dependence in the double t copula translates into a different default distribution for the double t copula model compared to the Gaussian.

5.2 Default Distributions of the Models

Both the Gaussian copula model and the double t copula produce implied cumulative default distributions for the portfolio at time τ_j . They do this by integrating the conditional probabilities of m defaults at time τ_j

$$P(m, \tau_j | S) = \frac{M!}{m!(M-m)!} Q(\tau_j | S)^m (1 - Q(\tau_j | S))^{M-m}$$

over the systematic factor S .

Given the fixed recovery rate of $R = 0.4$, we can calculate the number of defaults needed to wipe out each tranche's principal. The equity tranche gets wiped out if losses exceed the tranche principal of $125 \cdot 0.03 = 3.75$. This means that if at least $\lceil \frac{3.75}{0.6} \rceil = \lceil 6.25 \rceil = 7$ reference entities default during the duration of the contract, the equity tranche principal is wiped out. The $(0.03, 0.06)$ tranche is hit when losses exceed 3.75 and is completely wiped out when losses exceed $125 \cdot 0.06 = 7.5$ which happens after $\lceil \frac{7.5}{0.6} \rceil = \lceil 12.5 \rceil = 13$ defaults. Lastly, the $(0.06, 0.12)$ tranche gets hit when losses exceed 7.5 and has its tranche principal wiped out after losses exceed $125 \cdot 0.12 = 15$ which happens after $\lceil \frac{15}{0.6} \rceil = \lceil 25 \rceil = 25$ defaults. With a recovery rate of $R = 0.4$, the $(0.12, 1)$ tranche never gets completely wiped out. If every reference entity defaults, the total portfolio loss is 60%, and the senior tranche can incur maximum losses of $\frac{0.6-0.12}{1-0.12} \approx 54.5\%$.

In the below plot, we have plotted the default distribution at maturity $T = 5$ for both models for different values of ρ . We have also plotted dotted lines to indicate when each tranche's principal gets wiped out.

Impact of Correlation on Loss Distribution

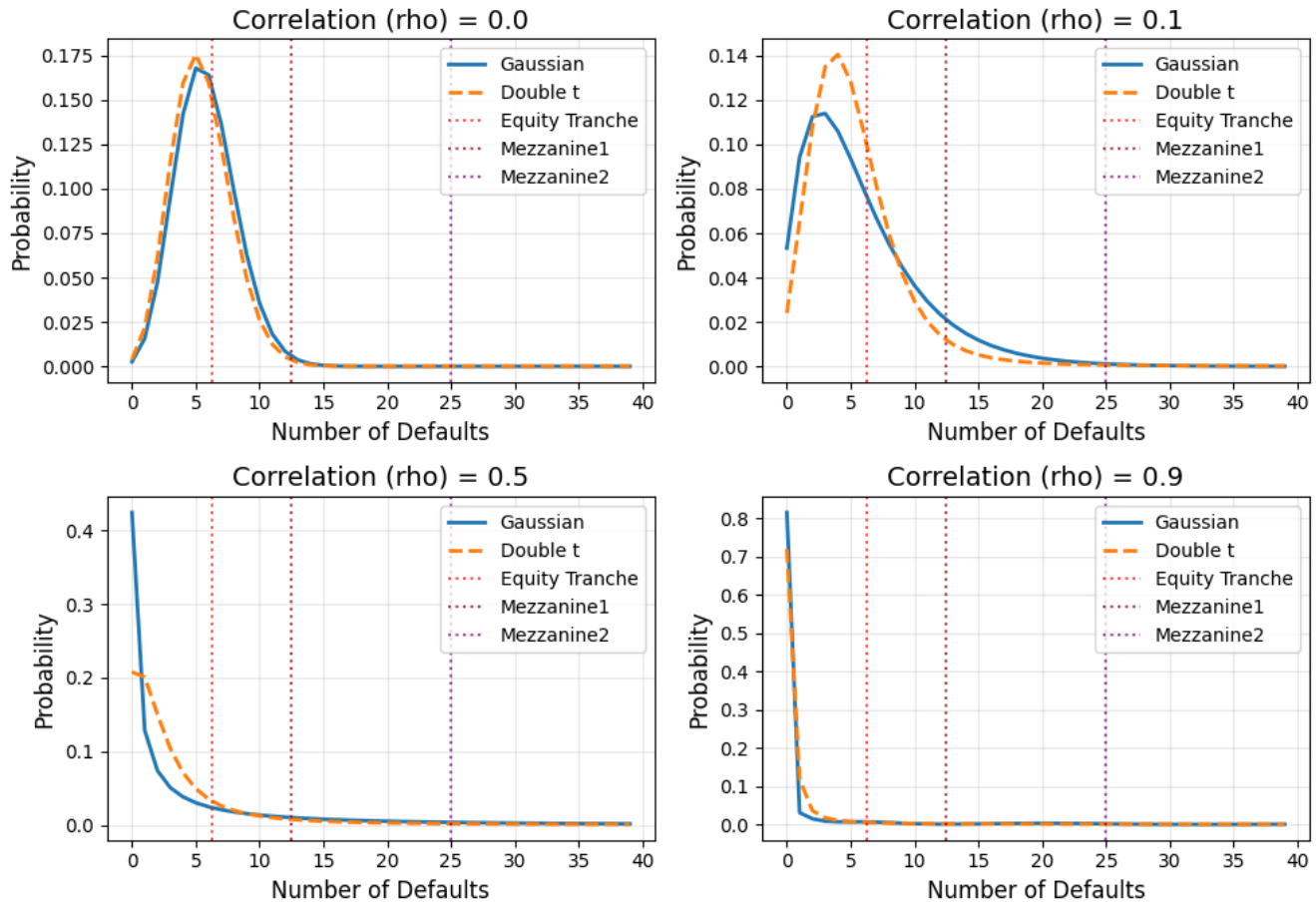


Figure 8: This plot shows the probability distribution of the number of defaults for both the Gaussian copula model and the double t copula model for different values of ρ at maturity $T = 5$. It also indicates the number of defaults needed for each tranche to be wiped out.

In Figure 8, we observe that for $\rho = 0$ there is a high probability of observing around 5 defaults for both models. We also note that the implied default distributions of the two models are nearly identical in shape. When correlation is increased to $\rho = 0.1$, the Gaussian model's probability distribution shifts its mode from 5 to 3 while also flattening out slightly, whereas the double t model's probability distribution only shifts slightly to the left. When correlation is increased to $\rho = 0.5$, the implied probability distributions of both models shift to the left, with both having a

mode of 0. Lastly, increasing the correlation to $\rho = 0.9$ shifts the distribution even more to the left. The probability of observing at least one default becomes very small for the Gaussian model, while the double t model retains slightly more probability mass for one or more defaults. This is due to the tail dependence of the double t copula, which makes the joint probability of extreme events higher in the double t model.

This shows that the value of the correlation parameter ρ in the generation of the default variable X_i plays a significant role in the distribution of defaults for both models. For $\rho = 0$, the copulas are independence copulas causing the defaults to simply be driven by the marginal cumulative default probabilities estimated from the index spreads. This makes the theoretical implied default distributions of the models equal for $\rho = 0$. The slight differences arise from the additional numerical procedures used in the double t copula model to recover the distribution of the default variable, which introduce small numerical inaccuracies. To sum up, we see a pattern that the Gaussian copula model's default distribution is more sensitive to the increases in ρ and more quickly shifts to the left.

From the default distributions, we can find the CDFs of observing at least n defaults at maturity. We have plotted the CDFs for different values of ρ in the plot below.

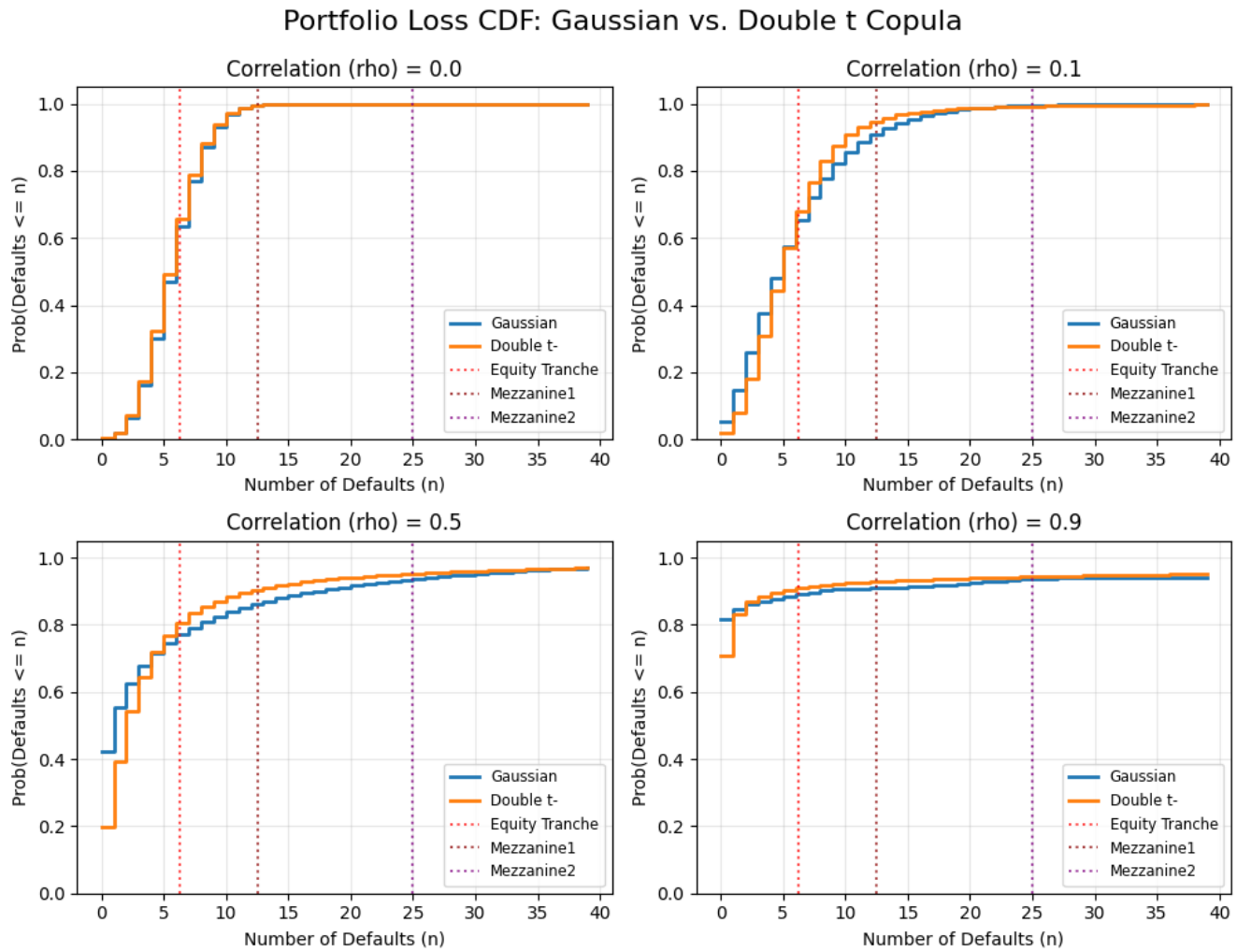


Figure 9: This plot shows the cumulative probability distributions of defaults at maturity $T = 5$ for both the Gaussian copula model and the double t copula model for different values of ρ . It also indicates the number of defaults needed for each tranche to be wiped out.

We see that the probability of observing fewer than 4 defaults is higher for the Gaussian copula across all the sampled values of ρ except $\rho = 0$, where the distributions are virtually equal. We have consolidated the probabilities of tranche wipeouts across models and values of ρ in the table below.

ρ	Tranche	Defaults for Tranche Wipeout	$P(\text{wipeout})$ Gaussian	$P(\text{wipeout})$ Double t
0	(0, 0.03)	7	0.229	0.211
0	(0.03, 0.06)	13	0.002	0.002
0	(0.06, 0.12)	25	0	0
0.1	(0, 0.03)	7	0.279	0.235
0.1	(0.03, 0.06)	13	0.071	0.041
0.1	(0.06, 0.12)	25	0.004	0.007
0.5	(0, 0.03)	7	0.208	0.166
0.5	(0.03, 0.06)	13	0.130	0.089
0.5	(0.06, 0.12)	25	0.062	0.047
0.9	(0, 0.03)	7	0.103	0.085
0.9	(0.03, 0.06)	13	0.090	0.070
0.9	(0.06, 0.12)	25	0.063	0.056

Table 3: This table shows the probability of tranche wipeout for the equity tranche and the two mezzanine tranches for different values of ρ for the Gaussian copula model and the double t copula model at maturity $T = 5$.

Table 3 shows the probability of the principal of each tranche being wiped out for the distributions plotted in Figure 9.

The reshaped default distribution in the double t copula model — with more mass at both very low and very high default counts — is what ultimately drives the pricing differences between the two models. In the next section, we examine how this property manifests in the compound correlations implied by the observed tranche spreads and upfronts.

5.3 Compound Correlation

From the spreads and upfronts, we can find the compound correlation, that is, the value of ρ that solves the model for the observed tranche spread and upfront. The computed compound

correlations for both models are shown in Table 4.

Tranche	Gaussian Compound Correlation	Double t Compound Correlation
(0, 0.03)	0.5792	0.6537
(0.03, 0.06)	0.9288	0.8006
(0.06, 0.12)	0.1655	0.3204
(0.12, 1)	0.7632	0.6836

Table 4: This table lists the calculated compound correlations for the Gaussian and double t copula models for iTraxx S42 5Y tranches on June 20, 2025.

The values are also shown in Figure 10.

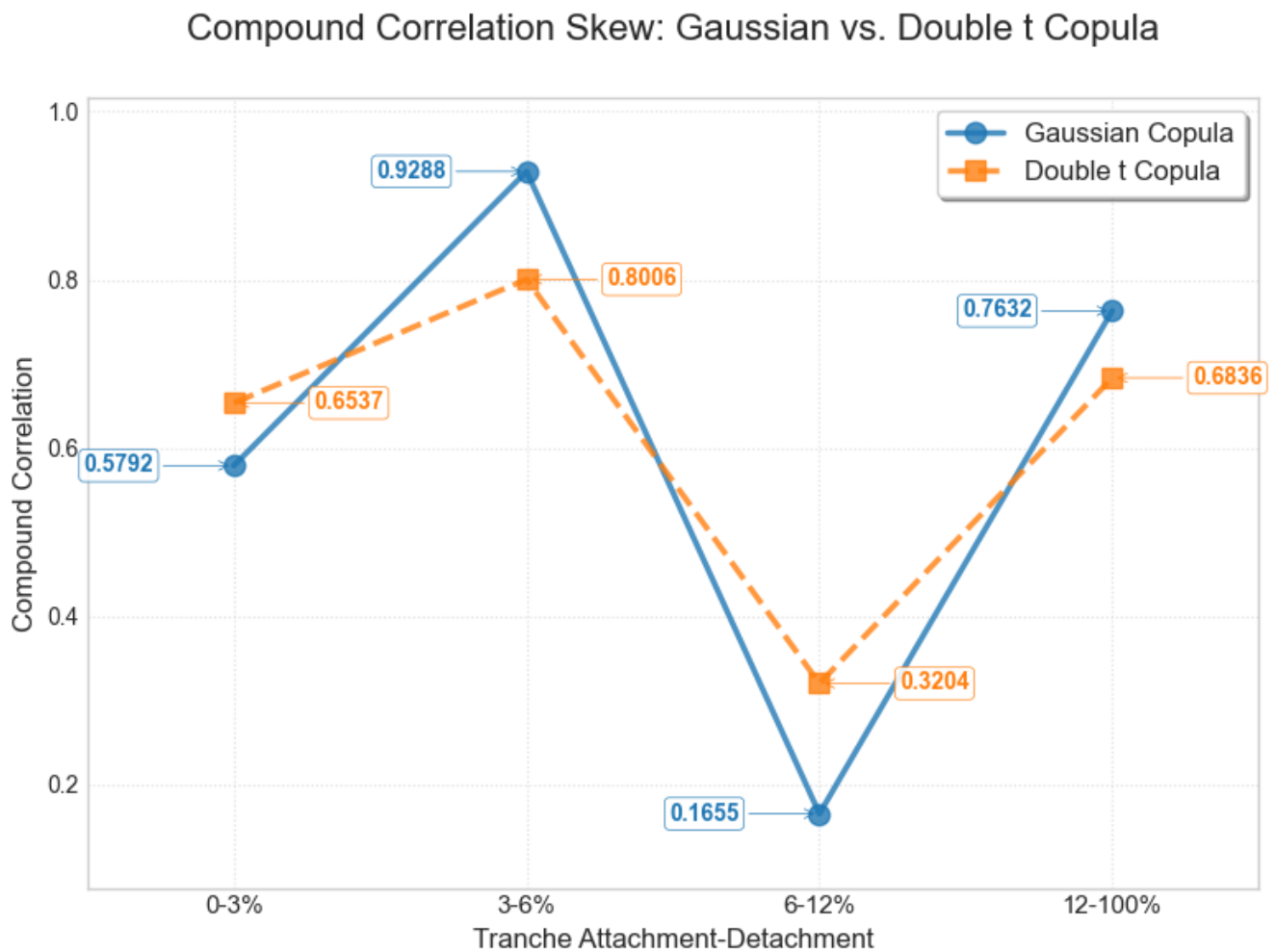


Figure 10: This plot shows the compound correlation calculated for different tranches using a Gaussian copula model and a double t copula model for iTraxx S42 5Y tranches on June 20, 2025.

Figure 10 shows that both the Gaussian copula model and the double t copula model have significant correlation smiles. This essentially means that when calculating fair spreads and upfronts for each tranche, we have to vary the value of ρ in the models significantly to match the observed spreads and upfronts. Had the models been accurate, the value of ρ would have been constant across all tranches, since under the homogeneity assumption there should be a single value of ρ that is consistent with the observed spreads and upfronts.

The range of the compound correlation is $0.93 - 0.17 = 0.76$ for the Gaussian copula model and

$0.80 - 0.32 = 0.48$ for the double t copula model. The correlation smile is therefore flatter in the double t copula model than in the Gaussian copula model. Since an accurate model would have no correlation smile and, therefore, would have compound correlation consistent across tranches, the double t model having a flatter smile is indicative of the model being a better fit. However, the correlation smile is not eliminated in the double t copula model.

The smaller correlation range of the double t copula model is due to the tail dependence demonstrated in Section 5.1. Tail dependence pushes the probability mass of the joint default distribution towards the extreme tails similarly to how increasing the correlation parameter of the model does it. Conversely, the only way for the Gaussian copula model to compensate for the lack of tail dependence is through increasing the correlation parameter, which shifts the loss distribution as shown in Figure 8. This results in the double t copula having a less pronounced smile.

One thing we must point out is the fact that the compound correlation exists for all tranches and that the values are unique. As laid out in Section 3.4, this is not guaranteed in general. Brent's Method was applied over the full interval $\rho \in [0, 1)$ and returned a unique solution for each tranche.

In the next section, we examine the base correlation of the models for the observed tranche spreads and upfronts.

5.4 Base Correlation

As with compound correlation, we can find the values of the base correlations for the tranches using the observed tranche spreads and upfronts. We can then compare these figures with those of Bloomberg. This is what we have done in the table below.

Tranche	BBG Base Correlation	Gaussian Base Correlation	Double t Base Correlation
(0, 0.03)	0.5874	0.5792	0.6537
(0.03, 0.06)	0.6934	0.6815	0.6772
(0.06, 0.12)	0.8055	0.7864	0.7160
(0.12, 1)	N/A	N/A	N/A

Table 5: This table lists the calculated base correlations for the Gaussian and double t copula models for iTraxx S42 5Y tranches on June 20, 2025 as well as Bloomberg's values.

We see that the base correlations calculated by the Gaussian copula model match the values from Bloomberg within approximately 0.02 across all tranches. Bloomberg uses a Gaussian copula model, which is the industry standard. Bloomberg's whitepaper on pricing CDO tranches can be found on the Bloomberg Terminal with the command `LPHP CDST:0:1 3124184 <G0>`, and does not have a public URL. Bloomberg uses a random recovery assumption, where the recovery rate R is treated as a random variable instead of a fixed value. This likely causes the discrepancies. We also see that for the senior tranche with detachment point $B = 1$, the base correlation does not exist within $[0, 1)$ for either model. This is consistent with the typically monotonically increasing base correlation curve approaching the upper bound $\rho = 1$ already at the $B = 0.12$ tranche (0.7864 for the Gaussian model), making it unattainable for the senior tranche under either model.

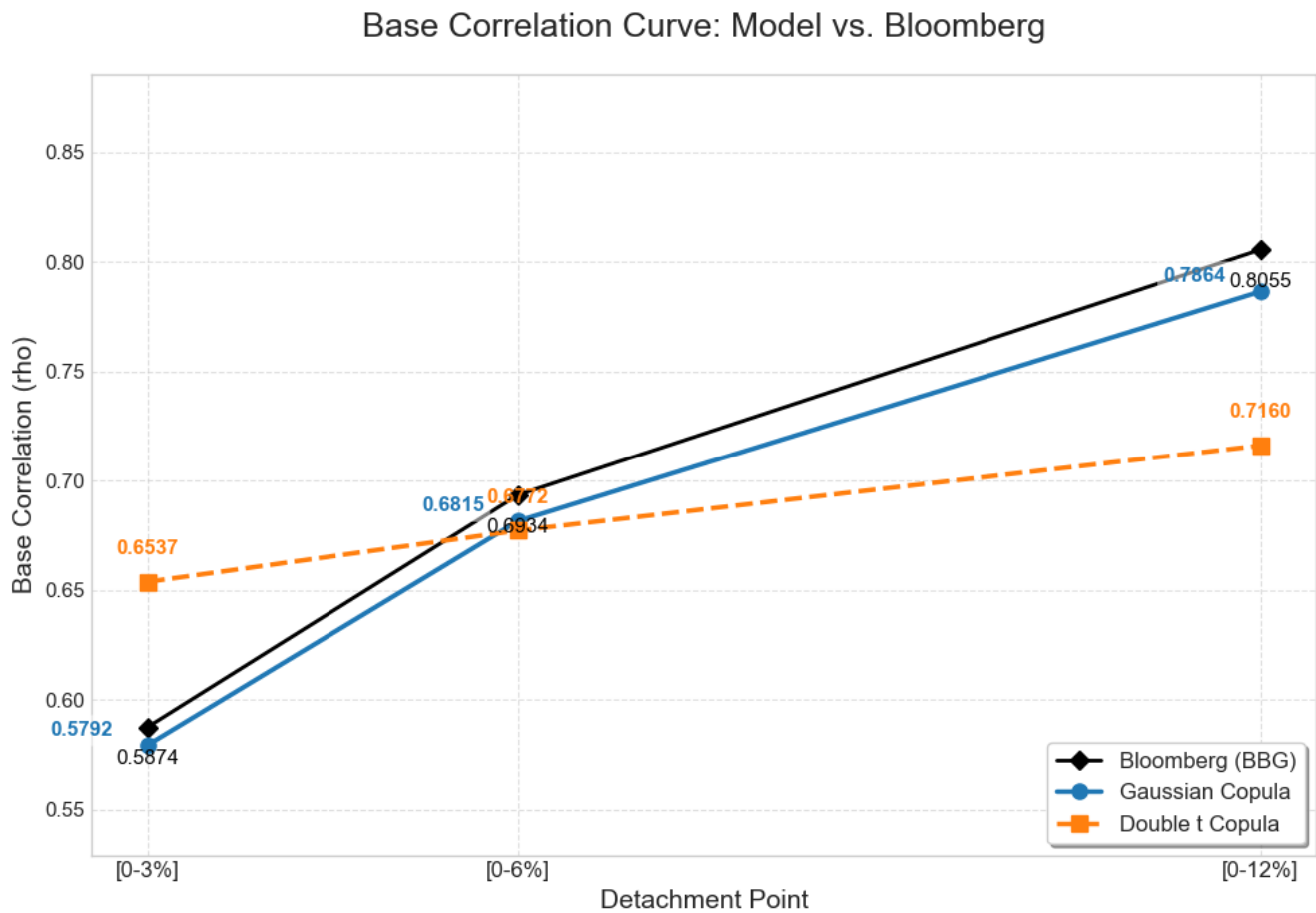


Figure 11: Base Correlation curve for the tranches of the CDO excluding the senior tranche for iTraxx S42 5Y tranches on June 20, 2025.

Figure 11 plots the values of Table 5. We see that the values from Bloomberg and our Gaussian copula model match reasonably well. We also see that the curve for the double t copula model is significantly flatter. The more senior tranches are priced largely by the likelihood of extreme events. This means that to match the observed tranche spreads and upfronts in the Gaussian copula model, ρ has to compensate for the unmatched likelihood of extreme events. The double t copula model, on the other hand, introduces increased likelihoods of extreme events through tail dependence, as shown in Section 5.1 and reflected in the default distributions of Figure 8. This means that the value of ρ does not have to vary as much to match the observed spreads and upfronts.

Both the flatter base correlation curve and the flatter compound correlation skew of the double t copula model can be traced back to its tail dependence and the resulting reshaping of the default distribution. We now turn to the fair spreads and upfronts produced by both models and examine how the differences in default distributions translate into tranche-level pricing.

5.5 Fair Spreads and Upfronts

For both models and a value of ρ , we can find the fair spread and upfront for each tranche. A typical convention is to fix the spread for the (0,0.03) and (0.03,0.06) tranches to 100 bps and then match the expected loss with an upfront. For simplicity's sake, we will assume that for each tranche, if the fair spread is greater than 100 bps, we will cap the spread at 100 bps and match the expected loss with an upfront.

$\rho = 0.5$	Spread (bps)		Upfront	
Tranche	Gaussian	Double t	Gaussian	Double t
0–3%	100	100	0.3084	0.3646
3–6%	100	100	0.1239	0.0730
6–12%	100	100	0.0425	0.0110
12–100%	12.72	17.18	0	0

Table 6: Fair spreads and upfronts at $\rho = 0.5$ for both models.

In Table 6, we see that the fair spreads and upfronts produced by both models vary. We will now dive into what causes these differences. From Section 2.5, we know that the fair tranche spread is given by

$$s^{A,B} = \frac{\pi_{PL}^{A,B}}{A_{SP}^{A,B} + A_{AP}^{A,B}}.$$

As in Section 2.5, we call $\pi_{PL}^{A,B}$ the protection leg and $s^{A,B} (A_{SP}^{A,B} + A_{AP}^{A,B})$ the payment leg.

First, we calculate the expected present values of the protection leg and the payment leg for all tranches for both models using $\rho = 0.5$.

$\rho = 0.5$	Payment Leg			Protection Leg		
Tranche	Gaussian	Double t	% Deviation	Gaussian	Double t	% Deviation
(0, 0.03)	4.0885	4.0396	-1.20%	0.3493	0.4049	15.9%
(0.03, 0.06)	4.4999	4.6068	2.32%	0.1689	0.1191	-29.5%
(0.06, 0.12)	4.6371	4.6816	0.96%	0.0889	0.0578	-35.0%
(0.12, 1)	4.7515	4.7460	-0.12%	0.0060	0.0082	36.7%

Table 7: This table lists the calculated expected present values of the protection and payment legs for the Gaussian and double t copula models as well as the deviation of the double t copula model values from the Gaussian copula model values using $\rho = 0.5$.

Looking at Table 7, we see that for $\rho = 0.5$ the present value of the payment leg for the Gaussian copula model is very similar to that of the double t copula model for all tranches. The present value of the protection leg differs substantially more between the two models. We can thus determine that the fair spreads and upfronts of the two models with $\rho = 0.5$ differ largely because of the present values of the protection leg. This is expected. In the double t copula model, we would expect a higher probability of having a few defaults and simultaneously a higher probability of having many defaults. This is due to the heavy tails of the t distribution and the tail dependence feature of the resulting copula. This exposure to the tails is precisely why the equity and senior tranches are priced higher in the double t copula model — the risk of the equity tranche is increased due to the increased probability mass on few defaults, while the senior tranche is exposed to the increased probability mass on many defaults. The mezzanine tranches, sitting in the middle of the default distribution, are less affected by what happens in either tail and see reduced protection leg values because the probability distribution is shifted away from the middle as shown in Figure 9. This leads to lower spreads and upfronts in the double t model. In Table 3 we observe that the probability of wipeout of the equity tranche is lower for $\rho = 0.5$ in the double t model compared to the Gaussian. At first glance, this should mean lower risk of the equity tranche and therefore lower spreads and upfronts. However, we observe an increased upfront of the equity tranche in the double t copula model. Looking at Figure 8, it becomes obvious that while the default distribution

in the double t copula model has a lower chance of tranche wipeout for the equity tranche, it has a higher chance of having just a few defaults leading to a higher tranche upfront.

Next, we perform a sensitivity analysis of fair spreads and upfronts to changes in ρ .

5.5.1 Sensitivity of Spreads and Upfronts to Changes in ρ

Calculating the fair tranche spreads and upfronts using both models for different values of ρ , we can examine the sensitivity of spreads and upfronts to changes in ρ . In the below plot, we calculate the fair spreads for both models using four different values of ρ .

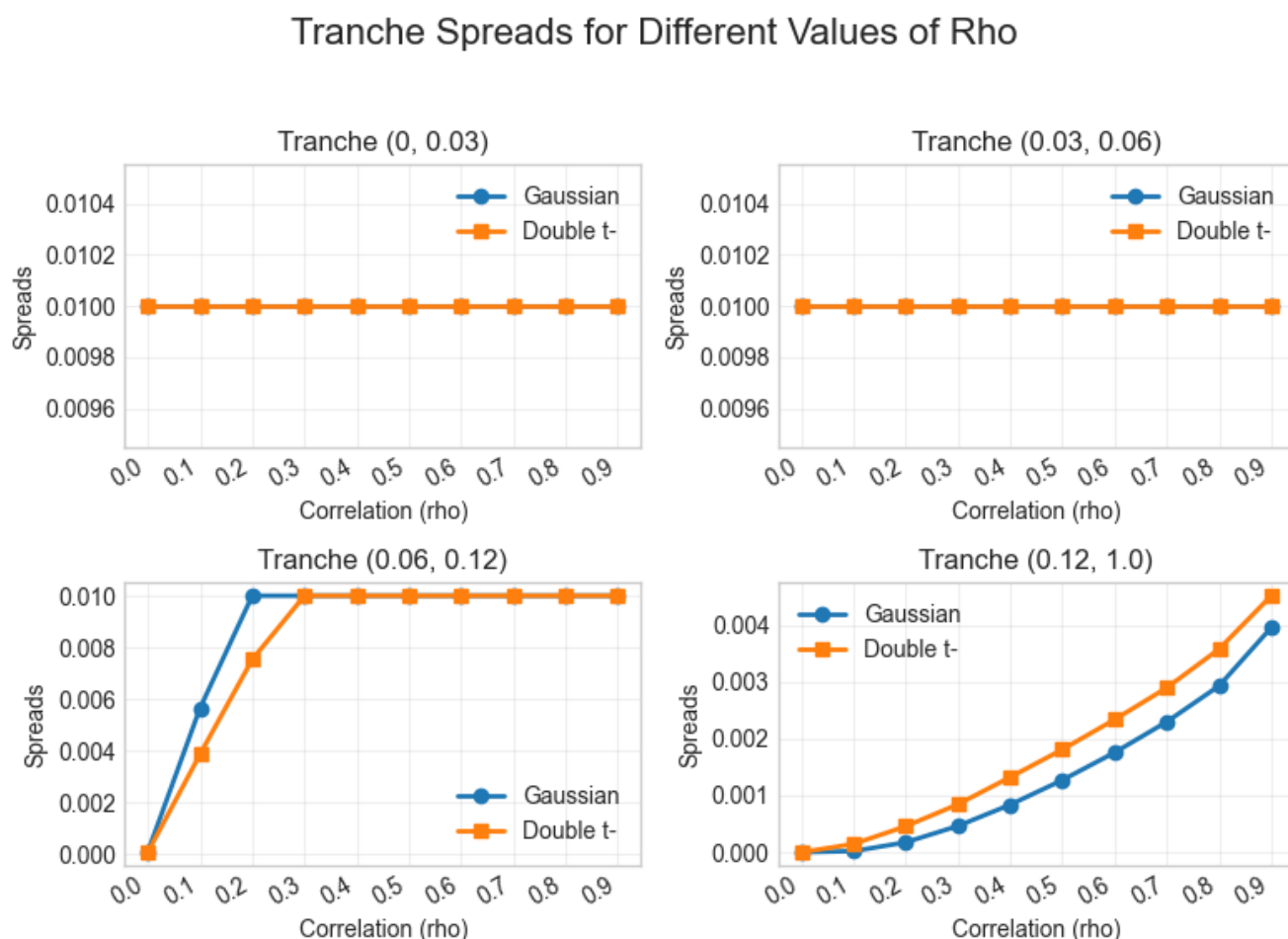


Figure 12: This plot shows the fair tranche spreads for different values of ρ for both the Gaussian copula model and the double t copula model.

In Figure 12, we see how the fair tranche spreads produced by both models change when the value of ρ is altered. However, because we cap the fair spreads at 100 bps, the spread sensitivity for the equity and mezzanine tranches is not very informative. Therefore, we also plot the fair upfronts of all tranches below.

Tranche Upfronts for Different Values of Rho

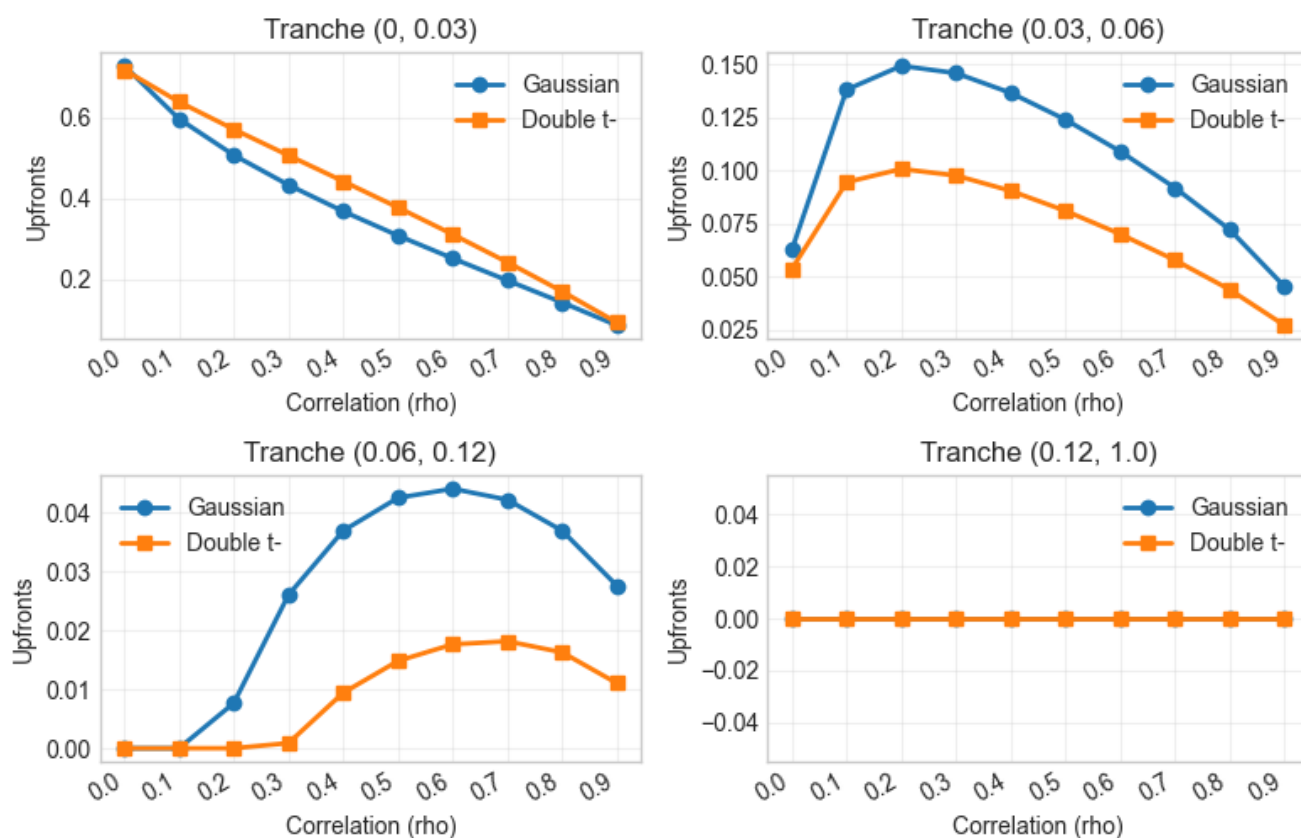


Figure 13: This plot shows the fair tranche upfronts for different values of ρ for both the Gaussian copula model and the double t copula model.

In Figure 13, we see how the fair tranche upfronts produced by both models change when the value of ρ is altered. We observe that for the equity tranche, higher correlations result in lower upfronts for both models, and for the senior tranche higher correlations result in higher upfronts. However, for the two mezzanine tranches, the upfronts are non-monotonic in ρ . This is because of

how ρ shifts the default probability distribution and how this affects the probability of the tranches incurring losses. As shown in Table 3, the probability of a complete wipeout of tranche principal for the (0.03, 0.06) tranche is non-monotonic in ρ for both models. This non-monotonicity of the default distribution translates into the risk of the tranche causing non-monotonicity of the fair tranche spread and upfront. For the (0.06, 0.12) tranche, we also observe non-monotonicity in ρ . However, the wipeout probabilities of this tranche shown in Table 3 are monotonic. Instead, the non-monotonicity in ρ is likely observed as a result of the non-monotonicity in wipeout probabilities of the preceding (0.03, 0.06) tranche, since this translates to when the (0.06, 0.12) tranche starts to incur losses.

In the next section, we discuss the choices of the values of the fixed parameters in both models and discuss the implementation of the double t copula.

6 Discussion

In defining the models, we have made some assumptions to simplify the math. We have assumed homogeneity of the portfolio and have also assumed a fixed recovery rate of $R = 0.4$. Lastly, we also fixed the degrees of freedom for both the systematic and idiosyncratic factors in the double t copula model to $\nu = 4$.

This section serves primarily as a sensitivity analysis of the fixed parameter values, but we will also examine how the models would act if we changed the portfolio assumption to be heterogeneous. To finish off, we discuss the possible downsides of using the double t copula instead of the Gaussian copula to model CDO tranche pricing.

In Section 6.1, we examine how the pricing of tranches could vary if we had instead used a heterogeneous portfolio.

6.1 Homogeneous Versus Heterogeneous Portfolio

This section serves as a purely qualitative analysis of how having a heterogeneous portfolio could change the fair spreads and upfronts of a CDO. Performing a quantitative study would require building a new model for the sole purpose of benchmarking our existing homogeneous model to it. This can of course be done, but it is not the scope of this paper. For a method of building a Gaussian copula model with a heterogeneous portfolio, we refer to Andersen, Sidenius, and Basu (2003) and Hull and White (2004). We view this qualitative study as instrumental in pointing out a key caveat in using the homogeneity assumption.

For the homogeneous portfolio, we get the following implied marginal cumulative default probabilities for different times, as pointed out in Section 4.

T	0.5Y	1Y	1.5Y	2Y	2.5Y	3Y	3.5Y	4Y	4.5Y	5Y
Index Spread	3.79	5.16	11.08	17.26	23.93	31.28	38.65	45.47	48.51	55.71
$P(\tau < T)$	0.032%	0.086%	0.28%	0.58%	1.00%	1.58%	2.27%	3.05%	3.65%	4.66%

Table 8: This table shows the implied cumulative default probabilities calculated for the observed index spreads with different times to maturity.

The index spread of a CDO is slightly lower than the average of the credit default swap spreads of the entities in the portfolio (Hull 2022). However, averages often do not provide a fair representation of reality. If we suppose that there are a handful of entities in the portfolio that have significantly higher credit default swap spreads because they have a perceived higher default risk, then their spreads could be offset by other less risky companies pulling the average in the other direction.

If we imagine the CDS spread curve of a reference entity to be roughly double that of the index spread curve, we get the following default probabilities:

T	0.5Y	1Y	1.5Y	2Y	2.5Y	3Y	3.5Y	4Y	4.5Y	5Y
Spread (bps)	7.6	10.3	22.2	34.5	47.9	62.6	77.3	90.9	97.0	111.4
$P(\tau < T)$	0.063%	0.17%	0.56%	1.15%	2.00%	3.14%	4.51%	6.04%	7.21%	9.16%

Table 9: Implied cumulative default probabilities calculated for the constructed CDS spreads with different times to maturity.

It is clear that the default probabilities in Table 9 are higher than those in Table 8 and the slight concavity mentioned in Section 4 is also shown. We see that doubling the spreads results in close to a doubling of the default probabilities, but as shown in Section 4, the relationship is concave, not linear. Higher default probabilities mean that these entities have a higher risk of default over the different maturities. This means that compared to the homogeneous portfolio, it is far more likely for the equity tranche for example to be wiped out earlier. The increased risk of the equity tranche leads to a higher fair upfront. Conversely, this should lead to a lower risk for the senior tranche since more reference entities have low CDS spreads, and therefore the senior tranche should have a lower fair spread. Since the index spread is roughly an average over all CDS spreads in the portfolio, it means that the model is underestimating the risk of the equity tranche while simultaneously overestimating the risk of the senior tranche. However, since this is a qualitative study, the conclusion is not necessarily definitive, and performing a quantitative study would allow for stronger conclusions to be drawn.

Next, we perform a sensitivity analysis of the recovery rate assumption.

6.2 Sensitivity Analysis of Recovery Rate

In the models, we have also made a recovery assumption. We have assumed that when a reference entity defaults, the holder of a bond suffers a loss of $1 - R$ per unit of notional, with $R = 0.4$. Reality, however, is not that simple. Not only do recovery rates vary across firms, but according to Andersen and Sidenius (2004), they are also hard to estimate prior to default.

We will now have a look at how the fair spreads and upfronts of the model vary when we increase

or decrease the value of the fixed recovery rate.

Tranche Spreads vs Recovery Rate ($\rho=0.5$)

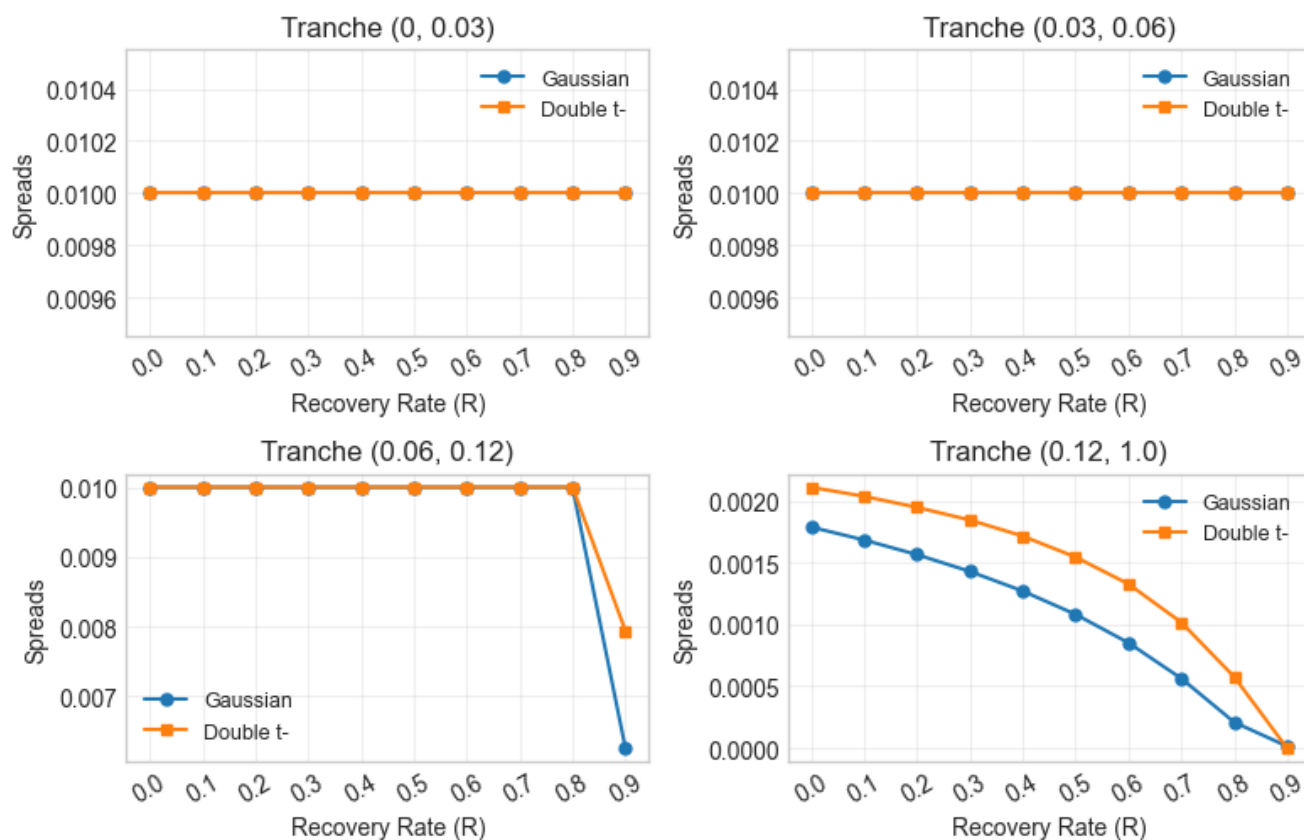


Figure 14: In this plot, we see how the fair tranche spreads change when the value of the fixed recovery rate is changed for both models.

In Figure 14, we see the fair tranche spreads of both models for different values of R . Once again, since the tranche spreads are capped at 100 bps, we also plot the fair upfronts below.

Tranche Upfronts vs Recovery Rate ($\rho=0.5$)

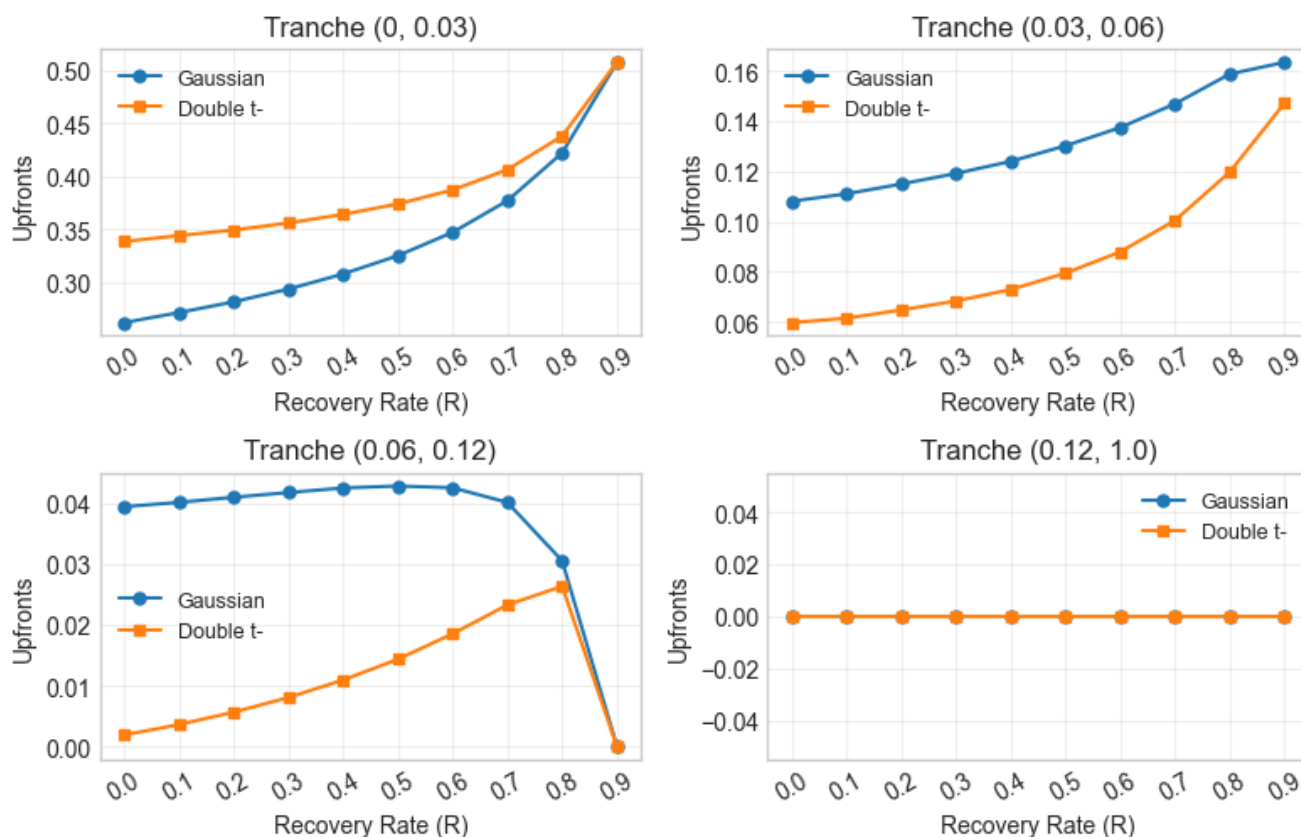


Figure 15: In this plot, we see how the fair tranche upfronts change when the value of the fixed recovery rate is changed for both models.

In Figure 15, we see the fair upfronts for both models for different values of R . When we increase the recovery rate, we expect the riskiness of all tranches to decrease. After all, a higher recovery rate means that more defaults need to occur before a tranche is wiped out. We would therefore expect to see lower tranche spreads and upfronts. But when we look at Figures 14 and 15 above, that is not what we observe. For the equity tranche and the (0.03, 0.06) mezzanine tranche, we actually see that the upfronts increase as we increase the recovery rate. This happens because recovery rates enter somewhere else in the model. Specifically, the CDS bootstrapping equation from Section 2.3.1 determines hazard rates by matching the protection leg — which depends on

$(1 - R)$ times the cumulative default probability — to the market spread. Increasing R reduces $(1 - R)$, so the hazard rates must increase to keep the protection leg at the market-quoted level. The result is higher implied default probabilities under a higher recovery rate. The decreased risk from needing a higher number of defaults before a tranche is wiped out is then offset by the increased risk of marginal defaults.

Once again, we see a non-monotonic pattern for the $(0.06, 0.12)$ mezzanine tranche. The fair upfronts increase for both models as we increase the recovery rate, until they drop sharply to 0 at $R = 0.9$. This is because the increased risk caused by the increased marginal default probabilities is offset by the increasing number of defaults that need to occur before the tranche incurs losses. At $R = 0.9$, $\frac{0.06 \cdot 125}{1 - 0.9} = 75$ defaults must occur before the principal of the $(0.06, 0.12)$ tranche gets affected, which is very unlikely in both models.

For the senior tranche, we observe the expected effect: fair spreads decrease as R increases. It seems that the increased marginal default risk is not enough to offset the decreased risk from increasing the recovery rate at the extreme tail. As argued for the $(0.06, 0.12)$ tranche, increasing the value of R increases the number of defaults that need to occur before the senior tranche starts to incur losses.

The choice of the fixed recovery rate value has a significant impact on the fair spreads and upfronts produced by both models. Fixing a recovery rate value is a calibration exercise and can be done if enough market spreads are available. Instead of fixing a value of the recovery rate, one proposed solution is to make the recovery rate stochastic (Andersen and Sidenius 2004). Empirically, there is a correlation between implied default probabilities and recovery rates (De Servigny and Renault 2002). If a firm becomes financially distressed, their CDS spreads increase leading to increases in their implied default probabilities, and if the firm defaults, the recovery rate will most likely be lower, due to the firm being financially distressed. This means that there is an inverse correlation between recovery rates and default probabilities. However, implementing a stochastic recovery rate is not within the scope of this paper.

Next, we examine the sensitivity to the fixed degrees of freedom parameter $\nu = 4$ in the double t

copula model.

6.3 Sensitivity Analysis of Degrees of Freedom

The choice of $\nu = 4$ in the double t copula model was suggested by Hull and White (2004) as they showed that it displayed a good fit to the market prices. With $\nu = 4$, the marginal distributions had fat enough tails to account for extreme events, and the resulting copula had sufficient tail dependence to capture default clustering, allowing the model to match observed tranche spreads.

Tranche Spreads of Double t- Model vs Degrees of Freedom

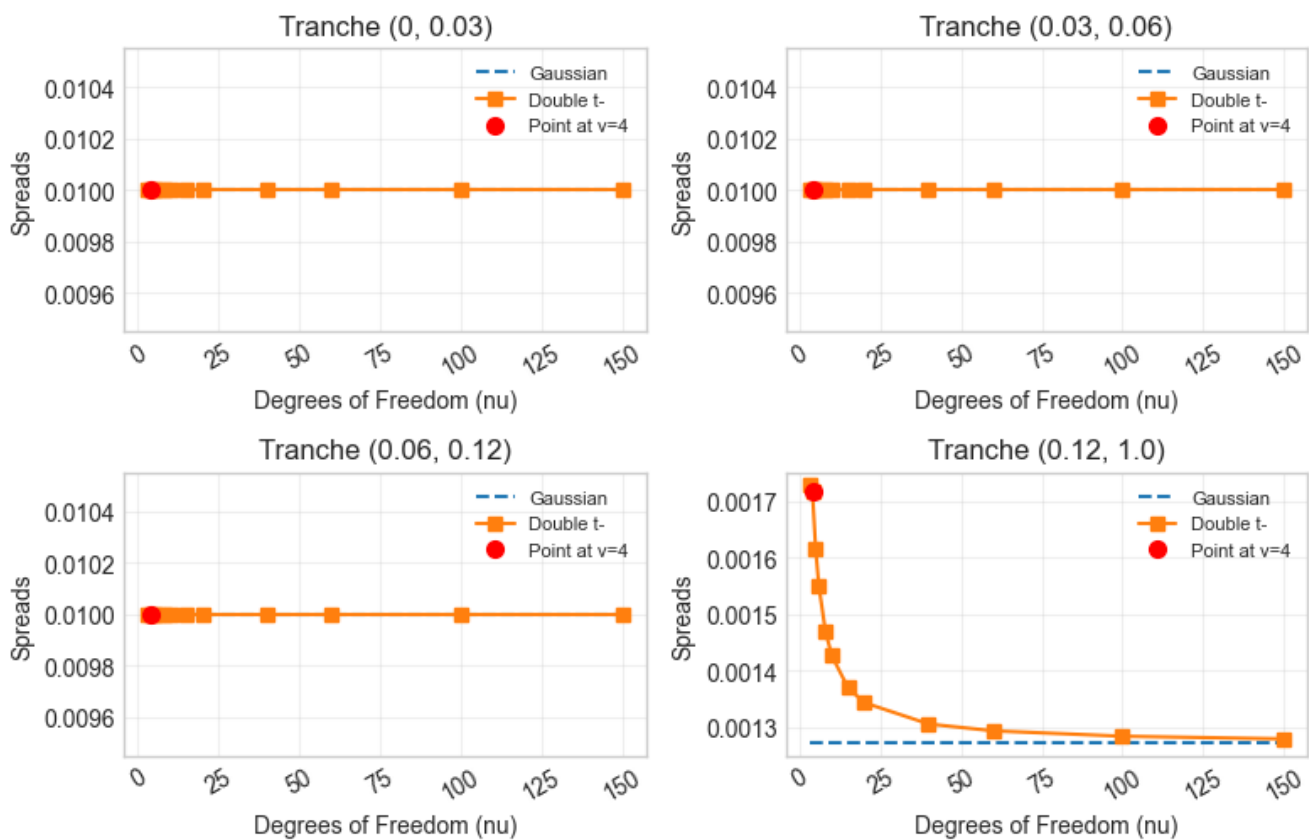


Figure 16: This plot shows how the fair tranche spreads change in the double t model as we change the degrees of freedom. In the senior tranche subplot, we clearly see the convergence towards the Gaussian model as the degrees of freedom increase. As $\nu \rightarrow \infty$ the scaled t distribution converges to the standard normal and the double t copula reduces to the Gaussian copula.

Since we have capped the fair tranche spread at 100 bps, the subplots for the equity tranche and two mezzanine tranches are not very instructive. But when we look at the fair senior tranche spread, we see that the fair tranche spread is a monotonically decreasing function of the degrees of freedom. This is because the degrees of freedom determine how likely extreme events are, for example, observing a high number of defaults. For low values of ν , extreme events become more probable, and therefore the risk of observing many defaults increases, leading to a higher fair spread for the senior tranche.

Tranche Upfronts of Double t- Model vs Degrees of Freedom

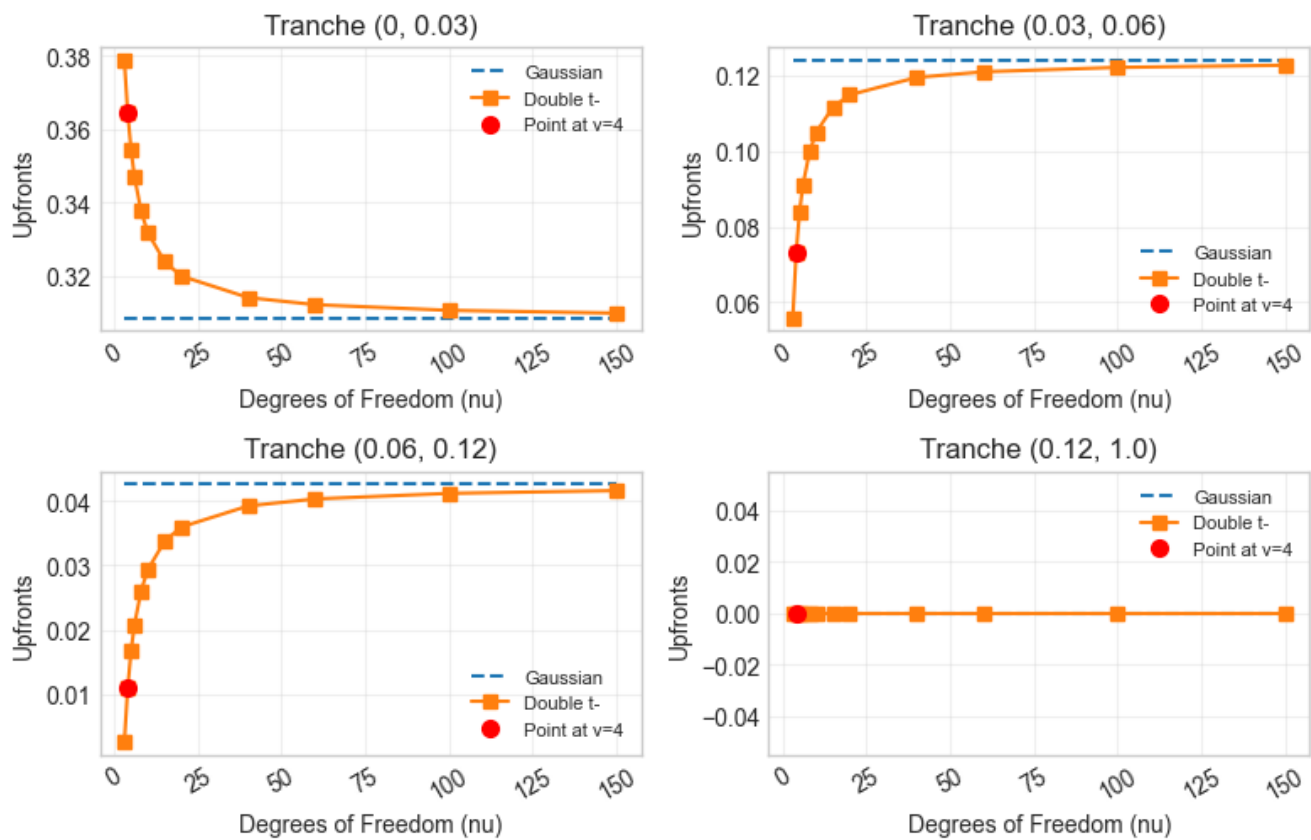


Figure 17: This plot shows how the fair tranche upfronts change in the double t model as we change the degrees of freedom.

Similarly to the senior tranche's fair spread, the equity tranche's fair upfront decreases as ν in-

creases, which we see in Figure 17. As $\nu \rightarrow \infty$, the double t copula converges to the Gaussian copula. In Section 5.5, we argued that the equity tranche upfront is larger for the double t copula because of a higher probability of fewer than 7 defaults. This logic also applies here - as ν increases the higher probability of a few defaults decreases and tends toward the probabilities in the Gaussian model leading to a lower upfront.

Once again, the two mezzanine tranches show the opposite behavior of the equity and senior tranches. Table 3 illustrates the wipeout probabilities for mezzanine tranches (0.03, 0.06) and (0.06, 0.12) at $\rho = 0.5$. The Gaussian copula model yields probabilities of 0.130 and 0.062, respectively, while the double t copula model produces lower estimates of 0.089 and 0.047. As the degrees of freedom increases, the wipeout probabilities of the double t copula model tend toward the wipeout probabilities of the Gaussian copula model and the fair upfront tends towards the higher Gaussian value.

It is also possible to fit a model where the degrees of freedom for the systematic and idiosyncratic factors are different. For example, one could let the degrees of freedom for the idiosyncratic factor tend towards infinity, thus making the idiosyncratic factor standard normal distributed as was experimented with in Hull and White (2004).

Next, we discuss the implementation of the double t copula model, focusing on its increased computational complexity.

6.4 Increased Complexity Introduced by the Double t Copula Model

No comparison of models is complete without having a discussion of the difficulties of applying each of them. The results suggest that the double t copula model addresses some of the shortcomings of the Gaussian copula model, such as the flatter correlation smile, through its improved treatment of extreme events. However, the double t copula model also increases the complexity of the model. It introduces the additional model parameter ν which means that the Gaussian copula model is the more parsimonious of the two. Having more parameters results in more work related to calibrating a model. In this paper, we have relied on the calibration work of our predecessors like the Hull

and White (2004) suggestion of $\nu = 4$. The intention of this paper was also not to calibrate and find the model parameters that fit the market spreads and upfronts the best. To do so, we would need many more observations to draw any kind of robust conclusions. If for example a credit trading desk at a financial institution wanted to implement a double t copula model to quote CDO tranches, they would need to perform this calibration exercise for ρ, ν and R , whereas in the Gaussian copula model, they would only need to calibrate the values of ρ and R . The degrees of freedom of the systematic and idiosyncratic factors ν_S and ν_Z in the double t copula model could also be calibrated to different values.

As pointed out previously, under the homogeneity assumption the Gaussian copula model can be built using standard Gauss-Hermite quadrature, while the double t copula model cannot. Instead, to build the double t copula, we need to use the generalized approach of adaptive Gauss-Hermite quadrature. This approach comes with additional computations, such as finding the mode of a curve, the curvature of that curve around the mode, and the cumulative distribution function of the default variable from characteristic functions. One also has to find appropriate values for n_{herm}, L and N through convergence checks. Having to do additional computations not only requires more computing power, but also adds to the risk of numerical instability that can significantly complicate the implementation of a model. For reference, using the author's computer and the parameter values listed in Section 5, calculating the three base correlations seen in Figure 11 took 182 seconds for the Gaussian model and 269 seconds for the double t model. The double t copula model thus takes 48% longer to produce the same measures. These figures reflect the implementation in this paper and are not optimized for production use; the relative penalty in a production implementation could be smaller. In practice, this can still be a significant constraint, as trading desks often require fast models that can quickly adjust to market movements. However, the increased computation time for the double t copula model can be reduced through tuning the numerical parameters n_{herm}, L and N to optimize numerical accuracy with respect to speed. Likewise, the Gaussian model can be optimized for speed through tuning of n_{herm} . Lastly, the Gaussian copula model has been used as the standard market model for over two decades. This means that the Gaussian copula model is likely familiar to everyone trading in CDOs. The notion

of base correlation is one of the primary measures of risk for CDOs, and since we have shown that base correlations differ between the Gaussian copula model and the double t copula model, base correlations are model-dependent. This complicates the usage of base correlation as a market communication tool. Therefore, it will be hard to completely shift from the Gaussian to the double t copula model. This could also explain why the Gaussian copula model remains the standard market model even though the consensus is that it has its limitations.

7 Conclusion

In this thesis, we investigated how the double t copula model performs CDO tranche pricing compared to the Gaussian copula model.

We found that the double t copula model has a flatter correlation smile with a range of 0.48 compared to 0.76 for the Gaussian copula model, a significant reduction within the possible interval of $[0, 1)$. The correlation smile is therefore reduced, but not eliminated.

We showed how the default distribution shifts in the double t copula model and how this results in increases in the expected present value of the protection legs of the equity and senior tranches of 15.9% and 36.7%, respectively. However, the two mezzanine tranches saw decreases in the protection leg values of 29.5% and 35.0%. These differences are caused by the shifting of the default distribution and the resulting changes in wipeout probabilities. The equity and senior tranches absorb the increased tail risk, while the expected loss of the mezzanine tranches decreases because the probability mass of the default distribution is shifted towards the tails.

We performed sensitivity analyses of the fixed model parameters ρ , R and ν concluding that the tranche spreads and upfronts are significantly affected by the choices of the fixed values. We note the monotonic relations of spreads and upfronts of the equity and senior tranches as functions of ρ , R and ν . Conversely, we note that the relations between the mezzanine tranche spreads and upfronts and the fixed parameter values are non-monotonic. The differences are caused by how the changing default distributions affect the tranches. We found a counterintuitive result that increasing the recovery rate R does not necessarily decrease tranche risk, since increasing the

recovery rate also changes the implied default probabilities bootstrapped from the index spreads.

Lastly, we discussed whether the additional complexity introduced by the double t copula model poses a significant challenge to its implementation. The slower calculations and the model-specific base correlations make an industry-wide methodology shift impractical, even though tuning the numerical parameters n_{herm} , L and N can narrow the runtime gap. We conclude that the double t copula model rests on a stronger theoretical foundation than the Gaussian copula model due to its treatment of tail risk, but that these practical barriers explain why an institutional shift away from the Gaussian standard remains unlikely.

From a methodological standpoint, we have provided a detailed implementation of the double t copula model using characteristic functions and adaptive Gauss-Hermite quadrature, mirroring the structure of the standard Gaussian implementation. To our knowledge, this implementation approach has not previously been documented in the literature.

However, our findings are based on a single day's tranche quotes (iTraxx Europe S42 5Y on June 20, 2025) and are therefore limited in their robustness. In further research, repeating the analysis over a longer time series of several months or even years while also performing a calibration of the model parameters would produce more robust models. This calibration could show if the suggestion from Hull and White (2004) of $\nu = 4$ still fits market prices reasonably well or if this has changed. Moreover, a thorough quantitative comparison of homogeneous and heterogeneous models using the methods from Andersen, Sidenius, and Basu (2003) and Hull and White (2004) could provide additional insights into the impact of the homogeneity assumption.

References

- Andersen, Leif and Jakob Sidenius (Winter 2004). “Extensions to the Gaussian Copula: Random Recovery and Random Factor Loadings”. In: *The Journal of Credit Risk* 1.1, pp. 29–70.
- Andersen, Leif, Jakob Sidenius, and S. Basu (Jan. 2003). “All Your Hedges in One Basket”. In: *Risk* November.
- Brigo, Damiano, Andrea Pallavicini, and Roberto Torresetti (2010). “Gaussian Copula Model and Implied Correlation”. In: *Credit models and the crisis: A Journey into CDOs, Copulas, correlations and dynamic models*. John Wiley & Sons. Chap. 3, pp. 38–49.
- Castellacci, Giuseppe (Apr. 2012). “On Bootstrapping Hazard Rates From CDS Spreads”. In: *SSRN Electronic Journal*.
- De Servigny, Arnaud and Olivier Renault (2002). “Default correlation: empirical evidence”. In: *Standard and Poors*.
- Feller, William (1971). “Characteristic Functions”. In: *An introduction to probability theory and its applications. Vol. II*. Second edition. John Wiley & Sons. Chap. 15, pp. 498–530.
- Hull, John C. (2022). “Credit Derivatives”. In: *Options, Futures, and Other Derivatives*. Eleventh. Pearson. Chap. 25, pp. 587–613.
- Hull, John C. and Alan White (2004). “Valuation of a CDO and an nth to Default CDS Without Monte Carlo Simulation”. In: *Journal of Derivatives* 12.2, pp. 8–23.
- International Swaps and Derivatives Association (2014). *2014 ISDA Credit Derivatives Definitions*. Accessed: 11-05-2026. International Swaps and Derivatives Association, Inc. (ISDA). New York, NY. URL: <https://www.isda.org/en/uploads/sites/66/content/docs/2014-Credit-Derivatives-Definitions.pdf>.
- Li, David X. (Mar. 2000). “On Default Correlation: A Copula Approach”. In: *Journal of Fixed Income*, pp. 43–54.
- Liu, Qing and Donald A. Pierce (Aug. 1994). “A note on Gauss-Hermite Quadrature”. In: *Biometrika* 81.3, pp. 624–629.
- Press, William H. et al. (2007). “Numerical Derivatives”. In: *Numerical Recipes 3rd Edition: The Art of Scientific Computing*. 3rd ed. Cambridge University Press. Chap. 5, pp. 229–232.

- Ruppert, David and David S. Matteson (2016). “Copulas”. In: *Statistics and Data Analysis for Financial Engineering: With R examples*. Second. Springer. Chap. 8, pp. 196–198.
- Salmon, Felix (Feb. 2009). “The Formula That Killed Wall Street”. In: *Wired*.
- Sklar, Abe (1959). “Fonctions de répartition à n dimensions et leurs marges”. In: *Publications de l’Institut Statistique de l’Université de Paris*, 8, pp. 229–231.
- Torresetti, Roberto, Damiano Brigo, and Andrea Pallavicini (Nov. 2006). “Implied correlation in CDO tranches: A paradigm to be handled with care”. In: *SSRN Electronic Journal*.
- Vasicek, Oldrich (1987). “Probability of Loss on a Loan Portfolio”. In: *KMV*.
- Witkovský, Viktor (Aug. 2015). “Numerical Inversion of a Characteristic Function: An Alternative Tool to Form the Probability Distribution of Output Quantity in Linear Measurement Models”. In: *XXI IMEKO World Congress “Measurement in Research and Industry”*. IMEKO. Prague, Czech Republic. URL: <https://www.imeko.org/publications/wc-2015/IMEKO-WC-2015-TC21-424.pdf>.

A Appendix

A.1 Brent's Method

In cases where one needs to find a one-dimensional root where one only has the function values, Press et al. (2007) recommends using Brent's Method. Brent's method combines the other root finding methods of *bracketing*, *bisection*, and *inverse quadratic interpolation* to find the root of a function. For three point pairs $[a, f(a)]$, $[b, f(b)]$ and $[c, f(c)]$, the interpolation formula is

$$x = \frac{[y - f(a)][y - f(b)]c}{[f(c) - f(a)][f(c) - f(b)]} + \frac{[y - f(b)][y - f(c)]a}{[f(a) - f(b)][f(a) - f(c)]} + \frac{[y - f(c)][y - f(a)]b}{[f(b) - f(c)][f(b) - f(a)]}$$

Setting $y = 0$ we get

$$x = \frac{f(a)f(b)c}{[f(c) - f(a)][f(c) - f(b)]} + \frac{f(b)f(c)a}{[f(a) - f(b)][f(a) - f(c)]} + \frac{f(c)f(a)b}{[f(b) - f(c)][f(b) - f(a)]}$$

This can be rewritten as

$$x = b + P/Q$$

with

$$P = S[T(R - T)(c - b) - (1 - R)(b - a)]$$

$$Q = (T - 1)(R - 1)(S - 1)$$

where $R = f(b)/f(c)$, $S = f(b)/f(a)$ and $T = f(a)/f(c)$. From this form, b is the best estimate of the root and the fraction P/Q is a correction that should be small around the root. If P/Q is out of bounds or when the bounds are not collapsing fast enough, Brent's method takes a bisection step.

A.2 Inverse Fast Fourier Transform of Characteristic Functions

For a characteristic function $\varphi(x)$, the density function can be recovered using Fourier inversion

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} e^{-i\xi x} \varphi(\xi) d\xi \quad (9)$$

as by Feller (1971). Using the approximation $\int_a^b f(x) dx \approx f(a)(b-a)$, we can approximate this interval by choosing a sufficiently large interval (a, b) . Since we are dealing with characteristic functions, we want to sample in the region that captures most of the probability mass (Witkovský 2015).

The following derivation is from Witkovský (2015).

Let $j, k = 0, \dots, N-1$, $\delta_y = (b-a)/N$, and $y_k = a + k\delta_y$. For large N and $T = \pi/\delta_y$ we get from equation 9 using $\xi = 2\pi u$, $d\xi = 2\pi du$, and $du = \frac{1}{b-a}$, we get

$$f(y_k) \approx \int_{-\frac{1}{2\delta_y}}^{\frac{1}{2\delta_y}} e^{-i2\pi u y_k} \varphi(2\pi u) du.$$

By repeatedly using the approximation $\int_a^b f(x) dx \approx f(a)(b-a)$ for each of the N intervals, we get

$$f(y_k) \approx \frac{1}{b-a} \sum_{j=0}^{N-1} e^{-i2\pi u_j y_k} \varphi(2\pi u_j),$$

where $u_j = \frac{j - \frac{N}{2}}{b-a}$, $j = 0, \dots, N-1$.

A.3 Convergence Check of n_{herm} , L , and N

n_{herm}	Fair Tranche Spread/Upfront				Runtime (s)			
	0–0.03	0.03–0.06	0.06–0.12	0.12–1.0	0–0.03	0.03–0.06	0.06–0.12	0.12–1.0
10	0.3054	0.1070	0.0666	10.703	0.117	0.117	0.119	0.149
20	0.3051	0.1344	0.0360	13.145	0.235	0.238	0.237	0.303
40	0.3089	0.1230	0.0416	12.876	0.480	0.476	0.485	0.608
60	0.3083	0.1241	0.0425	12.710	0.708	0.708	0.719	0.903
80	0.3084	0.1239	0.0426	12.706	1.005	0.937	0.955	1.223
100	0.3084	0.1240	0.0425	12.721	1.316	1.174	1.218	1.623
140	0.3084	0.1239	0.0425	12.716	1.796	1.697	1.827	2.181
180	0.3084	0.1239	0.0425	12.715	2.256	2.123	2.340	2.777
240	0.3084	0.1239	0.0425	12.716	2.831	2.838	2.954	3.779
300	0.3084	0.1239	0.0425	12.716	3.541	3.515	3.601	4.555

Table 10: Convergence check of n_{herm} for fair spreads and upfronts for the Gaussian copula model. We also show the runtimes. We see convergence across tranches around 140 nodes.

n_{herm}	Fair Tranche Spread/Upfront				Runtime (s)			
	0–0.03	0.03–0.06	0.06–0.12	0.12–1.0	0–0.03	0.03–0.06	0.06–0.12	0.12–1.0
10	0.3626	0.0669	0.0057	16.293	0.895	0.791	0.473	0.692
20	0.3639	0.0704	0.0084	16.714	1.015	0.925	0.609	0.915
40	0.3643	0.0721	0.0099	16.960	1.308	1.187	0.864	1.298
60	0.3644	0.0725	0.0104	17.110	1.569	1.470	1.117	1.560
80	0.3645	0.0727	0.0106	17.086	1.801	1.705	1.392	1.851
100	0.3645	0.0728	0.0107	17.111	2.099	1.944	1.661	2.163
140	0.3645	0.0729	0.0108	17.146	2.669	2.480	2.164	2.894
180	0.3645	0.0730	0.0109	17.152	3.098	3.067	2.693	3.467
240	0.3645	0.0730	0.0110	17.153	3.869	3.732	3.453	4.465
300	0.3645	0.0730	0.0110	17.170	4.834	4.523	4.216	5.411

Table 11: Convergence check of n_{herm} for fair spreads and upfronts for the double t copula model. We also show the runtimes. We see convergence across tranches around 200 nodes.

N	Fair Tranche Spread/Upfront				Runtime (s)			
	0–0.03	0.03–0.06	0.06–0.12	0.12–1.0	0–0.03	0.03–0.06	0.06–0.12	0.12–1.0
256	0.3071	0.0432	93.0662	12.759	3.337	3.227	3.018	3.539
512	0.3366	0.0578	0.0037	15.349	3.364	3.336	3.121	3.809
1024	0.3504	0.0651	0.0072	16.227	3.465	3.418	3.035	3.782
2048	0.3576	0.0691	0.0091	16.705	3.402	3.153	2.912	3.752
4096	0.3611	0.0711	0.0100	16.941	3.362	3.207	2.951	3.794
8192	0.3629	0.0721	0.0105	17.063	3.322	3.202	2.940	3.816
16384	0.3638	0.0726	0.0107	17.123	3.315	3.194	2.956	3.878
32768	0.3643	0.0728	0.0109	17.153	3.305	3.192	2.952	3.847
65536	0.3645	0.0730	0.0109	17.168	3.301	3.198	2.934	3.815
131072	0.3646	0.0730	0.0110	17.176	3.310	3.211	2.943	3.817

Table 12: Convergence check of N for fair spreads and upfronts for the double t copula model. We also show the runtimes. We see convergence across tranches around $N = 2^{16} = 65536$ grid points.

L	Fair Tranche Spread/Upfront				Runtime (s)			
	0–0.03	0.03–0.06	0.06–0.12	0.12–1.0	0–0.03	0.03–0.06	0.06–0.12	0.12–1.0
5	0.3574	0.0728	0.0107	16.842	3.336	3.981	3.111	3.824
10	0.3643	0.0731	0.0110	17.162	3.389	3.590	3.105	3.792
15	0.3646	0.0730	0.0110	17.176	3.313	3.306	3.148	3.821
20	0.3646	0.0730	0.0109	17.173	3.296	3.367	2.972	3.825
25	0.3645	0.0730	0.0109	17.171	3.347	3.528	2.976	3.787
30	0.3645	0.0730	0.0109	17.168	3.487	3.344	2.940	3.791
40	0.3644	0.0729	0.0109	17.163	3.394	3.309	2.955	4.441
50	0.3644	0.0729	0.0109	17.158	3.462	3.330	3.131	4.210
75	0.3642	0.0728	0.0108	17.146	3.331	3.397	2.991	3.985
100	0.3640	0.0727	0.0108	17.133	3.870	3.277	2.954	4.008

Table 13: Convergence check of L for fair spreads and upfronts for the double t copula model. We also show the runtimes. We see convergence across tranches around $L = 15$ grid range.

A.4 Markit iTraxx Europe Index Bloomberg Description

Index Information			
The Markit iTraxx Europe index comprises 125 equally weighted credit default swaps on investment grade European corporate entities, distributed among 4 sub-indices: Financials (Senior & Subordinated), Non-Financials and HiVol. The composition of each Markit iTraxx index is determined by the Index Rules. Markit iTraxx indices roll every 6 months in March & September.			
Contract Information		Identifiers	
Currency	EUR	Day Cnt	ACT/360
Tenor	5Y	Cpn Fr...	Q
Dsc Curv	EU Custom	Swap Curve	
Region	N.A.		
ISDA Definitions	Year	2014	
Cpn (bps)	100	Factor	1.0
Recovery	0.40	Version	1
Quote Ty...	Spread	Series	44
Start Date	09/22...	Start Cnst	125
Mty Date	12/20...		
Restruct	Modified-Modified	Res...	
Index	MARKIT ITRX EUROPE 12/30		
Short Name	EUR/ Corp		
Full Name	ITRX EUR CDSI S44 5Y Corp		
BBID	ITXEB544		
RED Code	2I666VDM4		

Figure 18: Screenshot taken of the description function Markit iTraxx Europe Index S42 5Y in the Bloomberg Terminal.

A.5 Gaussian Copula Model Python Code

```

1 import numpy as np
2 from scipy.stats import norm
3 import math
4 import matplotlib.pyplot as plt
5 import pandas as pd
6 from scipy.optimize import brentq, minimize_scalar
7
8 class GaussianCopula:
9     def __init__(self, M, R, rho, T, delta, n_herm):
10         self.M = M
11         self.R = R
12         self.rho = rho
13         self.T = T
14         self.delta = delta
15         self.n_herm = n_herm
16
17         self.nodes, self.weights = np.polynomial.hermite.hermgauss(
18             n_herm)
19
20         self.N = int(self.T / self.delta)

```

```

21     self.times = np.arange(0, self.N + 1) * self.delta
22
23     #Load rates
24     df = pd.read_excel(r'C:\Speciale\rates_20-06-2025.xlsx')
25     rates = []
26     for i in range(1, len(df.columns)):
27         rates.append(df.iloc[0,i])
28
29     #Load index spreads
30     df = pd.read_excel(r'C:\Speciale\index_spread_20-06-2025.xlsx')
31     index_spread = (df.iloc[0, 1:])/10000
32     index_spread = index_spread.tolist()
33     self.index_spread = index_spread
34
35     #Calculate default probabilities
36     rate_times = np.arange(0, len(rates)) * 0.125
37     self.zcb = np.exp(-np.array(rates)/100 * rate_times)
38
39     default_probs = np.zeros(self.N+1)
40
41     default_probs = np.zeros(self.N+1)
42     maturities = [0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5]
43     hazard_rates = self.bootstrap_hazard_rates(index_spread,
44         maturities)
45     h1 = hazard_rates[0]
46     h2 = hazard_rates[1]
47     h3 = hazard_rates[2]
48     h4 = hazard_rates[3]
49     h5 = hazard_rates[4]
50     h6 = hazard_rates[5]
51     h7 = hazard_rates[6]
52     h8 = hazard_rates[7]
53     h9 = hazard_rates[8]
54     h10 = hazard_rates[9]
55
56     for i in range(3):
57         default_probs[i] = 1-np.exp(-h1*i*self.delta)
58     for i in range(3,5):
59         default_probs[i] = 1-(1-default_probs[2])*np.exp(-h2*(i-2)*
60             self.delta)
61     for i in range(5,7):
62         default_probs[i] = 1-(1-default_probs[4])*np.exp(-h3*(i-4)*
63             self.delta)
64     for i in range(7,9):
65         default_probs[i] = 1-(1-default_probs[6])*np.exp(-h4*(i-6)*
66             self.delta)
67     for i in range(9,11):
68         default_probs[i] = 1-(1-default_probs[8])*np.exp(-h5*(i-8)*
69             self.delta)
70     for i in range(11,13):
71         default_probs[i] = 1-(1-default_probs[10])*np.exp(-h6*(i-10)*
72             self.delta)
73     for i in range(13,15):
74         default_probs[i] = 1-(1-default_probs[12])*np.exp(-h7*(i-12)*
75             self.delta)
76     for i in range(15,17):
77         default_probs[i] = 1-(1-default_probs[14])*np.exp(-h8*(i-14)*
78             self.delta)
79     for i in range(17,19):
80         default_probs[i] = 1-(1-default_probs[16])*np.exp(-h9*(i-16)*
81             self.delta)
82     for i in range(19,21):

```

```

74         default_probs[i] = 1-(1-default_probs[18])*np.exp(-h10*(i
75             -18)*self.delta)
76     self.default_probs = default_probs
77     print(default_probs)
78
79     self.default_barriers = norm.ppf(np.clip(default_probs, 1e-12,1
80         - 1e-12))
81
82     def bootstrap_hazard_rates(self, spreads, maturities):
83         hazard_rates = []
84
85         def survival(t, hazard_rates, maturities):
86             total = 0.0
87             prev_T = 0.0
88
89             for h, T in zip(hazard_rates, maturities):
90                 if t <= T:
91                     total += h * (t - prev_T)
92                     return np.exp(-total)
93                 else:
94                     total += h * (T - prev_T)
95                     prev_T = T
96
97             return np.exp(-total)
98
99         for k in range(len(spreads)):
100
101             s_k = spreads[k]
102             T_k = maturities[k]
103
104             def cds_equation(hk):
105
106                 temp_hazards = hazard_rates + [hk]
107                 temp_maturities = maturities[:k+1]
108
109                 total = 0.0
110
111                 N = int(T_k / self.delta)
112
113                 for i in range(1, N + 1):
114                     Ti = i * self.delta
115                     Ti_m1 = (i - 1) * self.delta
116
117                     # discount factor
118                     P = self.zcb[2*i]
119
120                     # survival probabilities
121                     S_i = survival(Ti, temp_hazards, temp_maturities)
122                     S_im1 = survival(Ti_m1, temp_hazards,
123                         temp_maturities)
124
125                     # CDS equation
126                     total += P * (
127                         (1 - self.R) * (S_im1 - S_i)
128                         - s_k * self.delta * (S_im1 + S_i) / 2
129                     )
130
131                     return total
132
133             # solve for hk
134             hk = brentq(cds_equation, 1e-6, 5.0)

```

```

134         hazard_rates.append(hk)
135
136     return hazard_rates
137
138     def calc_default_barrier(self,t):
139         default_barrier = norm.ppf(self.calc_default_probs(t))
140         return default_barrier
141
142     def integrate_gauss_hermite(self, func):
143         nodes = self.nodes
144         weights = self.weights
145
146         total = 0.0
147         for i in range(self.n_herm):
148             S = np.sqrt(2) * nodes[i]
149             total += weights[i] * func(S)
150
151         return total / np.sqrt(np.pi)
152
153     def calc_expected_tranche_principal(self, A, B, S, k):
154         S = np.asarray(S).item()
155         S = float(S)
156         # conditional default probability
157         barrier = self.default_barriers[k]
158
159         sqrt_rho = np.sqrt(self.rho)
160         sqrt_1mr = np.sqrt(1-self.rho)
161
162         p = norm.cdf((barrier - sqrt_rho*S) / sqrt_1mr)
163         p = min(max(p, 1e-16), 1-1e-16)
164
165         M = self.M
166         R = self.R
167
168         nl = A * M / (1 - R)
169         nh = B * M / (1 - R)
170
171         il = min(math.ceil(nl), M+1)
172         ih = min(math.ceil(nh), M+1)
173
174         # compute binomial probabilities recursively
175         probs = np.zeros(M+1)
176         probs[0] = (1 - p)**M
177
178         ratio = p/(1-p)
179
180         for n in range(M):
181             probs[n+1] = probs[n] * (M-n)/(n+1) * ratio
182
183         sum1 = probs[:il].sum()
184
185         sum2 = 0.0
186         for j in range(il, ih):
187             loss_fraction = (B - j*(1-R)/M)/(B-A)
188             sum2 += probs[j] * loss_fraction
189
190         return sum1 + sum2
191
192     def calc_PV_expected_regular_spread_payments(self,A,B): #A in Hull
193         delta = self.delta # assume equidistant payments
194         def integrand(S): #conditional expected payments * density
195             N = self.N
196             sum = 0

```

```

197         for k in range(1, N+1):
198             sum += delta * self.calc_expected_tranche_principal(A,B,
199                 S,k) * self.zcb[2*k]
200         return sum
201     return self.integrate_gauss_hermite(integrand)
202
203
204     def calc_accrual_payment(self,A,B): #B in Hull
205         delta = self.delta # assume equidistant payments
206
207         def integrand(S): #conditional accrual payments * density
208             N = self.N
209             sum = 0
210             for k in range(1, N+1):
211                 sum += 0.5*delta*(self.calc_expected_tranche_principal(A
212                     ,B,S,(k-1))-self.calc_expected_tranche_principal(A,B,
213                         S,k))*self.zcb[2*k-1]
214             return sum
215         return self.integrate_gauss_hermite(integrand)
216
217     def calc_PV_expected_payoffs(self,A,B): #C in Hull
218         delta = self.delta # assume equidistant payments
219         def integrand(S): #conditional PV_expected_payoffs * density
220             N = self.N
221             sum = 0
222             for k in range(1, N+1):
223                 sum += (self.calc_expected_tranche_principal(A,B,S,k-1)-
224                     self.calc_expected_tranche_principal(A,B,S,k))*self.
225                     zcb[2*k-1]
226             return sum
227         return self.integrate_gauss_hermite(integrand)
228
229     def calc_fair_spread_and_upfront(self,A,B):
230         fair_spread = float((self.calc_PV_expected_payoffs(A,B))/(self.
231             calc_PV_expected_regular_spread_payments(A,B)+self.
232             calc_accrual_payment(A,B)))
233         if fair_spread > 0.01:
234             spread = 0.01
235             upfront = float(self.calc_PV_expected_payoffs(A,B)-spread*(
236                 self.calc_PV_expected_regular_spread_payments(A,B) + self
237                 .calc_accrual_payment(A,B)))
238             return spread,upfront
239         else:
240             return fair_spread,0
241
242     def tranche_spread_for_rho(self, A, B, rho):
243         old_rho = self.rho
244         self.rho = rho
245
246         C = self.calc_PV_expected_payoffs(A, B)
247         P = (
248             self.calc_PV_expected_regular_spread_payments(A, B)
249             + self.calc_accrual_payment(A, B)
250         )
251         self.rho = old_rho
252         return C / P

```

```

251
252     def calc_compound_correlation(self, A, B, s, U):
253         def objective(rho):
254             def f(rho):
255                 old_rho = self.rho
256                 self.rho = rho
257
258                 C = self.calc_PV_expected_payoffs(A, B)
259                 P = (
260                     self.calc_PV_expected_regular_spread_payments(A, B)
261                     + self.calc_accrual_payment(A, B)
262                 )
263
264                 self.rho = old_rho
265
266                 return U + s * P - C
267             return (f(rho))**2
268
269         res = minimize_scalar(objective, bounds=(0.0001,0.9999), method=
270             'bounded')
271         return res.x
272
273     def calc_base_correlation(self, target_aq, market_quotes):
274         total_loss_0_aq = 0.0
275         for (A, B, s, U) in market_quotes:
276             if B > target_aq:
277                 break
278
279             comp_rho = self.calc_compound_correlation(A, B, s, U)
280
281             old_rho = self.rho
282             self.rho = comp_rho
283
284             C_q_absolute = self.calc_PV_expected_payoffs(A, B) * (B - A)
285             total_loss_0_aq += C_q_absolute
286
287             self.rho = old_rho
288
289         def objective(rho):
290             old_rho = self.rho
291             self.rho = rho
292             model_loss = self.calc_PV_expected_payoffs(0, target_aq) *
293                 target_aq
294             self.rho = old_rho
295             return model_loss - total_loss_0_aq
296         return brentq(objective, 0.0001, 0.9999)

```

A.6 Double t Copula Model Python Code

```

1  import numpy as np
2  import math
3  import matplotlib.pyplot as plt
4  import pandas as pd
5  from scipy.stats import t
6  from scipy import special
7  from scipy.interpolate import interp1d
8  from scipy.optimize import brentq, minimize, minimize_scalar
9
10 class t_Copula:
11     def __init__(self, M, R, rho, T, delta, n_herm, v1, v2):

```

```

12     self.M = M
13     self.R = R
14     self.rho = rho
15     self.T = T
16     self.delta = delta
17     self.v1 = v1
18     self.v2 = v2
19     self.n_herm = n_herm
20
21     self.nodes, self.weights = np.polynomial.hermite.hermgauss(
22         n_herm)
23
24     self.N = int(self.T / self.delta)
25
26     self.times = np.arange(0, self.N + 1) * self.delta
27
28     #Load rates
29     df = pd.read_excel(r'C:\Speciale\rates_20-06-2025.xlsx')
30     rates = []
31     for i in range(1, len(df.columns)):
32         rates.append(df.iloc[0,i])
33
34     #Load index spreads
35     df = pd.read_excel(r'C:\Speciale\index_spread_20-06-2025.xlsx')
36     index_spread = (df.iloc[0, 1:])/10000
37     index_spread = index_spread.tolist()
38     self.index_spread = index_spread
39
40     #Calculate default probabilities
41     rate_times = np.arange(0, len(rates)) * 0.125
42     self.zcb = np.exp(-np.array(rates)/100 * rate_times)
43
44     default_probs = np.zeros(self.N+1)
45
46     default_probs = np.zeros(self.N+1)
47     maturities = [0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5]
48     hazard_rates = self.bootstrap_hazard_rates(index_spread,
49         maturities)
50     h1 = hazard_rates[0]
51     h2 = hazard_rates[1]
52     h3 = hazard_rates[2]
53     h4 = hazard_rates[3]
54     h5 = hazard_rates[4]
55     h6 = hazard_rates[5]
56     h7 = hazard_rates[6]
57     h8 = hazard_rates[7]
58     h9 = hazard_rates[8]
59     h10 = hazard_rates[9]
60
61     for i in range(3):
62         default_probs[i] = 1-np.exp(-h1*i*self.delta)
63     for i in range(3,5):
64         default_probs[i] = 1-(1-default_probs[2])*np.exp(-h2*(i-2)*
65             self.delta)
66     for i in range(5,7):
67         default_probs[i] = 1-(1-default_probs[4])*np.exp(-h3*(i-4)*
68             self.delta)
69     for i in range(7,9):
70         default_probs[i] = 1-(1-default_probs[6])*np.exp(-h4*(i-6)*
71             self.delta)
72     for i in range(9,11):

```

```

68         default_probs[i] = 1-(1-default_probs[8])*np.exp(-h5*(i-8)*
        self.delta)
69     for i in range(11,13):
70         default_probs[i] = 1-(1-default_probs[10])*np.exp(-h6*(i-10)
        *self.delta)
71     for i in range(13,15):
72         default_probs[i] = 1-(1-default_probs[12])*np.exp(-h7*(i-12)
        *self.delta)
73     for i in range(15,17):
74         default_probs[i] = 1-(1-default_probs[14])*np.exp(-h8*(i-14)
        *self.delta)
75     for i in range(17,19):
76         default_probs[i] = 1-(1-default_probs[16])*np.exp(-h9*(i-16)
        *self.delta)
77     for i in range(19,21):
78         default_probs[i] = 1-(1-default_probs[18])*np.exp(-h10*(i
        -18)*self.delta)
79
80     self.default_probs = default_probs
81     print(default_probs)
82
83     self.default_barriers = self.quantile_func(np.clip(default_probs
        , 1e-12,1 - 1e-12))
84
85     def bootstrap_hazard_rates(self, spreads, maturities):
86         hazard_rates = []
87
88         def survival(t, hazard_rates, maturities):
89             total = 0.0
90             prev_T = 0.0
91
92             for h, T in zip(hazard_rates, maturities):
93                 if t <= T:
94                     total += h * (t - prev_T)
95                     return np.exp(-total)
96                 else:
97                     total += h * (T - prev_T)
98                     prev_T = T
99
100             return np.exp(-total)
101
102     for k in range(len(spreads)):
103
104         s_k = spreads[k]
105         T_k = maturities[k]
106
107         def cds_equation(hk):
108
109             temp_hazards = hazard_rates + [hk]
110             temp_maturities = maturities[:k+1]
111
112             total = 0.0
113
114             N = int(T_k / self.delta)
115
116             for i in range(1, N + 1):
117                 Ti = i * self.delta
118                 Ti_m1 = (i - 1) * self.delta
119
120                 # discount factor
121                 P = self.zcb[2*i]
122
123                 # survival probabilities

```

```

124         S_i = survival(Ti, temp_hazards, temp_maturities)
125         S_im1 = survival(Ti_m1, temp_hazards,
126                           temp_maturities)
127
128         # CDS equation
129         total += P * (
130             (1 - self.R) * (S_im1 - S_i)
131             - s_k * self.delta * (S_im1 + S_i) / 2
132         )
133
134         return total
135
136         # solve for hk
137         hk = brentq(cds_equation, 1e-6, 5.0)
138         hazard_rates.append(hk)
139
140     return hazard_rates
141
142 def t_characteristic_function(self, v, t):
143     t = np.atleast_1d(t)
144     t_abs = np.abs(t)
145     res = np.ones_like(t_abs)
146     mask = t_abs > 1e-12
147     arg = np.sqrt(v) * t_abs[mask]
148
149     # The t-distribution CF formula
150     log_bessel = np.log(special.kve(v/2, arg)) - arg
151
152     log_numerator = (v/2) * np.log(arg) + log_bessel
153     log_denominator = special.gammln(v/2) + (v/2 - 1) * np.log(2)
154
155     res[mask] = np.exp(log_numerator - log_denominator)
156     return res
157
158 def char_func_prod(self, t):
159     # Standardizing factors
160     scale1 = np.sqrt((self.v1 - 2) / self.v1) if self.v1 > 2 else
161         1.0
162     scale2 = np.sqrt((self.v2 - 2) / self.v2) if self.v2 > 2 else
163         1.0
164
165     # Apply scales to the characteristic functions
166     phi1 = self.t_characteristic_function(self.v1, t * np.sqrt(self.
167         rho) * scale1)
168     phi2 = self.t_characteristic_function(self.v2, t * np.sqrt(1 -
169         self.rho) * scale2)
170
171     return phi1 * phi2
172
173 def recover_all(self, N=2**16, L=15):
174     dx = 2 * L / N
175     x = np.linspace(-L, L, N, endpoint=False)
176
177     dt = 2 * np.pi / (2 * L)
178     t = np.fft.ifftshift(np.arange(-N/2, N/2)) * dt
179
180     phi = self.char_func_prod(t)
181
182     density = np.fft.ifft(phi) * (N * dt / (2 * np.pi))
183     density = np.real(np.fft.ifftshift(density))
184
185     mean = np.trapezoid(density * x, x)

```

```

182     var = np.trapezoid(density * (x**2), x) - mean**2
183     cdf = np.cumsum(density) * dx
184
185     return x, density, cdf, mean, var
186
187     def quantile_func(self,q):
188         x, density, cdf, mean, var = self.recover_all()
189         inv_cdf = interp1d(cdf, x, bounds_error=False, fill_value='
extrapolate')
190         return inv_cdf(q)
191
192     def cdf_function(self,x_input):
193         x_grid, density, cdf_vals, mean, var = self.recover_all()
194         cdf_interp = interp1d(x_grid, cdf_vals, bounds_error=False,
fill_value='extrapolate')
195         return cdf_interp(x_input)
196
197
198     def integrate_gauss_hermite(self,A,B,func):
199         nodes = self.nodes
200         weights = self.weights
201         nu = self.v1
202
203         mu = self.calc_mode(A,B,func)
204         sigma_lp = self.calc_obs_inf(A,B,mu,func)
205         if sigma_lp <= 1e-10:
206             sigma_lp = 1.0 # Default scale
207             mu = 0.0 # Default center
208
209         t_nodes = mu + np.sqrt(2)* sigma_lp*nodes
210         total = 0.0
211         for i in range(self.n_herm):
212             g = func(A,B,t_nodes[i]) * t.pdf(t_nodes[i],df = nu)
213             gaussian_weight = weights[i] * np.exp(nodes[i]**2)
214             total += g * gaussian_weight
215
216         return np.sqrt(2) * sigma_lp * total
217
218     def calc_mode(self, A, B, func):
219         def f(x):
220             val = func(A, B, x) * t.pdf(x, self.v1)
221             return -val if val > 0 else 0.0
222         best_mode = 0
223         min_val = f(0)
224         for start in [-2, 0, 2]:
225             res = minimize(f, x0=start, bounds=[(-10, 10)])
226             if res.fun < min_val:
227                 min_val = res.fun
228                 best_mode = res.x.item()
229         return best_mode
230
231     def calc_obs_inf(self, A,B, mode, func):
232         def log_g(x):
233             val = func(A, B, x)
234             pdf = t.pdf(x, df=self.v1)
235
236             val = max(val, 1e-300)
237             pdf = max(pdf, 1e-300)
238
239             return np.log(val) + np.log(pdf)
240
241         def second_derivative(f, x, h=1e-4):
242             return -(f(x + 2*h) + f(x - 2*h) - 2*f(x)) / (4*h**2)

```

```

243     j = second_derivative(log_g, mode)
244
245     if j > 0:
246         return 1 / np.sqrt(j)
247     else:
248         def second_derivative(f, x, h=1e-3):
249             return -(f(x + h) + f(x - h) - 2*f(x)) / (h**2)
250         j = second_derivative(log_g, mode)
251         if j > 0:
252             return 1 / np.sqrt(j)
253         else:
254             return 0.0
255
256
257
258
259 def calc_expected_tranche_principal(self, A, B, S, k):
260     S = np.asarray(S).item()
261     S = float(S)
262     barrier = self.default_barriers[k]
263
264     # Standardizing factors
265     scale1 = np.sqrt((self.v1 - 2) / self.v1) if self.v1 > 2 else
266         1.0
267     scale2 = np.sqrt((self.v2 - 2) / self.v2) if self.v2 > 2 else
268         1.0
269
270     sqrt_rho = np.sqrt(self.rho)
271     sqrt_1mr = np.sqrt(1 - self.rho)
272
273     numerator = barrier - (sqrt_rho * S * scale1)
274     denominator = sqrt_1mr * scale2
275
276     p = t.cdf(numerator / denominator, df=self.v2)
277     p = min(max(p, 1e-16), 1 - 1e-16)
278
279     M = self.M
280     R = self.R
281
282     nl = A * M / (1 - R)
283     nh = B * M / (1 - R)
284
285     il = min(math.ceil(nl), M+1)
286     ih = min(math.ceil(nh), M+1)
287
288     # compute binomial probabilities recursively
289     probs = np.zeros(M+1)
290     probs[0] = (1 - p)**M
291
292     ratio = p/(1-p)
293
294     for n in range(M):
295         probs[n+1] = probs[n] * (M-n)/(n+1) * ratio
296
297     sum1 = probs[:il].sum()
298     sum2 = 0.0
299     for j in range(il, ih):
300         loss_fraction = (B - j*(1-R)/M)/(B-A)
301         sum2 += probs[j] * loss_fraction
302
303     return sum1 + sum2

```

```

304 def plot_expected_tranche_principal(self,A,B):
305     N = self.N
306     def integrand(A,B,S):
307         return self.calc_expected_tranche_principal(A, B, S, self.N)
308     yvals = []
309     xvals = []
310     rho = np.linspace(0,1,100)
311     old_rho = self.rho
312     for i in rho:
313         self.rho = i
314         yvals.append(self.integrate_gauss_hermite(A,B,integrand))
315         xvals.append(self.rho)
316
317     plt.plot(xvals,yvals)
318     plt.show()
319
320 def calc_PV_regular_spread_payments(self,A,B,S): #A in Hull
321     delta = self.delta # assume equidistant payments
322     N = self.N
323     sum = 0
324     for k in range(1, N+1):
325         sum += delta * self.calc_expected_tranche_principal(A,B,S,k)
326         * self.zcb[2*k]
327     return sum
328
329 def calc_accrual_payment(self,A,B,S): #B in Hull
330     delta = self.delta # assume equidistant payments
331     N = self.N
332     sum = 0
333     for k in range(1, N+1):
334         sum += 0.5*delta*(self.calc_expected_tranche_principal(A,B,S
335             ,(k-1))-self.calc_expected_tranche_principal(A,B,S,k))*
336             self.zcb[2*k-1]
337     return sum
338
339 def calc_PV_expected_payoffs(self,A,B,S): #C in Hull
340     N = self.N
341     sum = 0
342     for k in range(1, N+1):
343         sum += (self.calc_expected_tranche_principal(A,B,S,k-1)-self
344             .calc_expected_tranche_principal(A,B,S,k))*self.zcb[2*k
345             -1]
346     return sum
347
348 def calc_fair_spread_and_upfront(self,A,B):
349     fair_spread = float((self.integrate_gauss_hermite(A,B,self.
350         calc_PV_expected_payoffs))/(self.integrate_gauss_hermite(A,B,
351         self.calc_PV_regular_spread_payments)+self.
352         integrate_gauss_hermite(A,B,self.calc_accrual_payment)))
353     if fair_spread > 0.01:
354         spread = 0.01
355         upfront = float(self.integrate_gauss_hermite(A,B,self.
356             calc_PV_expected_payoffs)-spread*(self.
357             integrate_gauss_hermite(A,B,self.
358             calc_PV_regular_spread_payments) + self.
359             integrate_gauss_hermite(A,B,self.calc_accrual_payment)))
360     return spread,upfront
361 else:
362     return fair_spread,0
363

```

```

355     def tranche_spread_for_rho(self, A, B, rho):
356
357         old_rho = self.rho
358         self.rho = rho
359
360         C = self.integrate_gauss_hermite(A,B,self.
            calc_PV_expected_payoffs)
361         P = (
362             self.integrate_gauss_hermite(A,B,self.
            calc_PV_regular_spread_payments)
363             + self.integrate_gauss_hermite(A,B,self.calc_accrual_payment
            )
364         )
365
366         self.rho = old_rho
367
368         return C / P
369
370     def calc_compound_correlation(self, A, B, s, U):
371         def objective(rho):
372             def f(rho):
373                 old_rho = self.rho
374                 self.rho = rho
375
376                 C = self.integrate_gauss_hermite(A,B,self.
            calc_PV_expected_payoffs)
377                 P = (
378                     self.integrate_gauss_hermite(A,B,self.
            calc_PV_regular_spread_payments)
379                     + self.integrate_gauss_hermite(A,B,self.
            calc_accrual_payment)
380                 )
381
382                 self.rho = old_rho
383
384                 return U + s * P - C
385             return (f(rho))**2
386
387         res = minimize_scalar(objective, bounds=(0.0001,0.9999), method=
            'bounded')
388         return res.x
389
390     def calc_base_correlation(self, target_aq, market_quotes):
391         total_loss_0_aq = 0.0
392
393         for (A, B, s, U) in market_quotes:
394             if B > target_aq:
395                 break
396
397             comp_rho = self.calc_compound_correlation(A, B, s, U)
398
399             old_rho = self.rho
400             self.rho = comp_rho
401
402             C_q_absolute = self.integrate_gauss_hermite(A,B,self.
            calc_PV_expected_payoffs) * (B - A)
403             total_loss_0_aq += C_q_absolute
404
405             self.rho = old_rho
406
407         def objective(rho):
408             old_rho = self.rho
409             self.rho = rho

```

```
410 |         model_loss = self.integrate_gauss_hermite(0,target_aq,self.  
411 |             calc_PV_expected_payoffs) * target_aq  
412 |         self.rho = old_rho  
413 |         return model_loss - total_loss_0_aq  
414 |     return brentq(objective, 0.0001, 0.9999)
```